Focus-app — Brutal Audit

Scope: the whole app as it stands on main @ 8438f5a. 50 independent audit slices, read-only, every finding backed by a file/line the auditor actually read.

Totals: 251 findings — 45 critical, 118 high, 80 medium, 8 low.

The short version

• CI has never built anything. 5/5 main runs failed; HEAD is blocked by a gitignored debug.keystore the build demands. The only 'release' is a manually built debug APK mislabelled as a full release.
• Release signing is a trap. No upload keystore exists in the repo, and my-upload-key.jks at its default path is NOT git-ignored — one git add . after generating it commits the private Play upload key.
• The 'cryptographic integrity' is theatre. A blank HMAC signature is accepted as valid, and there is a hard-coded, publicly known fallback HMAC key with a silent downgrade to it. Anyone can forge a 'valid' receipt.
• The app shows the user invented numbers. ViewModel flows ship fabricated defaults (fake guard integrity, a focus total coerced to at least 35 with an initial 75, hard-coded tasks when the list is empty), and estimatedMinutes = 25 contradicts its own documented '0 = no estimate given' invariant.
• The blocking core is unreliable. Blocking is 2-second polling over a UsageStats lastTimeUsed guess; one exception in the monitor loop silently kills blocking while the service still claims to protect; onNewIntent drops re-intercepts while the activity is alive.
• Data-loss landmines. The app-catalogue prune wipes the entire catalogue irreversibly if the installed-apps list comes back empty or partial; the Room migration chain starts at v7, so every pre-v7 install crashes on upgrade; the database singleton's double-checked locking is broken.
• Privacy exposure. Other apps' notification titles and bodies are persisted indefinitely, unencrypted, in a device-protected (pre-unlock) database.
• Structure is decaying. Repository is 2,932 lines, ViewModel 2,548, BlockitCoreViews 2,416 — with circular data<->service coupling.
• Some tests are theatre. ScreenshotHarnessTest is a screenshot factory where most tests assert nothing.
• Docs lie. The .aistudio satisfaction matrix stamps VERIFIED/PASSED on files and APIs that do not exist, and the feature catalogue presents a deleted feature universe as shipped.
• Real users are locked out twice over: zero screen-reader semantics in the entire UI, and six separate special-access permission trips through Settings before any value.

Recommended fix order (safest first)

1. Git-ignore .jks / .keystore (one line, zero behaviour change) and fix CI debug signing so CI can build at all (#3).
2. CI hygiene: run tests before uploading any APK, add paths filter, Gradle cache, concurrency and a timeout (#3).
3. Make the unit tests compile and actually assert things (#2) — a candidate local fix already exists on a parked branch; verify it, don't trust it.
4. Guard the catalogue prune against empty/partial installed lists (small change, kills a data-loss path).
5. Fix the database singleton locking and the estimatedMinutes default (tiny, surgical).

6. Handle intercept intents in onNewIntent (small, restores re-intercepts).
7. Then the decisions only Sarthak can make: the HMAC/integrity design (blank signatures + public fallback key), notification-content storage, and the blocking mechanism (polling vs real foreground detection). These change what the app is — no agent should guess them.

Full findings by slice

build-gradle

Verdict: This is an untouched AI Studio template build — broken/leaky release signing, no release optimization, unverified Gradle distribution, and an incoherent version matrix — not a production build configuration.

• [CRITICAL] Release signing is broken and the upload keystore is not git-ignored
  - Location: app/build.gradle.kts:28-33; .gitignore:18
  - Evidence: create("release") defaults to val keystorePath = System.getenv("KEYSTORE_PATH") ?: "${rootDir}/my-upload-key.jks" with storePassword/keyPassword from env (nullable, no fail-fast), but no .jks/my-upload-key.jks exists in the repo (ls: cannot access '.jks'), and root .gitignore line 18 covers only 'debug.keystore' — not '*.jks' or 'my-upload-key.jks' — so the upload key the config expects at repo root is both absent for a release build and unignored if dropped there, risking accidental commit and Play signing compromise.

• [HIGH] Release build has minification, shrinking and PNG crunching all disabled
  - Location: app/build.gradle.kts:44-47; app/proguard-rules.pro:1-21
  - Evidence: release { isCrunchPngs=false; isMinifyEnabled=false; proguardFiles(...) } with no isShrinkResources, and app/proguard-rules.pro is the untouched stock template (only commented examples). The listed ProGuard files are dead config: release ships unshrunk, unobfuscated code/resources.

• [HIGH] Gradle wrapper performs no distribution integrity verification
  - Location: gradle/wrapper/gradle-wrapper.properties:3-5
  - Evidence: distributionUrl=https://services.gradle.org/distributions/gradle-9.3.1-bin.zip, validateDistributionUrl=false, and no distributionSha256Sum entry at all. A tampered/corrupted Gradle distribution would be accepted without checksum verification.

• [HIGH] Version matrix is incoherent: bleeding-edge AGP/Gradle/Kotlin vs stale Compose BOM and Java 11
  - Location: gradle/libs.versions.toml:2,9-10; app/build.gradle.kts:13,58-61
  - Evidence: agp="9.1.1", kotlin="2.2.10", gradle-9.3.1 and compileSdk release(36){minorApiLevel=1} sit alongside composeBom="2024.09.00" (Sep 2024), lifecycle 2.8.7, activityCompose 1.10.1, and compileOptions VERSION_11 (app/build.gradle.kts:58-61) while CI installs JDK 17. Bleeding-edge toolchain pinned against a year-plus-older Compose BOM is an untested compatibility matrix, not a coherent version set.

• [MEDIUM] App identity is still AI Studio template junk and versioning is frozen
  - Location: app/build.gradle.kts:12,16,19-20; settings.gradle.kts:25
  - Evidence: namespace="com.example", applicationId="com.aistudio.monolith.kxtpzr", versionCode=1, versionName="1.0", and settings.gradle.kts:25 rootProject.name="Monolith" — none identify Focus-app.

With versionCode frozen at 1, Play cannot accept a second upload, and the generic namespace leaks into generated code/manifest identity.

ci-workflow

Verdict: This is a rescue script, not CI: it uploads an untested debug APK on every unfiltered push, with no PR gate, no cache, no timeout, and no release/lint/instrumented coverage.

• [CRITICAL] Untested debug APK is uploaded before unit tests run
  - Location: .github/workflows/rescue-build.yml:23-29
  - Evidence: Artifact step precedes tests: lines 23-27 'uses: actions/upload-artifact@v4' with 'name: monolith-debug'/'retention-days: 3', then lines 28-29 '- name: Unit tests' 'run: ./gradlew --console=plain testDebugUnitTest'. A build that then fails its ~43 unit tests still leaves a downloadable, apparently-good APK in Artifacts.

• [HIGH] No pull_request trigger, no branch/path filter — CI does not gate merges
  - Location: .github/workflows/rescue-build.yml:2-4
  - Evidence: Lines 2-4: 'on: workflowdispatch:' and bare 'push:' with no branches, paths, or pullrequest key (grep for pull_request/branches in .github/ returns no match). Every push to any branch burns a full run, while a PR gets no dedicated check/status gate before merge.

• [HIGH] Proves only debug builds — release, lint and instrumented tests never run
  - Location: .github/workflows/rescue-build.yml:19-29
  - Evidence: Only proof offered is lines 19-22 'Assemble debug' / './gradlew --console=plain assembleDebug' and line 29 testDebugUnitTest. No assembleRelease, lint, or connectedAndroidTest anywhere in the 29-line file, so the release signingConfig in app/build.gradle.kts (env KEYSTOREPATH/STOREPASSWORD/KEY_PASSWORD), R8/proguard config, and the sole androidTest are never exercised.

• [HIGH] No caching, no timeout, no concurrency — maximum wasted minutes per run
  - Location: .github/workflows/rescue-build.yml:5-18
  - Evidence: No 'timeout-minutes', 'concurrency', 'cache', or 'permissions' anywhere (grep returns no match). setup-java at lines 10-13 has no 'cache: gradle', and line 18 reinstalls '"platforms;android-36.1" "build-tools;36.1.0"' every run, so each run re-downloads Gradle 9.3.1, all dependencies and the SDK from zero, and a hung Gradle job defaults to GitHub's 360-minute limit.

• [MEDIUM] Fragile, unpinned and failure-swallowing SDK setup
  - Location: .github/workflows/rescue-build.yml:7-18
  - Evidence: Line 17: 'yes | "$SDKMANAGER" --licenses > /dev/null 2>&1 || true' discards all output and swallows failure; line 16 hardcodes '$ANDROID_HOME/cmdline-tools/latest/bin/sdkmanager'; lines 7,9,10,23 use floating 'ubuntu-latest' and mutable '@v4' tags, not SHAs. Git history confirms the fragility: 'run #2 failed: sdkmanager not on PATH' and 'CI run #3 died on chmod gradlew'.

signing-secrets

Verdict: Signing hygiene is negligent: the release upload key would be committed by a routine git add because only debug.keystore is ignored, a forgeable HMAC secret is hardcoded in source, and no keystore, generation doc or CI provisioning exists to make a signed build reproducible.

• [CRITICAL] Release upload keystore at its default path is not gitignored
  - Location: .gitignore:18; app/build.gradle.kts:29

  - Evidence: .gitignore:18 contains only the bare name 'debug.keystore'; app/build.gradle.kts:29 defaults release storeFile to "${rootDir}/my-upload-key.jks". Verified with git check-ignore: 'my-upload-key.jks', 'release.jks', 'upload.jks' and 'release.keystore' are all NOT-IGNORED. One 'git add .' after generating the upload key at its default path commits the private Play upload key.

• [HIGH] Hardcoded deterministic HMAC secret committed in source
  - Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:27-31
  - Evidence: Private val at lines 28-31: SecretKeySpec("MonolithHardwareHMACFallback32B!".toByteArray(UTF8), HMACALGO), commented lines 27-28 'Deterministic 256-bit fallback keys used when AndroidKeyStore hardware TEE is unavailable'. Used at lines 69,72,78 whenever the hardware key fails. Anyone with the public repo can forge valid session HMACs on the fallback path, defeating the tamper-detection this class claims to provide.

• [HIGH] .gitignore secrets coverage is single-name theatre, not patterns
  - Location: .gitignore:17-18; app/.gitignore:1
  - Evidence: .gitignore has only '.env' (line 17) and 'debug.keystore' (line 18) for secrets. Verified git check-ignore returns NOT-IGNORED for: .env.local, .env.production, google-services.json, app/google-services.json, keystore.properties, key.properties, credentials.json, service-account.json, .pem and .p12. No '.jks', '.keystore', '.key' or 'keystore.properties' pattern exists in .gitignore or app/.gitignore (app/.gitignore is only '/build').

• [MEDIUM] Secrets plugin defaults to the tracked .env.example, inviting a committed, APK-packaged API key
  - Location: app/build.gradle.kts:73-78; .env.example:5-7
  - Evidence: app/build.gradle.kts:75-78: secrets { propertiesFileName='.env'; defaultPropertiesFileName='.env.example' } so the tracked example file supplies values baked into the build when .env is absent. .env.example:5-7 instructs 'Uncomment the line below if your app uses the Gemini API' above '# GEMINIAPIKEY=<placeholder>' — following its own instructions puts a real key in a tracked file and packages it in the APK. Current value is only a commented placeholder (verified: no AIza/gh/sk- key in tracked tree).

• [MEDIUM] Signing is env-only, undocumented and unprovisioned — no reproducible release (or clean-clone debug) build
  - Location: app/build.gradle.kts:28-40; .github/workflows/rescue-build.yml:19-22
  - Evidence: app/build.gradle.kts:28-40: release storePassword=System.getenv("STOREPASSWORD") and keyPassword=System.getenv("KEYPASSWORD") are nullable with no validation/fail-fast, alias hardcoded 'upload', and debugConfig requires file("${rootDir}/debug.keystore"). No .jks/.keystore exists on disk or in git history (find and git log --all -- 'keystore' empty) and git grep finds no keytool/generation instructions. .github/workflows/rescue-build.yml:19-22 runs only assembleDebug and provisions neither keystore nor any signing env var.

manifest-permissions

Verdict: The backup lockdown (allowBackup=false plus total excludes in both rules files) and the lean, INTERNET-free permission set are genuinely tight, but the privilege declarations lie to the user: the device-admin consent screen wears the accessibility service's description, the accessibility disclosure understates an all-apps window-content scraper, and a never-used limit-password policy is requested as uninstall friction.

• [HIGH] Device-admin consent screen shows the accessibility service's description — misleading consent for an uninstall guard
  - Location: app/src/main/AndroidManifest.xml:123
  - Evidence: MonolithDeviceAdminReceiver declares android:description="@string/accessibilityservicedescription", which strings.xml:4 resolves to 'Detects when a blocked app is launched during an active Monolith focus session and presents the Intention Airlock screen.' — an accessibility description shown on the device-admin activation/consent screen for an uninstall guard. The user is told the wrong thing about the powerful privilege they are granting.

• [HIGH] Accessibility service can read the content of every window in every app, while its disclosure claims it only detects blocked-app launches
  - Location: app/src/main/res/xml/accessibilityserviceconfig.xml:2-8
  - Evidence: Config sets android:canRetrieveWindowContent="true", flags flagIncludeNotImportantViews|flagReportViewIds, event types typeWindowStateChanged|typeWindowContentChanged, notificationTimeout=80, and no android:packageNames restriction — i.e. content scraping of every app's windows (the manifest itself, line 77, calls it a 'URL Bar Interceptor'). Yet the user-facing description (strings.xml:4) claims it merely 'Detects when a blocked app is launched... and presents the Intention Airlock screen.' The declared capability vastly exceeds the disclosed purpose.

• [MEDIUM] Device-admin policy declares limit-password, a policy the code never exercises — pure overreach
  - Location: app/src/main/res/xml/deviceadminreceiver.xml:4
  - Evidence: deviceadminreceiver.xml's sole policy is <limit-password />. A grep of the codebase shows DevicePolicyManager is used exactly once — MonolithRepository.kt:708-711 — and only for dpm.isAdminActive(adminComponent); the receiver (MonolithDeviceAdminReceiver.kt) only overrides onEnabled/onDisabled/onDisableRequested. No password policy is ever set or enforced, so the app requests a password-control policy it never uses, purely as uninstall friction.

• [MEDIUM] Battery-exemption prompt launched without declaring REQUESTIGNOREBATTERY_OPTIMIZATIONS
  - Location: app/src/main/AndroidManifest.xml:6-13
  - Evidence: MainActivity.kt (WatchItemId.BATTERY branch, ~line 392) fires Intent(Settings.ACTIONREQUESTIGNOREBATTERYOPTIMIZATIONS, Uri.parse("package:...")), an intent Android restricts to holders of REQUESTIGNOREBATTERYOPTIMIZATIONS — but the manifest's entire permission list (AndroidManifest.xml:6-13: VIBRATE, FOREGROUNDSERVICE, FOREGROUNDSERVICESPECIALUSE, POSTNOTIFICATIONS, RECEIVEBOOTCOMPLETED, SYSTEMALERTWINDOW, PACKAGEUSAGESTATS) omits it, leaving the direct prompt reliant on the runCatching fallback to the generic settings screen.

• [LOW] MainActivity is directBootAware for no stated reason, exposing the launcher UI before first unlock
  - Location: app/src/main/AndroidManifest.xml:41
  - Evidence: MainActivity — a full launcher UI — is declared android:directBootAware="true" alongside the boot receiver and services, with no comment or rationale anywhere in the manifest (contrast the boot receiver at lines 93-106, where pre-unlock delivery is the point). This exposes the activity to launch before first unlock, when only device-protected storage is available, for no documented benefit.

main-activity

Verdict: MainActivity is a fragile single point of failure: its core intercept intent is dropped on re-delivery, one rotation permanently kills audio and leaks engines, Back escapes its own lockdown overlays, and its error handling is to silently do nothing.

• [CRITICAL] onNewIntent drops intercept intents — re-intercepts while activity is alive are silently lost
  - Location: app/src/main/java/com/example/MainActivity.kt:99-124
  - Evidence: Intercept handling exists only in setContent: 'LaunchedEffect(Unit) { handleIncomingInterceptIntent(intent, viewModel) }' (lines 99-100), which never re-runs for a new intent. onNewIntent (lines 121-124) is just 'super.onNewIntent(intent); setIntent(intent)' — it never calls handleIncomingInterceptIntent and setIntent is not Compose state, so nothing recomposes. Yet MonolithShieldService.kt:181-193 re-fires startActivity with FLAGACTIVITYNEWTASK|FLAGACTIVITYSINGLETOP and EXTRAINTERCEPTEDPACKAGE every poll, and AndroidManifest.xml:43 sets launchMode="singleTop", so a re-intercept of an already-running/foreground MainActivity is delivered to onNewIntent and silently discarded — the Airlock never triggers.

• [CRITICAL] Rotation kills the ViewModel's sound engine and orphans a new threaded engine on every recreation
  - Location: app/src/main/java/com/example/MainActivity.kt:85-93,145-152
  - Evidence: onCreate unconditionally builds a new engine: 'soundEngine = TactileSoundEngine(applicationContext)' (line 85) and passes it only via the ViewModel factory (lines 89-93). A retained ViewModel ignores that factory after a configuration change, so the new engine is orphaned while every UI call site uses the old 'viewModel.soundEngine' (e.g. lines 702, 709). Worse, onDestroy (lines 145-152) calls stopAllAudio() on the Activity's own field, and TactileSoundEngine.stopAllAudio() (TactileSoundEngine.kt:847-857) sets 'running = false' and releases sealTrack with no restart path, while its init (lines 327-341) starts two threads per instance. One rotation therefore permanently kills the ViewModel's synth/seal threads (all sound/haptic feedback dead until process death) and leaks the replacement engine's two idle threads.

• [HIGH] No Back handling for any full-screen overlay — Back finishes the Activity, including from the Airlock intercept
  - Location: app/src/main/java/com/example/MainActivity.kt:347-351
  - Evidence: The only BackHandlers in the entire codebase are MainActivity.kt:347-351: 'if (parkOpen) BackHandler { closePark(...) } else if (currentDestination != DECK) BackHandler { navigateTo(DECK) }'. There is no handler for the full-screen AirlockInterceptOverlay, DisarmChallengeOverlay, AlwaysOnDeskClockOverlay or CapturedNotificationsDigestSheet rendered unconditionally at the end of MonolithRootApp (lines ~762-821), and grep finds no BackHandler in any overlay component. On DECK with any of them visible, system Back falls through to the default dispatcher and finishes the Activity instead of dismissing/answering the overlay — for the Airlock that means Back is an escape from the intercept screen back toward the blocked app (until the shield service re-fires).

• [MEDIUM] PARK/receipt host state is remember-only and evaporates on recreation
  - Location: app/src/main/java/com/example/MainActivity.kt:268-277
  - Evidence: Host UI state is plain remember, not rememberSaveable, and the manifest declares no android:configChanges: 'var dismissedReceiptSessionId by remember { mutableStateOf(-1) }' (line 268), 'var parkOpen by remember { mutableStateOf(false) }' and 'var parkReturnTick by remember {

mutableStateOf(0) }' (lines 276-277). Any rotation or activity recreation resets them: PARK snaps shut mid-diagnosis, an already-dismissed receipt slip reappears (its visibility test at lines ~790-800 compares against dismissedReceiptSessionId), and the return-tick diagnostic restarts.

• [MEDIUM] FIX-pane navigation swallows all failures silently — a broken Settings intent leaves the user stranded with zero feedback
  - Location: app/src/main/java/com/example/MainActivity.kt:362-410
  - Evidence: openFixPane wraps everything in a bare 'runCatching { when(item){ ... context.startActivity(...) ... } }' (lines 362-363) with no getOrElse/fallback/logging for ACCESSIBILITY, USAGEACCESS, NOTIFICATIONLISTENER or OVERLAY (only DEVICEADMIN and BATTERY have inner fallbacks, lines 376-378, 397-400), and 'else -> Unit' silently ignores unknown items. The Deck's own fix callbacks are the same pattern: 'runCatching { context.startActivity(Intent(Settings.ACTIONACCESSIBILITY_SETTINGS)) }' (lines 540, 543). On any OEM where one of these Settings actions throws ActivityNotFoundException, tapping FIX — the only route to grant the permissions the blocker depends on — does literally nothing, with no error, fallback pane, or user feedback.

viewmodel

Verdict: A 2,548-line god ViewModel that fabricates default stats, races its own dial and finish state, busy-polls services, and leaves its most dangerous orchestration — ticker, seal freeze and finish machine — essentially untested.

• [CRITICAL] State flows ship fabricated defaults: fake guard integrity, fake focus totals, fake tasks
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:393-404,502,343-350
  - Evidence: totalFocusedMinutes ends '.coerceAtLeast(35)' with stateIn initial 75 (line 502/504); _systemGuardStatus is pre-filled with invented facts 'isMonotonicClockIntact = true, isHmacSignatureValid = true, systemBootCount = 1, installedLauncherAppsCount = 12' before any check (lines 393-404); checklist invents 3 hard-coded tasks when the real list is empty (lines 343-350). UI can print security/focus numbers that were never measured.

• [HIGH] extendActiveTaskOvertime double-writes _dialMinutes in a race between sync and async paths
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:1814-1824
  - Evidence: extendActiveTaskOvertime launches async work that sets 'dialMinutes.value = updated.estimatedMinutes' (lines 1818-1821) and then synchronously also executes 'dialMinutes.value = (_dialMinutes.value + extraMinutes).coerceIn(5, 180)' (line 1823). Whichever lands last wins, so the dial can double-apply the extension or be silently overwritten by the repository result depending on timing.

• [HIGH] God ViewModel: 2,548 lines, dozens of concurrent collectors, core orchestration effectively untested
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:204-2548
  - Evidence: File is 2548 lines with 73 viewModelScope.launch sites and an init block (lines ~750-945) spawning seed/sync, activeTask, WatchStore, activeSession (twice), armProving.phase, combine collectors, freeze-watch, ticker, finish controller, shot-ground driver and recovery concurrently. Tests found only exercise the extracted pure helpers evaluateTheme/peekOfferFor/peekSecondsRemaining and construct the VM for rendering; there is no direct test of the ticker, seal-freeze collectors, or finish controller that own most side effects.

• [HIGH] Finish/receipt handling races on a shared MutableSet and busy-polls the shield service every

25ms
  - Location:
app/src/main/java/com/example/ui/MonolithViewModel.kt:742,1169,1200-1213,1266-1303,1351-1357
  - Evidence: onFinishE0 adds the id to plain 'private val handledFinishIds = mutableSetOf<Int>()' (line 742, 1200), then its launched coroutine removes it on failure: 'handledFinishIds -= sessionId' (line 1213), while sessionHistory collector (line 1169) and publishReceipt() (lines 1351-1357) read/mutate the same unsynchronised set from other coroutines. The detention itself is a busy poll: 'while (true) { ... MonolithShieldService.instance == null ... delay(25L) }' (lines 1266-1303).

• [MEDIUM] totalQueueMinutes is stale-by-construction: reads _dialMinutes.value inside a tasks-only map
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:337-341
  - Evidence: 'val totalQueueMinutes: StateFlow<Int> = tasks.map { list -> ... .sumOf { it.estimatedMinutes }.coerceAtLeast(dialMinutes.value) }' (lines 337-341): dialMinutes is read as a one-off .value snapshot inside a map driven only by tasks, not combined with the dial flow, so changing the dial alone never recomputes this total until an unrelated task emission happens.

arm-proving

Verdict: The cascade's happy path is carefully built, but its failure paths are not: once the sampler hands off to frames and the seal coroutine every deadline dies with it, a single clock stall deletes the retry window entirely, a routine reconcile fires for sessions that never proved, and the tests are constructed so none of this can ever be caught.

• [HIGH] No watchdog after all-confirmed: frame wait and seal launch can strand the attempt in Proving forever
  - Location: app/src/main/java/com/example/ui/ArmProvingController.kt:497-500,556-610
  - Evidence: runAttempt returns immediately after declareT0 (lines 497-500: 'if (sampled.allConfirmed()) { declareT0(attemptId); return }'), so no deadline/annul check ever runs again. postT0Frame then re-posts itself forever while waiting out the dwell (lines 580-583: 'if (frameMono < candidate) { postT0Frame(attemptId, candidate); return@framePoster }') with no timeout, and the seal at lines 591-610 is 'scope.launch { val sealedOk = transition.executeSeal(...) }' with no try/catch and no timeout — the production transition only runCatching's sealRow, while writeSealMarker/publish/playSealThrow are unguarded (SessionSealTransition.kt executeSeal). If the frame callback never fires, or executeSeal throws or hangs, the phase stays Proving forever, onLatch early-returns for Proving/Jammed (lines 306-307), and _annulCount is never incremented — the exact stranded-latch harm the file header claims is 'closed by construction'.

• [HIGH] Retry window collapses to zero on a single clock jump: jam and annul fire in the same sampler iteration
  - Location: app/src/main/java/com/example/ui/ArmProvingController.kt:502-536
  - Evidence: In runAttempt, 'val now = clock()' is read once (line 502), the Proving->Jammed transition fires at lines 503-514 using that now (publishing Jammed, playing jam haptics, re-dispatching the shield), and the very next check at lines 526-533 reuses the SAME now: 'if (afterJam is ArmPhase.Jammed && now >= latchMono + annulMs && ...) { annul(attemptId); return }'. The sampler only delays at the loop bottom (line 536), so any >1s stall between iterations — the controller runs on viewModelScope (Main) in MonolithViewModel.kt:695 — goes Proving->Jammed->annulled in a single iteration: Jammed is never rendered for a frame, the 'retry' shield start and haptics fire for an attempt already being annulled, and the promised L+500..L+1500 retry window is zero. Tests only tick

stepwise (l+600 then l+1600), so this jump case is untested.

• [MEDIUM] SESSION_EDGE reconcile fires even when L1 never proved — and even for a dead attempt
  - Location: app/src/main/java/com/example/ui/ArmProvingController.kt:363-380
  - Evidence: The port contract at lines 167-168 says onSessionStartedEdge is 'The routines SESSIONEDGE reconcile, fired once L1 lands.' But onSessionRowInserted calls it unconditionally at line 379, OUTSIDE the 'if (snapshotId == sessionId)' block at lines 375-378 that actually proves L1, and there is no attemptId re-check after the suspending snapshot read at line 374 — applyMark would no-op for a dead attempt, the edge does not. So the RoutineHeartbeat SESSIONEDGE reconcile fires when the snapshot returned null or a different row's id (L1 never landed), and even if the attempt was jammed/annulled while the snapshot read was in flight.

• [MEDIUM] Seal landed 'through the other door' is published with this attempt's invented t0, not the real seal time
  - Location: app/src/main/java/com/example/ui/ArmProvingController.kt:591-608
  - Evidence: Lines 599-605: 'if (transition.isSealed(sessionId)) { // The seal landed through the other door ... _phase.value = ArmPhase.SealedRunning(sessionId = sessionId, t0Mono = t0) }' — t0 here is THIS attempt's candidate (maxOf(candidate, frameMono), line 585), not the t0 the routine door actually sealed and re-anchored the row with via its own sealSession(sessionId, itsT0). The controller publishes a SealedRunning whose t0Mono can differ arbitrarily from the row's real anchors, so any surface reading phase.t0Mono as the session's t=0 is being shown a fabricated time for a seal this attempt did not strike.

• [MEDIUM] Tests fake away the session-keying they exist to prove, and cover none of the lethal edges
  - Location: app/src/test/java/com/example/ArmProvingControllerTest.kt:140-145
  - Evidence: FakeWitnesses (test lines 140-145) ignores its argument entirely: 'override fun shieldProtectingMono(sessionId: Int): Long? = shieldMono' and likewise for a11y — so a controller bug that accepted a witness stamp for the WRONG session id would pass every test, despite ArmLink's definitions requiring the shield/a11y witnesses be armed 'for this row' (controller lines 44-61, LiveArmWitnesses keying at 216-235). The suite also has no test for a frame that never arrives, an executeSeal that throws, a single tick jumping past both deadlines, or the SESSION_EDGE firing on a failed snapshot — i.e. none of the failure edges above can be caught by these tests; all 11 tests advance time in cooperative steps through fakes that cannot express the dangerous cases.

dao

Verdict: The DAO is a flat list of mostly sound, fully-bound queries with zero indices, zero @Transaction composites, zero row-count returns and several unbounded SELECT * reads on tables designed to grow forever — correctness and performance both rest on caller discipline and small data, neither of which is enforced here.

• [HIGH] Zero secondary indices in the entire schema — every hot filter/sort is a full table scan
  - Location: app/src/main/java/com/example/data/MonolithDao.kt:206,308,323; app/src/main/java/com/example/data/MonolithModels.kt:100,267,442,451
  - Evidence: Every entity is bare '@Entity(tableName=...)' with only a @PrimaryKey — MonolithModels.kt:100 focustasks, :267 focussessions, :442 emergencybreakspends, :451 interceptlogs — and grep finds zero 'indices=', '@Index' or 'CREATE INDEX' anywhere in

app/src/main/java or any exported schema. The DAO then filters/sorts on exactly those unindexed columns: focustasks 'status = COMPLETED AND completedAtMillis >= :fromMillis ... ORDER BY completedAtMillis DESC' (MonolithDao.kt:206), focussessions routineId/windowEndMillis/isActive/routineOutcome (308-310), interceptlogs timestampMillis range (323), spends weekAnchorMillis/sessionId (119,130).

• [HIGH] Unbounded SELECT * flows/snapshots on permanently growing tables
  - Location: app/src/main/java/com/example/data/MonolithDao.kt:69,77,136,140,158
  - Evidence: MonolithDao.kt:140-141 '@Query("SELECT FROM interceptlogs ORDER BY timestampMillis DESC") fun getInterceptLogsFlow(): Flow<List<InterceptLogEntity>>' has no LIMIT, and the DAO offers no delete/prune for interceptlogs at all (only insert at :143-144), so the table grows per blocked attempt and every insert re-emits the whole wide table. Same shape: getSessionHistoryFlow/Snapshot (69,77), getAllSpendsSnapshot (136) and the deliberately unbounded getCapturedNotificationsSnapshot (158) all return SELECT with no cap, while the Drawer header (192-199) declares COMPLETED focus_tasks retention 'permanent'.

• [HIGH] No @Transaction composites in the DAO — multi-write invariants depend purely on caller discipline
  - Location: app/src/main/java/com/example/data/MonolithDao.kt:122,213-220,268-276
  - Evidence: The file contains no '@Transaction' method at all. Its own comments admit the invariants live elsewhere: stampReceiptNumber (220) is 'written only inside the claim transaction' and writeReceiptClaim (268-276) updates guard_settings separately — two DAO calls whose atomic pairing, like deactivateAllSessions (122) + insertSession (80) for session start, is guaranteed only if every caller remembers database.withTransaction in the repository. Any direct DAO caller (test, future feature, ViewModel) silently breaks the receipt-number latch the comments claim 'the guarantee rests on' (213-220).

• [MEDIUM] All targeted UPDATEs return Unit (silent no-ops) and stampReleaseRetries is a lost-update-prone absolute write
  - Location: app/src/main/java/com/example/data/MonolithDao.kt:28,40,101,112,254-286
  - Evidence: Every @Query UPDATE returns Unit: setAppTier (28), updatePeekLeaseAndCooldown (31), latchShieldGap (101), stampReleaseFault (107-110), writeReceiptClaim and the guardsettings toggles (254-286, all 'WHERE id = 1'). If the target row is absent — e.g. guardsettings not yet seeded — the write affects 0 rows and the caller cannot tell; Room would expose this with an Int return. Worse, stampReleaseRetries (112) does 'SET releaseRetriesDone = :done' with an absolute caller-supplied value, while the same file proves it knows the safe pattern at :40 'interceptCount = interceptCount + 1' — concurrent retry writers read-modify-write the same stale count and lose increments.

• [MEDIUM] Drawer search: user input fed to LIKE unescaped — % and _ act as wildcards
  - Location: app/src/main/java/com/example/data/MonolithDao.kt:212
  - Evidence: MonolithDao.kt:212: "(title LIKE '%' || :q || '%' OR categoryTag LIKE '%' || :q || '%' OR notesOrSubsteps LIKE '%' || :q || '%')" with no ESCAPE clause and no escaping of :q in the DAO. A query of '%' or '' matches every stored task, and '' matches any single character, so search results are wrong for literal titles containing those characters. The leading '%' also makes the predicate non-sargable even if an index existed. (Classic SQL injection is absent — all parameters are Room-bound and there is no @RawQuery — this is pattern injection, not code injection.)

database

Verdict: The 7→12 migrations themselves are carefully documented, but the surrounding plumbing is reckless: a race-prone fake singleton, a migration chain that abandons every pre-v7 install to a crash loop, and exportSchema=false so none of it can be verified against a canonical schema.

• [CRITICAL] Broken double-checked locking: singleton can be built twice
  - Location: app/src/main/java/com/example/data/MonolithDatabase.kt:229-247
  - Evidence: getInstance is 'return INSTANCE ?: synchronized(this) { ... val instance = Room.databaseBuilder(...).build(); INSTANCE = instance; instance }' (lines 229-247) with @Volatile INSTANCE at 31-33 but NO second null-check inside synchronized. Two threads racing at boot (BootReceiver, AccessibilityService, ShieldService all call getInstance) can both see null, enter sequentially, build two Room instances for the same 'monolithtactiledb' file, second overwrites first; first is leaked, never closed, and callers hold different database objects.

• [CRITICAL] Migration chain starts at 7: every pre-v7 install crashes on upgrade
  - Location: app/src/main/java/com/example/data/MonolithDatabase.kt:25,242
  - Evidence: @Database(version = 12) at line 25, but addMigrations at line 242 registers only MIGRATION78, MIGRATION89, MIGRATION910, MIGRATION1011, MIGRATION1112. No 1->2 through 6->7 migration exists anywhere in the file, and comment at 222-228 boasts fallbackToDestructiveMigration is 'GONE'. Any install still on v1-v6 upgrading to v12 throws IllegalStateException 'A migration from X to 12 was required but not found' on first DB access — a hard crash loop, not preservation.

• [HIGH] exportSchema=false: migrations are unverifiable against any canonical schema
  - Location: app/src/main/java/com/example/data/MonolithDatabase.kt:26
  - Evidence: Line 26: exportSchema = false, and app/build.gradle.kts declares Room runtime/compiler (lines 101-102,136) with no schemaLocation argument; no app/schemas directory exists. The five hand-written ALTER/CREATE migrations therefore have no Room-generated schema JSON to diff or validate against, and no MigrationTestHelper/validateMigration usage exists in tests — tests hand-replicate old schemas, so a drift typo in a migration and in the test fixture cancels out undetected until Room's runtime identity-hash check fails on a real phone.

• [MEDIUM] Immortal singleton: no close/reset path ever
  - Location: app/src/main/java/com/example/data/MonolithDatabase.kt:31-33,244
  - Evidence: INSTANCE is assigned once at line 244 and never nulled or closed anywhere in the file — there is no close(), reset, or test hook. A closed/corrupted instance can never be replaced in-process, and tests calling MonolithDatabase.getInstance(context) (e.g. BreakTimerGuardTest, ReceiptClaimRepositoryTest) share the real on-disk singleton instead of an isolated database, making test state order-dependent and unrecoverable.

• [MEDIUM] No downgrade path: older build on a v12 database crashes
  - Location: app/src/main/java/com/example/data/MonolithDatabase.kt:242
  - Evidence: Only forward Migrations 7->8...11->12 are registered (line 242); there is no downgrade migration and no fallbackToDestructiveMigrationOnDowngrade. Opening a v12 database with any older build (rollback, sideload, backup restore onto an older APK) throws IllegalStateException instead of degrading or recovering, bricking the app until the newer build returns.

models

Verdict: The models document precise invariants in long comments and then enforce almost none of them — defaults contradict the comments, state is stringly-typed with silent fallbacks, and duplicated fields can diverge in the database.

• [CRITICAL] estimatedMinutes default directly contradicts its own documented invariant
  - Location: app/src/main/java/com/example/data/MonolithModels.kt:106-111
  - Evidence: Comment at lines 106-110: 'New tasks are stored with 0 = no estimate was ever given... surfaces print an estimate only where one was actually given (estimatedMinutes > 0)' — but line 111 declares 'val estimatedMinutes: Int = 25'. Any direct entity construction, copy(), or Room default therefore fabricates a 25m estimate that downstream >0 checks will print as if the user gave it.

• [HIGH] Zero validation: every entity accepts out-of-range times, masks and counts
  - Location: app/src/main/java/com/example/data/MonolithModels.kt:252-282,545-560
  - Evidence: No init{}/require/check exists in the file (grep finds none). FocusProfileEntity accepts 'startHour:Int, startMinute:Int, endHour:Int, endMinute:Int, defaultMinutes:Int' (lines 257-266), RoutineEntity accepts 'startMinutes/endMinutes/daysMask:Int' (553-555) and FocusSessionEntity accepts negative 'targetDurationSeconds/elapsedSeconds' and unbounded 'totalStudyBlocks=1, currentBlockIndex=1' (272-282). Invalid values (hour=99, daysMask=-1, blocks=0, currentBlockIndex=999) persist to Room; RoutineSchedule.durationMinutes then silently 'coerceIn(30,1080)' (584) instead of rejecting them.

• [HIGH] Stringly-typed state plus silent fallbacks mask corrupted rows
  - Location: app/src/main/java/com/example/data/MonolithModels.kt:115,125-126,237,245-249,314,346,556,796
  - Evidence: State is stored as raw String, not the enums that exist in the same file: FocusTaskEntity 'status:String = TaskQueueStatus.QUEUED.name' (115), BlockedAppEntity 'blockTier:String' (237), FocusSessionEntity 'routineOutcome:String?' (314) and 'outcome:String=""' (346), RoutineEntity 'loadoutName:String' (556), DayNoteEntity 'word:String' (796) despite its comment claiming a closed REST/ILL/FAMILY list. Resolvers hide corruption: resolvedStatus (125-126) 'runCatching{valueOf(status)}.getOrDefault(QUEUED)' and resolvedTier (245-249) falls back from isBlocked, so a typo/garbage row silently becomes QUEUED/OPEN instead of failing.

• [HIGH] BlockedAppEntity has two divergent sources of truth: isBlocked vs blockTier
  - Location: app/src/main/java/com/example/data/MonolithModels.kt:233-249
  - Evidence: BlockedAppEntity stores the same fact twice: 'isBlocked:Boolean' (236) and 'blockTier:String = if(isBlocked) LOCKED else OPEN' (237). The default is only computed at construction; copy(blockTier=OPEN) on a blocked row (or vice-versa) leaves isBlocked=true with tier=OPEN, and resolvedTier (245-249) trusts blockTier first while other readers can trust isBlocked — no invariant ties them after construction.

• [MEDIUM] Session span math splits break duration between a magic 600 and a named constant
  - Location: app/src/main/java/com/example/data/MonolithModels.kt:399-401,417-420
  - Evidence: spanTargetSeconds hardcodes the break: 'totalStudyBlocks effectiveTargetSeconds + (totalStudyBlocks-1).coerceAtLeast(0) 600' (399-401), while spanElapsedSeconds in the same class uses the named constant '(MonolithRepository.SAFEBREAKDURATION_SECONDS - safeBreakSecondsRemaining)' (417-420, constant = 10*60 in MonolithRepository.kt:110). Change the constant and target and elapsed silently diverge; the model also hardcodes 600 regardless of StudyModePreset breakMinutes (5/10/15 at lines 68-71).

repository

Verdict: A 2932-line god-object that inverts layering (data -> services/system/UI), hides a DB singleton behind its injected DAO, loses concurrent updates via stale whole-row writes, commits most multi-write operations non-atomically, and swallows nearly every error into a plausible-looking negative.

• [HIGH] God-object with inverted layering and a hidden DB singleton: data layer depends on service/UI/system layers and is untestable as constructed
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:3-23,91-92,1382,1831
  - Evidence: 2932 lines, ~78 suspend + ~30 sync functions; imports at lines 3-23 pull in AppOpsManager, DevicePolicyManager, UsageStatsManager, AudioManager, PowerManager, Settings AND com.example.service.* (MonolithAccessibilityService, MonolithShieldService, RoutineScheduler, WatchStore). It starts services (line 1099: 'MonolithShieldService.startShield(context)' in materialiseRoutineSession), plays sounds (line 1831: 'private val breakSoundEngine: TactileSoundEngine by lazy'), schedules alarms (reseedRoutinesV1 calls RoutineScheduler.cancelChain/rearmChain) and computes UI history/receipts. Worse, the injected DAO is bypassed: lines 1382, 1423, 1482, 2226 do 'val database = MonolithDatabase.getInstance(context)' inside startSession/sealSession/claimReceiptNumber/completeEmergencyBreak, so a fake-DAO test still hits the real singleton DB.

• [HIGH] Lost updates by design: hot paths write whole rows from stale caller-supplied snapshots with no re-read, lock or transaction
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:1934,2104-2126,932-957
  - Evidence: updateSessionProgress(session: FocusSessionEntity) at line 1934 never re-reads the row; it does dao.updateSession(session.copy(...)) whole-row writes (e.g. lines 1969, 2046, 2095) from the caller's stale copy. Its own accrual comment admits concurrency: line 2423 'two tickers call it (the ViewModel loop and the Shield service)'. Same pattern in recordIntercept lines 2104-2126: 'dao.updateSession(active.copy(phantomReachesBlocked = active.phantomReachesBlocked + 1))' and recordAttendanceEvidence lines 932-957 (read snapshot, copy, updateSession, no lock/transaction). sealSession's comment (lines 1414-1423) explicitly fixes this race with a transaction there — proving the authors knew, and left the hot tick/intercept paths unfixed.

• [HIGH] Multi-write operations are non-atomic: only 4 transactions in 2932 lines, the rest commit piecemeal
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:192,2297-2360,2104-2126
  - Evidence: Only 4 withTransaction sites exist in the file (lines 1383, 1424, 1483, 2227). disarmActiveSession (line 2297) does dao.updateSession(...) then a separate dao.insertInterceptLog(...) — crash/kill between them leaves a disarmed session with no log, or vice versa. recordIntercept does 3 separate writes (incrementInterceptCount, insertInterceptLog, updateSession). ensureSeededAndSyncPackages (lines 192-443) performs dozens of seeding/pruning/dedup deletes and inserts under only a Mutex, no transaction, so an interrupted first run leaves a half-seeded, half-pruned database that the isEmpty() guards will then treat as fully seeded.

• [HIGH] Pervasive error swallowing conflates failure with 'no/denied/none' — including a fabricated boot count fed into tamper logic
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:272,531-608,674-720

  - Evidence: At least 12 'catch (: Exception)' blocks discard the error and return a plausible negative: queryRealTodayUsageMinutes line 542 returns emptyMap() (no usage vs failure indistinguishable), isPhoneOrVoipCallActive line 559 returns false (so the call-bypass fails closed and a live call can be blocked), usageAccess line 608 returns UsageAccess.UNSET, hasOverlay/isAccessibility/isDeviceAdmin lines 664-712 return false, and querySystemBootCount lines 718-720 returns fabricated '1' — a value computeVerifiedSessionElapsed (line 735) then uses for reboot/tamper detection, so a Settings read failure can manufacture a reboot/tamper verdict. ensureSeededAndSyncPackages line 272: 'catch (: Exception) { // Safe fallback if package querying is restricted }' silently abandons catalog sync.

• [MEDIUM] No threading contract: dispatcher use is inconsistent inside suspend functions and blocking system queries are exposed as plain synchronous functions
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:188,531,585,1243,1934
  - Evidence: Only 20 of ~78 suspend functions wrap work in withContext(Dispatchers.IO); the rest (applyProfile line 1243, finaliseRoutineSession line 1002, updateSessionProgress line 1934, disarmActiveSession line 2297) execute Room plus CryptoIntegrityManager signing, SharedPreferences and Settings reads on whatever dispatcher the caller used. Conversely ~20 public functions are not suspend at all yet do blocking IPC/system work: queryRealTodayUsageMinutes line 531 (queryUsageStats over 12h), usageAccess line 585 (AppOps), standingWarrant line 626, sealedSessionIdFromMarker line 188 and pendingPruneNote line 476 (SharedPreferences) — nothing in their signatures warns a Main-thread caller. Threading policy is per-function folklore, not a repository contract.

app-catalog

Verdict: The reconcile is a one-way shredder: it trusts a launcher-only, possibly partial list absolutely, deletes user rules with no tombstone or sanity check, lets stale usage never reset to zero, and never corrects names or categories on surviving rows — the table converges to the query's blind spots, not to the phone.

• [CRITICAL] Unconditional prune: an empty or partial installed list wipes the whole catalog, irreversibly
  - Location: app/src/main/java/com/example/data/AppCatalogReconcile.kt:77-79
  - Evidence: val toPrune = existing.filter { it.packageName !in installedByPackage }.map { it.packageName } — with installed=emptyList() this returns every existing package, and AppCatalogPlan (lines 44-48) carries only List<String> package names: no tombstone, snapshot, or recovery data. The caller-side isNotEmpty() check is the only guard, and it does nothing against a non-empty but truncated/visibility-filtered launcher result.

• [HIGH] 'Installed' actually means 'currently launchable': transient absence destroys the user's rule and rebirth defaults to OPEN
  - Location: app/src/main/java/com/example/data/AppCatalogReconcile.kt:77-79
  - Evidence: Header (lines 8-11) defines the input as 'launchable, installed — the PackageManager launcher query', and lines 77-79 prune anything absent from it. A disabled app, an app with no LAUNCHER activity, or one in another profile is installed but absent, so its row is deleted; on rediscovery lines 88-92 assign tier purely from preset/category ('else -> AppBlockTier.OPEN'), so a user's LOCKED ruling on an 'other'-category app silently comes back OPEN and unblocked.

• [HIGH] Usage never follows the real figure down to zero — stale usage is immortal

  - Location: app/src/main/java/com/example/data/AppCatalogReconcile.kt:105-109
  - Evidence: if (realMinutes != null && realMinutes > 0 && realMinutes != current.todayUsageMinutes) — a real figure of 0 produces no update, and a package missing from usageMinutes also produces none, directly contradicting the header at lines 19-22 ('its usage figure, which follows the real one'). An app unused today keeps yesterday's todayUsageMinutes forever, so it can start the day already over its daily limit.

• [MEDIUM] Existing rows are never reconciled except usage: stale names and categories live forever
  - Location: app/src/main/java/com/example/data/AppCatalogReconcile.kt:104-111
  - Evidence: The else branch (lines 104-111) only ever copies todayUsageMinutes; appName and category from the installed list are used only in the insert branch (lines 93-96). A renamed/rebranded app, a locale label change, or a birth mis-categorization is therefore never corrected for the life of the row, despite the header promising the table equals the phone's truth — only tier/limit preservation (a user setting) justifies freezing a row, and name/category are facts, not settings.

• [MEDIUM] associateBy silently picks the last duplicate — multi-activity apps get an arbitrary label
  - Location: app/src/main/java/com/example/data/AppCatalogReconcile.kt:74
  - Evidence: val installedByPackage = installed.associateBy { it.packageName } — associateBy keeps the LAST duplicate, and PackageManager returns one entry per launcher activity, whose labels can differ (activity label vs application label) and whose order is not an app-identity guarantee. The only test (AppCatalogReconcileTest.kt:109-112) feeds two identical InstalledPackage("com.duo","Duo") entries, so divergent-label duplicates are untested and the surviving label/category (categorize is called on info.label at line 86) is arbitrary.

app-rules

Verdict: AppRules.kt is not rule evaluation at all — it is a 26-line display sorter with a lying header, non-exhaustive silent-drop filters, no dedupe or total order, and a count guarantee its only consumer ignores.

• [MEDIUM] Documented ordering guarantee is false: no total order, and not the catalogue's order
  - Location: app/src/main/java/com/example/data/AppRules.kt:5-25
  - Evidence: Header promises 'alphabetical inside each group (a ledger's order, identical to the catalogue's, so a name lives at the same height in both)' (AppRules.kt:5-8), but the sort key is only 'it.appName.lowercase()' (AppRules.kt:22-25) with no packageName tie-breaker, and the catalogue feed is 'ORDER BY isBlocked DESC, interceptCount DESC, appName ASC' (MonolithDao.kt:13) — intercept-count order, binary collation, not this alphabetical order. Ties (same/clone display names) therefore inherit DAO/intercept order and can swap as intercept counts change.

• [MEDIUM] Silent-drop construction: non-exhaustive filters make apps disappear with no trace
  - Location: app/src/main/java/com/example/data/AppRules.kt:18-26
  - Evidence: groupedByRule() is just two equality filters: 'filter { it.resolvedTier == AppBlockTier.PEEK_2M }' and 'filter { it.resolvedTier == AppBlockTier.LOCKED }' (AppRules.kt:21-26) — no exhaustive when, no remainder list, no invariant that peek+shut+open == input size. Anything not exactly those two tiers vanishes silently, and isEmpty (AppRules.kt:18) then reports 'empty' for a list containing only dropped-tier apps. OPEN is dropped by design today, but any future/corrupt tier is dropped by accident with no signal.

• [MEDIUM] Corrupt tiers are laundered into fake user decisions via resolvedTier fallback
  - Location: app/src/main/java/com/example/data/AppRules.kt:22-24

- Evidence: Grouping trusts 'it.resolvedTier' (AppRules.kt:22,24), whose getter launders bad data: 'runCatching { AppBlockTier.valueOf(blockTier) }.getOrElse { if (isBlocked) AppBlockTier.LOCKED else AppBlockTier.OPEN }' (MonolithModels.kt:245-248). A corrupted PEEK_2M string on a blocked row is presented in this 'HIS DECISIONS' ledger as a deliberate SHUT/LOCKED decision the user never made, with no corrupt-data flag anywhere in AppRules.

• [MEDIUM] The headline count guarantee is dead code: the only consumer recounts independently
- Location: app/src/main/java/com/example/data/AppRules.kt:16-17
- Evidence: Header claims 'the counts are the list sizes themselves, so a printed count cannot disagree with the lines under it' (AppRules.kt:8-10) and defines peekCount/shutCount (AppRules.kt:16-17), but the sole production consumer ignores them: ShieldMatrixScreen recomputes 'allApps.count { it.resolvedTier == AppBlockTier.LOCKED }' / 'PEEK_2M' (ShieldMatrixScreen.kt:139-140), derives 'openCount = allApps.size - shutCount - peekCount' (line 149) and prints the tally from those local counts (line 406), while rules = groupedByRule() (line 148) supplies only the lines. The guarantee rests on duplicated predicates staying in sync by hand, not on the data class — grep shows peekCount/shutCount have no production caller, only tests.

• [LOW] No dedupe or disjointness anywhere, and the edge cases are entirely untested
- Location: app/src/main/java/com/example/data/AppRules.kt:12-26
- Evidence: groupedByRule() does no distinctBy { packageName } and AppRules' public constructor (AppRules.kt:12-15) enforces no disjointness, so two stale copies of one package with different blockTier values land in BOTH peek and shut at once — contradictory standing orders, double-counted — and duplicate same-tier copies inflate counts. Tests cover none of this: AppCatalogReconcileTest.kt:117-139 tests only five distinct ASCII names plus an all-OPEN list; no duplicate names/packages, case ties, empty names, non-ASCII names, or corrupt tier strings.

crypto-integrity

Verdict: The integrity check is theatre: blank the signature and it passes, the fields that actually advance the timer aren't signed, and when the KeyStore key isn't used it silently falls back to a hardcoded key printed in the source anyone can forge with.

• [CRITICAL] Blank signature is accepted as valid — tamper check is opt-out by deleting one field
- Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:84
- Evidence: verifySessionState: 'if (session.hmacSignature.isBlank()) return true' and FocusSessionEntity defaults 'val hmacSignature: String = ""' (MonolithModels.kt:279). An attacker editing the local DB just clears the signature column to get a valid verdict and no penalty, and every legacy/unsigned row is exempt by design.

• [CRITICAL] Hardcoded, publicly-known fallback HMAC key in source with silent downgrade to it
- Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:27-30
- Evidence: 'private val fallbackHmacKey = SecretKeySpec("MonolithHardwareHMACFallback32B!".toByteArray(...), HMAC_ALGO)' — a literal shipped in the APK (actually 36 bytes, not 32). signSessionState silently uses it whenever getHardwareHmacKeyOrNull() returns null or hwKey init throws (lines 64-77 plus catch-all re-fallback at 72-77), with no log, flag, or marker in the signature distinguishing fallback from KeyStore signatures. Anyone who decompiles the APK knows the key and can forge any session signature for fallback-signed rows.

• [HIGH] Signature covers only 5 of the fields it claims to protect; the trusted elapsed-time fields are

unsigned
  - Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:60-61
  - Evidence: payload is only '"$startWallTimeMillis|$startElapsedRealtimeMillis|$bootCount|$targetDurationSeconds|$isStopwatch" ' (line 61). elapsedSeconds, accumulatedBeforeRebootSeconds, lastTickElapsedRealtimeMillis, penaltySecondsAdded, isActive, completedSuccessfully, isInSafeBreak/safeBreakSecondsRemaining are all unsigned — yet computeVerifiedSessionElapsed's reboot branch trusts them: 'val priorElapsed = max(session.accumulatedBeforeRebootSeconds, session.elapsedSeconds)' and returns priorElapsed+(nowMono/1000) (MonolithRepository.kt:761-763). Editing accumulated/elapsed in the DB to inflate elapsed time passes HMAC verification untouched.

• [HIGH] 'Hardware/TEE' claim is never verified — no StrongBox, no KeyInfo security-level check, all errors swallowed
  - Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:33-50
  - Evidence: KeyGenParameterSpec.Builder(alias, PURPOSESIGN or PURPOSEVERIFY).build() sets nothing else: no setIsStrongBoxBacked(true), no user-auth/unlocked-device requirement, and grep over app/src/main/java finds zero uses of KeyInfo/securityLevel/attestation. getHardwareHmacKeyOrNull() swallows every Exception to null. The header comment claims 'Hardware AndroidKeyStore HmacSHA256' and repository exposes it as isHmacSignatureValid, but nothing ever proves the key is hardware-backed rather than software KeyStore or the hardcoded fallback.

• [MEDIUM] Verify is non-constant-time and has a key-generation side effect
  - Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:83-93
  - Evidence: 'return expected == session.hmacSignature' uses short-circuiting String equality (timing side channel) instead of MessageDigest.isEqual. Worse, verify delegates to signSessionState (lines 85-92), whose getHardwareHmacKeyOrNull() generates and persists a new KeyStore key when none exists (lines 42-48) — verification mutates key storage, silently masking key deletion/loss and turning a read-only check into key creation.

dayword-drawerday

Verdict: DrawerDay's core 03:30 arithmetic is clean for a no-DST user who never travels, but DayWordStore is a leaky prefs shim whose clear path breaks after the Room drain, whose commit guarantee is unchecked, and whose Kotlin filing rule provably disagrees with its SQL twin on DST days — including a real month boundary.

• [HIGH] Clearing a day word silently fails once it has drained to Room — the word resurrects and can never be undeclared
  - Location: app/src/main/java/com/example/data/DayWordStore.kt:40-47
  - Evidence: DayWordStore.put(null) only removes the SharedPreferences key: 'word == null -> editor.remove(dateIso)' (DayWordStore.kt:43). But the read path is a union in which the drained Room table survives: MonolithRepository.kt:2857-2864 'return fromTable + dayWordStore.all()' and 'fun declareDayWord(dateIso, word) { dayWordStore.put(dateIso, word) }' — declareDayWord never touches daynotes, and a grep of app/src/main finds no DELETE/remove for daynotes at all. Once the seed-sync drain (MonolithRepository.kt:499-512) has moved a word to the table, tapping it off in the Drawer removes nothing that allDayWords() reads, and MonolithViewModel.kt:1955-1962 immediately reloads the union, so the cleared word reappears as still declared.

• [HIGH] Kotlin filing rule and its SQLite 'twin' are different functions on DST days — phantom/missing months at real month boundaries
  - Location: app/src/main/java/com/example/data/DrawerDay.kt:29-31
  - Evidence: DrawerDay.kt:28-31 defines filing as wall-clock arithmetic: 'Instant.ofEpochMilli(...).atZone(zone).toLocalDateTime().minusMinutes(DAYSTARTOFFSETMINUTES)'. Its claimed SQL twin subtracts elapsed seconds from the epoch instead: MonolithDao.kt:209 "(completedAtMillis / 1000) - 12600, 'unixepoch', 'localtime'". These are different functions on DST days. Verified with zone America/NewYork: on 2026-11-01 (US fall-back, a month boundary) a completion at 02:30 EST files as 2026-10-31 under the Kotlin rule but as November under the SQL rule; on 2026-03-08 completions 03:30-04:29 EDT file a day later in Kotlin than in SQL. So the months index can show a phantom month / hide the real one while the screen groups the row elsewhere. The only tests use a DST-free zone: DrawerDayTest.kt:20 'ZoneId.of("Asia/Kolkata")' — the one hazard this code exists around is untested.

• [MEDIUM] No timezone is ever recorded: changing device zone retroactively re-files all history and detaches day words from their day
  - Location: app/src/main/java/com/example/data/DrawerDay.kt:29
  - Evidence: Every DrawerDay entry point defaults to the live device zone — DrawerDay.kt:29, 44, 63 all use 'zone: ZoneId = ZoneId.systemDefault()' — and a grep of app/src/main shows no production caller ever passes a zone. Tasks persist only epoch millis, and DayWordStore keys are bare local-date strings with no zone recorded (DayWordStore.kt:24-43). Nothing pins the zone in force when a row or word was created, and the SQL index uses SQLite 'localtime' (query-time zone, MonolithDao.kt:209). Travel or a timezone change therefore retroactively re-files the entire history into different days/months, while already-declared words stay glued to their old-zone date string and land beside a different day's rows.

• [MEDIUM] put() throws away commit()'s result — the 'cannot be silently lost' guarantee is unverified and unreportable
  - Location: app/src/main/java/com/example/data/DayWordStore.kt:47
  - Evidence: DayWordStore.kt:47 is a bare 'editor.commit()' whose Boolean result is discarded, and put() returns Unit (line 40), as does its only caller MonolithRepository.declareDayWord (MonolithRepository.kt:2862-2864). The header comment (DayWordStore.kt:31-38) claims commit() was chosen so a word can no longer be 'silently lost' — but on a failed disk write commit() returns false, SharedPreferences still shows the value from memory, the ViewModel refresh displays it as declared, and the next process death loses it with zero signal anywhere. The same invisibility applies to the silent 'else -> return' refusal of out-of-list words (line 44).

• [MEDIUM] Validation is write-side-only and half-done: put() accepts any date string and all() returns any string in the file unfiltered
  - Location: app/src/main/java/com/example/data/DayWordStore.kt:25-28
  - Evidence: put() validates only the word, never the key: DayWordStore.kt:40-45 stores any dateIso string ('2026-13-40', '', garbage) verbatim. all() then validates nothing at all: lines 25-28 return '(value as? String)?.let { key to it }' for every entry — no check that the value is in WORDS and no check that the key parses as a date, despite the header's claim the list is 'closed by construction: REST, ILL, FAMILY — never TEST' (lines 17-18). Any stray, legacy or corrupted entry in monolithdaywords is served to the UI as a declared word; unparseable keys are silently skipped by DrawerScreen grouping (DrawerScreen.kt:94 runCatching/LocalDate.parse ?: continue), leaving invisible zombie entries that are never cleaned up.

loadout-store

Verdict: A 55-line store whose one job is durable truth about the last hand pick, yet it discards commit()'s failure signal, turns any corruption into a lying STUDY default, and can crash on a type-corrupted preference — persistence theatre, not persistence.

• [HIGH] put() ignores commit()'s result — the headline durability guarantee is unverified and can silently fail
  - Location: app/src/main/java/com/example/data/LoadoutStore.kt:34
  - Evidence: LoadoutStore.kt:34: 'prefs.edit().putString(KEY_LAST_HAND_LOADOUT, loadout.name).commit()' — the Boolean result is discarded. Caller MonolithViewModel.kt:2103-2104 then unconditionally does loadoutStore.put(loadout) followed by 'loadoutCapIndex.value = LoadoutStore.capIndexOf(loadout)', so a failed disk write (full storage, I/O error) is presented as a successful, durable pick until restart silently reverts it.

• [HIGH] Corrupt/unknown stored value silently collapses to STUDY, recreating the exact lie this store exists to fix
  - Location: app/src/main/java/com/example/data/LoadoutStore.kt:27-40
  - Evidence: LoadoutStore.kt:27-30 swallows any unparseable name: 'runCatching { SmartLoadout.valueOf(name) }.getOrNull()' with no log, cleanup, or migration, and :40 'capIndex(): Int = capIndexOf(lastHandLoadout() ?: SmartLoadout.STUDY)'. A renamed enum entry, a downgrade, or a corrupted value therefore shows STUDY (0) while the app tiers underneath remain on the previously applied SLEEP/WORK_SPRINT loadout — exactly the selector-lying-about-state bug the class doc (lines 5-21) claims to have cured. Corruption is also indistinguishable from 'never picked' (both return null). No test stores an invalid value; TruthCar3SweepTest.kt:271-291 only tests empty and valid round-trips.

• [MEDIUM] getString() sits outside runCatching — a type-corrupted preference crashes the app instead of being handled
  - Location: app/src/main/java/com/example/data/LoadoutStore.kt:28
  - Evidence: LoadoutStore.kt:28: 'prefs.getString(KEY_LAST_HAND_LOADOUT, null)?.let { name -> runCatching { SmartLoadout.valueOf(name) } }' — runCatching wraps only valueOf, not getString. SharedPreferences.getString throws ClassCastException when the key holds a non-String value, so that form of corruption escapes and crashes the read. The read runs at MonolithViewModel.kt:378 'MutableStateFlow(loadoutStore.capIndex())' during ViewModel construction, i.e. a startup crash loop until app data is cleared.

• [MEDIUM] Synchronous commit() runs on the main thread on every hand pick
  - Location: app/src/main/java/com/example/data/LoadoutStore.kt:33-34
  - Evidence: LoadoutStore.kt:33-34 performs a synchronous commit() (disk fsync) on the caller's thread. Its only caller, MonolithViewModel.applySmartLoadout (MonolithViewModel.kt:2098-2104), calls loadoutStore.put(loadout) directly — before and outside its viewModelScope.launch/Dispatchers.IO repository work — on the path invoked from the Shield UI tap handler (ShieldMatrixScreen.kt:287-292 onSelect). Every loadout tap blocks the main thread on disk I/O; the class doc justifies commit-over-apply for durability but never moves it off the UI thread.

• [LOW] 'ONE mapping' claim is false — the seat order is duplicated in at least two other places with no compiler link
  - Location: app/src/main/java/com/example/data/LoadoutStore.kt:44-52

- Evidence: LoadoutStore.kt:44-46 claims capIndexOf is 'the ONE mapping between a loadout and its seat on the cap', but the inverse mapping is re-hardcoded in ShieldMatrixScreen.kt:290-292 ('0 -> STUDY; 1 -> WORKSPRINT; else -> SLEEP', where any out-of-range index silently means SLEEP) and the same order is duplicated as a literal list in MonolithViewModel.kt:1522 'listOf(SmartLoadout.STUDY, SmartLoadout.WORKSPRINT, SmartLoadout.SLEEP)'. Adding or reordering an enum entry requires finding all three sites; nothing in the compiler links them, so the 'one mapping' is documentation fiction.

week-routine-derive

Verdict: The Monday anchor math itself is sound java.time, but everything built on it is timezone-fragile, retroactively forgiving of pauses, and invents MISSED judgements for migrated routines — the derivation is not the pure, provable function its own header claims.

• [CRITICAL] All week/window identity floats with the mutable system timezone — cap bypassable, claims break on travel
- Location: app/src/main/java/com/example/data/MonolithWeek.kt:28; app/src/main/java/com/example/data/RoutineDerive.kt:142,146
- Evidence: MonolithWeek.kt:28 'fun weekAnchorMillis(nowMillis: Long, zone: ZoneId = ZoneId.systemDefault())' and its doc (lines 12-16) makes a spend count iff its stored anchor equals the anchor recomputed 'at evaluation time' — change the phone's timezone across a Monday boundary and the stored anchor stops matching, so the 1-per-week cap silently resets and a second break is granted. RoutineDerive has the same flaw: derive()/deriveOne() take no zone (RoutineDerive.kt:79-108), the window math they delegate to uses Calendar.getInstance() (MonolithModels.kt:642), and claim identity is exact epoch equality 'active.windowStartMillis == window.startMillis' (line 142) / 'mine.firstOrNull { it.windowStartMillis == window.startMillis }' (line 146). After travel the same local window computes a different start instant, the stored claim no longer matches, and the code can mint a second session for a window it swears at line 149 is 'no second mint, ever', or judge an attended window MISSED in judgeLatestClosed (lines 205-206). The header's 'PURITY, ABSOLUTE' (line 8) is false: the system timezone is a hidden, mutable input.

• [HIGH] J6 pause amnesty is retroactive: a pause set minutes before window end erases the whole window's judgement
- Location: app/src/main/java/com/example/data/RoutineDerive.kt:204
- Evidence: RoutineDerive.kt:204 'if (routine.pausedUntilMillis >= closed.endMillis) return null' under the comment 'J6: a window the pause covered end-to-end never existed.' RoutineEntity stores only pausedUntilMillis — no pause-start exists anywhere — so the code cannot know the pause covered the window end-to-end. A pause set at 19:29 for a 09:00-19:30 window, extending past 19:30, retroactively erases a window that ran unpaused for ten and a half hours: no MISSED, no judgement, nothing. The test suite only covers a pause that genuinely covered the whole window, so this amnesty-by-accident is untested.

• [HIGH] Edit-horizon guard is defeated by the migration default: scheduleEditedAtMillis=0 makes every legacy window 'provably' MISSED
- Location: app/src/main/java/com/example/data/RoutineDerive.kt:226; app/src/main/java/com/example/data/MonolithDatabase.kt:59
- Evidence: RoutineDerive.kt:226 'if (closed.startMillis > routine.scheduleEditedAtMillis) return WindowJudgement.MISSED', guarded by the comment at lines 228-231 that before the edit horizon 'the record does not invent misses' (returns UNKNOWN). But the migration adds the column as 'scheduleEditedAtMillis INTEGER NOT NULL DEFAULT 0' (MonolithDatabase.kt:59) and the model

documents '0 = no edit recorded under the new engine' (MonolithModels.kt:568). Every migrated, never-re-edited routine therefore has horizon 0, every closed window start is > 0, and the UNKNOWN branch is dead code for exactly the legacy rows it was written to protect — the derivation invents MISSED judgements for windows it cannot prove were ever scheduled.

• [MEDIUM] Claim selection is order-dependent firstOrNull: a stamped duplicate masks an unstamped one and flips CLOSED_DUE to OUTSIDE
  - Location: app/src/main/java/com/example/data/RoutineDerive.kt:146,153
  - Evidence: RoutineDerive.kt:146 picks 'mine.firstOrNull { it.windowStartMillis == window.startMillis }' from the caller-supplied, unsorted sessions list, and line 153 derives state from that one row alone: 'if (claim.routineOutcome == null) CLOSEDDUE else OUTSIDE'. With duplicate claims for one window (the corruption case this codebase elsewhere drills for), a stamped row listed first yields OUTSIDE while the unstamped duplicate sits in finaliseDue — directly contradicting the no-window branch at lines 179-180, where any non-empty finaliseDue means CLOSEDDUE, and the state machine's own definition (line 45: CLOSED_DUE = 'an unfinalised routine session exists'). Same data, different DAO return order, different state.

• [MEDIUM] HELD-vs-MISSED judgement rests on an elapsed-time estimate that undercounts every paused session
  - Location: app/src/main/java/com/example/data/RoutineDerive.kt:243-255
  - Evidence: RoutineDerive.kt:243-255 overlapsAny() admits in its own doc that a session's span is 'estimated as [startWallTimeMillis, start + elapsed]' — spanEnd = 's.startWallTimeMillis + s.elapsedSeconds * 1000L' for inactive rows. elapsedSeconds is running time, not wall occupation: a session that sat paused through a routine window occupied it in wall time but its estimated span ends early, so overlapsAny returns false and judgeLatestClosed (lines 209-226) falls through HELD to a hard MISSED. A judgement the spec demands be 'provable' (WindowJudgement doc, lines 51-54) is being decided by an estimate that is wrong for every paused session.

receipt-state

Verdict: This is not a state machine but a bag of string constants and a one-slot pointer pretending to be one: routine finishes bypass the no-receipt valve law, the faulted flag can be lost between two writes, the sleep finish evaporates with the clock or a deleted routine, and the detention sentinel can freeze the Deck for the life of the process.

• [CRITICAL] Routine branch of finishKindOf bypasses the VALVE_BROKE 'no receipt ever' law
  - Location: app/src/main/java/com/example/data/ReceiptState.kt:90
  - Evidence: ReceiptState.kt:38-41 defines VALVEBROKE as 'the token was spent and the session BROKE. No receipt, no №, ever.' But finishKindOf's routine branch at ReceiptState.kt:90-98 returns ROUTINEATTENDED/ROUTINE_PARTIAL solely from routineOutcome and never checks row.outcome; only the SLEEP stand-down sub-branch (94-95) excludes a non-empty outcome. MonolithRepository.kt disarmRoutineSessionByBreak routes a valve-broken non-SLEEP routine into finaliseRoutineSession, which stamps routineOutcome ATTENDED/PARTIAL — so a token-spent routine break can still mint the receipt the file itself forbids.

• [CRITICAL] Detention null-pawl sentinel (Long.MAX_VALUE/4) permanently strands the finish state
  - Location: app/src/main/java/com/example/data/ReceiptState.kt:208
  - Evidence: rackStartMs(null) = maxOf(1200, Long.MAX_VALUE/4) (ReceiptState.kt:208-209) instead of 'undefined'. MonolithViewModel.kt:1308-1313 then computes rackEnd =

rackStartMs(pawlLiftMs)+280 and delay(remain) to retire FinishState — precisely on the faulted branch where pawlLiftMs stayed null. remain is ~2^61 ms, so the cleanup never fires in that process: FinishState stays non-null forever and TactileDeckScreen.kt:305-332 keeps rendering its frozen display/progress instead of live state.

• [HIGH] faulted is not persisted atomically with the claim — a crash in the gap silently un-faults the receipt
  - Location: app/src/main/java/com/example/data/ReceiptState.kt:182
  - Evidence: ReceiptState carries faulted (ReceiptState.kt:182) but the persisted store named in the file header (11-16) is only receiptSequence + lastReceiptSessionId + lastReceiptOutcome + the row's receiptNumber/endWallMillis — faulted is not in the claim. MonolithRepository.publishReceipt (1710-1715) commits claimReceiptNumber first and only afterwards calls dao.stampReleaseFault in a separate write; a process death between them leaves a claimed, faulted receipt whose rehydration (MonolithRepository.kt:1783, faulted = row.releaseFaultShield) permanently reads false.

• [HIGH] SLEEP stand-down finish is time- and caller-dependent, so it can flip back to null or be lost entirely
  - Location: app/src/main/java/com/example/data/ReceiptState.kt:94
  - Evidence: The stand-down branch (ReceiptState.kt:94-95) requires routineOutcome==null && caller-supplied isSleepLoadout && windowEndMillis in 1..nowWallMillis && outcome.isEmpty(). It is non-monotonic (a wall-clock rollback un-finishes a row) and the loadout fact is not on the row: MonolithViewModel.isSleepLoadout looks it up in the in-memory routines StateFlow / live routine row, so a deleted routine or an unloaded flow makes the finish vanish permanently, while MonolithViewModel.startFinishController marks first-sighting finishes older than 600_000 ms handled forever — a stand-down discovered late is discarded, never claimed.

• [MEDIUM] No persisted dismissed/cleared state: single-slot rehydration resurrects receipts the user already left behind
  - Location: app/src/main/java/com/example/data/ReceiptState.kt:11
  - Evidence: The header admits persistence is one slot only: '№ = receiptSequence, slot pointer = lastReceiptSessionId, outcome word = lastReceiptOutcome' and 'The slip itself is re-derived... never stored' (ReceiptState.kt:11-23). ReceiptState (166-185) has no dismissed/consumed/cleared field — only restored and a deliberately unpersisted receiptPublishedMonoMs ('never persisted, never restored', 161-164). Dismissal lives in MainActivity.kt:268 as remember{} state and the arm-time clear in MonolithViewModel.kt:2200 only nulls memory; nothing clears the persisted pointer, so after process death runSliceERecovery/rehydrateReceipt resurrects a slip the user already dismissed or superseded by arming again.

undo-wall

Verdict: This slice's math is sound only inside a single test harness instance: in the real app its accrual dedup is defeated by per-repository instances, its undo durability guarantee ignores commit failure, its window trusts the wall clock, and its break countdown leaks time every tick.

• [CRITICAL] WallAccrual dedup is per-instance, but VM and Shield tickers hold different instances — double-crediting is back
  - Location: app/src/main/java/com/example/data/WallAccrual.kt:55
  - Evidence: WallAccrual keeps its anchor in per-instance fields (WallAccrual.kt:55-58: 'private var sessionId... private var lastMonoMillis') and @Synchronized (line 67) only serializes calls on that one

object. But MonolithRepository.kt:2410 'private val taskAccrual = WallAccrual()' creates one accrual per repository, and the two tickers use different repositories: MainActivity.kt:84 'val repository = MonolithRepository(...)' for the ViewModel vs MonolithShieldService.kt:81 'val repository = MonolithRepository(applicationContext, dao)'. Each instance anchors independently and credits the full delta since its own last call, so the doc claim at WallAccrual.kt:11-17 that 'the second ticker's call simply finds the delta already consumed' is false in the shipped wiring — the tick-count double-credit it was built to kill survives as double wall-credit, with concurrent read-modify-write of actualSecondsSpent deciding how much lands. The unit test masks this: WallAccrualTest.kt:18 uses a single shared 'val accrual = WallAccrual()' for both simulated tickers.

• [HIGH] save() ignores commit()'s result, so the banner can promise an undo backed by nothing
  - Location: app/src/main/java/com/example/data/UndoDeleteStore.kt:36
  - Evidence: Header promises the copy is 'written with commit() — before the banner ever shows' and save()'s own doc: 'Synchronous (commit()): the copy must be durable before the UI presents the delete as reversible.' Yet save() ends in '.commit()' (line 50) discarding the Boolean result and returns Unit, and MonolithViewModel.kt:1780-1781 unconditionally does 'undoDeleteStore.save(deleted, System.currentTimeMillis()); _deletedTaskForUndo.value = deleted'. A failed commit (disk error/full) therefore presents a reversible delete whose only restore copy does not exist — silently.

• [HIGH] Undo window is pure wall-clock: a clock rollback resurrects expired deletes, a jump forward steals live undos
  - Location: app/src/main/java/com/example/data/UndoDeleteStore.kt:49
  - Evidence: save() stores only 'putLong(KEYDELETEDAT, deletedAtWallMillis)' (line 49) with no monotonic or boot-count anchor, and rehydration (MonolithViewModel.kt:754-756) computes 'WINDOW_MILLIS - (System.currentTimeMillis() - deletedAt)'. A wall-clock rollback (manual change, NTP correction) makes remaining exceed the window — deletedAt in the future yields remaining > 5,000ms — resurrecting an expired undo the header (lines 17-21) swears is 'never resurrected'; a forward jump kills a live undo instantly. The sibling code in WallAccrual.kt uses elapsedRealtime monotonic deltas precisely to avoid trusting wall time.

• [MEDIUM] breakRemainingAfter leaks the sub-second remainder on every persisted tick, so breaks systematically overrun
  - Location: app/src/main/java/com/example/data/WallAccrual.kt:47
  - Evidence: Line 47: 'val elapsedSeconds = ((nowMonoMillis - lastTickMonoMillis) / 1000L).coerceAtLeast(0L)' truncates to whole seconds, and the caller persists the truncated pair with the FULL current instant as the new anchor (MonolithRepository.kt updatedBreak copy: 'safeBreakSecondsRemaining = nextBreakRemaining, lastTickElapsedRealtimeMillis = nowMono'). The fractional remainder is discarded on every tick, unlike WallAccrual.observe (lines 97-101) which advances its anchor by only 'seconds * 1000L' to keep it. At any cadence that drifts off exact whole seconds (a 1s delay() loop plus work, e.g. 1,050ms ticks), each tick loses ~50ms and a 600s break overruns by ~30s. WallAccrualTest.kt:75-91 only ever ticks on exact 1,000/2,000ms multiples, so the leak is untested.

• [MEDIUM] WallAccrual's anchor is volatile and consumed before the credit is persisted — death gaps and crash windows vanish from stored seconds
  - Location: app/src/main/java/com/example/data/WallAccrual.kt:55
  - Evidence: The anchor exists only as in-memory vars (lines 55-58); after process death 'anchored' is false and observe() re-anchors returning 0 (lines 80-88), so all session time since the last persisted

credit is silently dropped from the task's permanent actualSecondsSpent — while breakRemainingAfter in the same file is deliberately persisted-pair based so 'a process-death gap counts' (lines 35-38). Worse, observe() advances lastMonoMillis (lines 98-100) before the caller has durably written the returned credit, so a crash between observe and the DAO write consumes seconds that were never recorded anywhere.

accessibility-service

Verdict: This is a maximally-privileged screen reader posing as a blocker: its call exemption disables everything, its async per-event design races the window it inspects, and its uninstall, Shorts and browser guards collapse on OEM, locale, browser or app-version variation.

• [CRITICAL] Call bypass is a global kill-switch, not a per-call exemption
  - Location: app/src/main/java/com/example/service/MonolithAccessibilityService.kt:190,297-303
  - Evidence: onAccessibilityEvent does if (isCallActive()) return at line 190 before the session/block coroutine, and isCallActive (297-303) only checks global AudioManager.mode == MODERINGTONE || MODEINCALL || MODEIN_COMMUNICATION. It never checks that event.packageName is the call app, so keeping any VoIP/voice-communication session alive (e.g. a WhatsApp/Discord call) disables app, sniper, site and uninstall blocking for every other app.

• [CRITICAL] Per-event unbounded IO coroutine races the window it claims to police
  - Location:
app/src/main/java/com/example/service/MonolithAccessibilityService.kt:192-197,225,255,273; app/src/main/res/xml/accessibilityserviceconfig.xml:3,8
  - Evidence: Config subscribes to typeWindowStateChanged|typeWindowContentChanged with notificationTimeout="80" (config:3,8), yet every event does scope.launch (192) doing getActiveSessionSnapshot (194), getGuardSettingsSnapshot (203), getAllAppsSnapshot twice (225,273) and getAllSitesSnapshot (255) with no throttle, dedup, debounce or cancellation of prior jobs. Inside the async job it re-reads rootInActiveWindow (197) — the current window, not the event's window — so by completion the tree can belong to a different app than pkgName, causing missed blocks, wrong-app intercepts, and repeated launchInterceptScreen/GLOBALACTIONBACK storms while scrolling.

• [CRITICAL] Uninstall/tamper guard is a 4-package, English-only text heuristic
  - Location: app/src/main/java/com/example/service/MonolithAccessibilityService.kt:309-329
  - Evidence: installerPackages is only 4 entries (310-315: google/android/Samsung/MIUI packageinstaller) and Settings protection only fires for exactly com.android.settings (318) and only when one English dump contains both 'monolith' and 'force stop|deactivate|uninstall|clear storage|clear data|device admin' (320-327). Play Store com.android.vending, OEM Settings packages (ColorOS/Oplus/Vivo etc.), launcher long-press uninstall, and any non-English locale are not covered at all — the headline uninstall guard evaporates off Pixel/Samsung/Xiaomi English devices.

• [HIGH] Maximum accessibility power with a materially narrower disclosure, exercised even with no session
  - Location:
app/src/main/java/com/example/service/MonolithAccessibilityService.kt:172-179,396,405-414; app/src/main/res/xml/accessibilityserviceconfig.xml:5-6; app/src/main/res/values/strings.xml:4
  - Evidence: Config grants canRetrieveWindowContent="true" plus flagIncludeNotImportantViews (config:5-6), and onAccessibilityEvent acquires rootInActiveWindow at 172-173 BEFORE any session

exists check, on every event. The code then dumps all Settings text+descriptions recursively in collectWindowTextLowercase (405-414) and reads browser URL-bar text (396). The user-facing description (strings.xml:4) claims only 'Detects when a blocked app is launched... and presents the Intention Airlock' — it does not disclose full window-content, Settings-text or URL reading, session or not.

• [HIGH] Shorts/Reels and browser enforcement are brittle hardcoded ID/package lists
  - Location: app/src/main/java/com/example/service/MonolithAccessibilityService.kt:67-68,335-357,368-395
  - Evidence: Sniper depends on 5 hardcoded internal view IDs (335-341, e.g. com.google.android.youtube:id/reelrecycler, com.instagram.android:id/clipsviewerviewpager) plus fallback exact English selected-tab text 'shorts'/'reels' (354-357) — an app update, A/B layout or non-English locale silently breaks it. Browser blocking is equally closed: only 5 browser packages (368-375) and 5 URL-bar IDs (383-389), skips the bar when node.isFocused (395), and matchBlockedDomain is raw substring urlText.contains(domain) (67-68), so any other browser, in-app WebView/Custom Tab, or a domain appearing in a query string/path produces bypasses and false positives respectively.

shield-service

Verdict: The shield is a fragile 2-second UsageStats poll dressed as a reliable blocker: one loop exception makes it a lying zombie, its start/stop paths swallow or invite platform FGS failures, and START_STICKY is its entire resurrection plan.

• [CRITICAL] One exception in the monitor loop silently kills blocking while the service and protecting flag stay alive
  - Location: app/src/main/java/com/example/service/MonolithShieldService.kt:83-193
  - Evidence: startMonitoringLoop at :83 launches while(isActive) with dao.getActiveSessionSnapshot(), repository.updateSessionProgress() (:108), dao.getAllAppsSnapshot() (:165,:180) and repository.queryForegroundBlockedApp() (:178) and no try/catch anywhere in the loop; a single Room/usage-stats exception cancels monitorJob while the foreground notification stays up and, if the flip at :155-160 already happened, the @Volatile companion 'protecting=true' stays stale-true - a zombie shield that advertises protection it no longer performs.

• [CRITICAL] Blocking is 2-second polling over a UsageStats lastTimeUsed guess, not foreground detection
  - Location: app/src/main/java/com/example/service/MonolithShieldService.kt:193; app/src/main/java/com/example/data/MonolithRepository.kt:564-580
  - Evidence: Shield polls with delay(2000L) (:193) and MonolithRepository.queryForegroundBlockedApp() uses queryUsageStats(INTERVALDAILY, end-15000, end) then filters 'lastTimeUsed >= (end - 8_000L)' and picks maxByOrNull lastTimeUsed (MonolithRepository.kt:564-580) - it does not know the actual foreground app, so it misses apps opened/used outside the 8s window, can intercept an app the user already left, and gives a blocked app up to seconds of free use between polls plus usage-stats latency.

• [HIGH] Unguarded startForeground on targetSdk 36, and start failures swallowed by the caller helper
  - Location: app/src/main/java/com/example/service/MonolithShieldService.kt:62-68,294-305
  - Evidence: ServiceCompat.startForeground(this, NOTIFICATIONID, ...,

FOREGROUNDSERVICETYPESPECIAL_USE) at :62-68 has no try/catch, while the app targets SDK 36 (app/build.gradle.kts:18 targetSdk=36) where specialUse FGS start/type failures throw (ForegroundServiceStartNotAllowedException/SecurityException); meanwhile startShield() at :294-305 catches Exception from startForegroundService() and comments 'Safe ignore in test environments', silently hiding real start failures from every caller with no retry or fallback.

• [HIGH] Kill/restart resilience is just START_STICKY - no onTaskRemoved, watchdog or self-rescheduling in this service
  - Location: app/src/main/java/com/example/service/MonolithShieldService.kt:75,240-282
  - Evidence: The only restart provision in the entire file is 'return STARTSTICKY' at :75; grep shows no onTaskRemoved override and the file schedules no alarm, WorkManager job or restart intent. STARTSTICKY does nothing after force-stop and its system restart re-enters onStartCommand through the same background-FGS-restricted startForeground path, so an OEM/process kill leaves only the in-memory @Volatile instance/protecting state (:255-282) - which dies with the process - to attest protection.

• [HIGH] stopShield uses plain startService and swallows its failure, so stopping can silently no-op
  - Location: app/src/main/java/com/example/service/MonolithShieldService.kt:308-316
  - Evidence: stopShield() at :308-316 always does context.startService(intent) - unlike startShield() it never uses startForegroundService() on API 26+ - so a background stop throws IllegalStateException, which is caught and dismissed as '// Safe ignore', i.e. the stop command can silently never reach onStartCommand's ACTIONSTOPSHIELD branch (:36-44), leaving a sticky foreground shield running against a session the rest of the app believes is ended.

boot-deviceadmin

Verdict: Boot restore is a plausible-but-uncaught coroutine racing the broadcast window, and the device-admin 'uninstall guard' is pure theater — it blocks nothing, checks no session, was never wired to a real activation prompt, and declares an unused password policy that puts the app at Play-policy risk.

• [HIGH] Device-admin 'uninstall guard' is theater: it blocks nothing and lies about session state
  - Location: app/src/main/java/com/example/service/MonolithDeviceAdminReceiver.kt:8-15
  - Evidence: Class doc claims it uses DeviceAdminReceiver 'to prevent uninstalling Monolith from the home screen while a focus session is active', but onDisableRequested() unconditionally returns one static string: 'Monolith Uninstall Guard is protecting your active study session...' It never queries whether any session is active, and onDisableRequested cannot block deactivation anyway — the user confirms, onDisabled() fires, and uninstall proceeds. The 'guard' is a warning label, not a guard.

• [HIGH] Unused limit-password policy declared — over-privileged and a Play policy/removal risk
  - Location: app/src/main/res/xml/deviceadminreceiver.xml:3-5; app/src/main/AndroidManifest.xml:120-130
  - Evidence: deviceadminreceiver.xml declares exactly one policy: '<limit-password />' — a password-enforcement power the app never exercises (the receiver only returns a string and stamps WatchStore; no DevicePolicyManager password call exists). The app thus requests a scary, unrelated device-admin policy purely as uninstall friction, which is exactly the pattern Google Play's Device Admin policy restricts, and the manifest even mislabels it to the user: android:description="@string/accessibilityservicedescription" on the device-admin receiver.

• [MEDIUM] Device admin is never actually requested in-app; activation relies on a fragile hardcoded

Settings deep-link
  - Location: app/src/main/java/com/example/MainActivity.kt:369-381
  - Evidence: grep for ACTIONADDDEVICEADMIN / addDeviceAdmin across app/src returns zero hits. The only enablement path is MainActivity.openFixPane() deep-linking to a hardcoded component 'com.android.settings.DeviceAdminSettings' with a fallback to generic ACTIONSECURITY_SETTINGS, leaving the user to find and toggle the entry manually. On OEM skins where that Settings activity differs, the flow degrades to a generic settings page.

• [MEDIUM] Boot work runs in an uncaught coroutine inside goAsync()'s short window; failures crash or silently lose the shield
  - Location: app/src/main/java/com/example/service/MonolithBootReceiver.kt:34-74
  - Evidence: Both branches use 'val pendingResult = goAsync(); CoroutineScope(Dispatchers.IO).launch { try { ... RoutineHeartbeat.tick(appContext, source, force = true) ... MonolithShieldService.startShield(appContext) } finally { pendingResult.finish() } }' — try/finally with no catch. Any exception from the forced heartbeat reconcile or DAO escapes as an uncaught coroutine exception and crashes the process during boot, and the full reconcile + DB work + service start must fit inside goAsync()'s ~10s broadcast lifetime or the system can kill the process mid-reconcile. Shield failure is also invisible: startShield() swallows all exceptions ('catch (_: Exception) { // Safe ignore }', MonolithShieldService.kt:303-305), so boot can complete believing the shield is up when it never started.

• [LOW] Neither receiver has a single test
  - Location: app/src/test, app/src/androidTest (no matches)
  - Evidence: grep for BootReceiver, DeviceAdminReceiver and onDisableRequested across app/src/test and app/src/androidTest returns zero files. The boot-stamp dedup logic (LOCKED_BOOT vs BOOT vs QUICKBOOT), the time-change reconcile path, and the admin enable/disable WatchStore stamping have no test coverage whatsoever.

notification-listener

Verdict: MonolithNotificationListener is an unbounded, plaintext, write-only memory for other people's messages: it hoards notification bodies forever with no dedup, no expiry, and no deletion tracking, and when Android inevitably disconnects it, it dies silently with zero self-recovery.

• [HIGH] Full notification bodies stored in plaintext forever — no TTL, no cap, no pruning on the insert path
  - Location: app/src/main/java/com/example/service/MonolithNotificationListener.kt:66-91; app/src/main/java/com/example/data/MonolithModels.kt:463-471; app/src/main/java/com/example/data/MonolithDao.kt:147-168
  - Evidence: Listener extracts verbatim body text (MonolithNotificationListener.kt:66-68 'val text = extras?.getCharSequence(Notification.EXTRA_TEXT)...') and inserts it (kt:80-91). CapturedNotificationEntity is plain String columns with no expiry field (MonolithModels.kt:463-471 'val senderTitle: String, val messageBody: String'). The DAO insert has no pruning and the table is uncapped — the LIMIT 100 is display-only (MonolithDao.kt:147) and the snapshot query is 'Deliberately NO LIMIT' (MonolithDao.kt:151-159). Deletion exists only as manual clear (MonolithDao.kt:165-168). Any non-OPEN app's OTPs, banking alerts, and private message text sit unencrypted on disk indefinitely until the user happens to clear the digest.

• [HIGH] Zero deduplication or notification-kind filtering: every update of the same notification inserts

a new row and re-fires cancelNotification
  - Location: app/src/main/java/com/example/service/MonolithNotificationListener.kt:50-104
  - Evidence: onNotificationPosted never inspects sbn.key, sbn.postTime, notification.flags, or group state, and CapturedNotificationEntity has no notification-key column to dedupe against (MonolithModels.kt:463-471). A repo-wide grep for FLAGGROUPSUMMARY, FLAGONGOING, isGroup and GROUPSUMMARY returns zero hits in app/src/main/java. Chat apps repost the same notification on every message/edit and media/download apps repost progress constantly — each repost becomes a fresh permanent digest row (kt:71-97) and another cancelNotification(sbn.key) attempt, including against ongoing/group-summary notifications the listener cannot meaningfully cancel.

• [HIGH] Recalled and deleted messages are resurrected permanently: onNotificationRemoved is never implemented
  - Location: app/src/main/java/com/example/service/MonolithNotificationListener.kt:24-109
  - Evidence: The class overrides only onListenerConnected, onListenerDisconnected, onNotificationPosted and onDestroy (MonolithNotificationListener.kt:35-109); a repo-wide grep for onNotificationRemoved returns zero hits. When a sender deletes a message for everyone or an app withdraws a notification, the system's copy vanishes but the Room copy captured at kt:80-91 persists in the digest until manual clear — the app silently defeats sender-side deletion and retains content its owner explicitly retracted.

• [MEDIUM] No rebind recovery and silent failure swallowing: a disconnected listener stays dead, a failed cancel is invisible
  - Location: app/src/main/java/com/example/service/MonolithNotificationListener.kt:41-48,93-95
  - Evidence: onListenerDisconnected only nulls the instance and stamps WatchStore (kt:41-48); a repo-wide grep shows requestRebind is never called anywhere in the app, so after the routine OEM/system disconnects that kill notification listeners, suppression just stops with no self-healing. Worse, the one operation that enforces suppression is wrapped in 'try { cancelNotification(sbn.key) } catch (_: Exception) {}' (kt:93-95) — if cancel fails, the notification stays visible AND a copy is stored in the digest, with no log, no retry, and no signal to the user.

• [MEDIUM] Capture decision is an unguarded async race: session state is read whenever the coroutine gets scheduled, and DB failures have no handler
  - Location: app/src/main/java/com/example/service/MonolithNotificationListener.kt:25,71-97; app/src/test/java/com/example/TruthCar3CaptureBodyTest.kt:29-36
  - Evidence: onNotificationPosted extracts title/text synchronously but defers the active-session and tier checks into scope.launch (kt:71-74), so the decision uses session state at DB-read time, not at post time — notifications posted at session start/end boundaries are mis-captured or missed, and back-to-back posts can be processed out of order. The scope is 'CoroutineScope(SupervisorJob() + Dispatchers.IO)' with no CoroutineExceptionHandler (kt:25), and the insert at kt:80 is outside any try/catch (only cancelNotification is guarded) — a Room failure inside the launch is an uncaught coroutine exception, i.e. a crash-or-silently-dead capture path, never a recorded failure. Test coverage confirms the neglect: the only listener test, TruthCar3CaptureBodyTest.kt, asserts just the empty-body case — nothing exercises dedup, cancel failure, disconnect, or boundary races.

routine-scheduler

Verdict: The entire routine timing mechanism rests on a single inexact Doze-deferrable alarm with no exact-alarm permission anywhere in the app, and its receiver runs a heavyweight database reconcile inside a ~10-second goAsync window with no exception handling and zero tests.

• [CRITICAL] Chain alarm is inexact and the app holds no exact-alarm permission, so routine boundaries fire late by design
  - Location: app/src/main/java/com/example/service/RoutineScheduler.kt:43
  - Evidence: RoutineScheduler.kt:43 uses only alarmManager.setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, boundaryMillis, intent) — setAndAllowWhileIdle is explicitly inexact and Doze can defer/batch it to maintenance windows. Grep of the whole app source finds no setExact, setExactAndAllowWhileIdle, or canScheduleExactAlarms, and AndroidManifest.xml lines 6-14 declare no SCHEDULE_EXACT_ALARM or USE_EXACT_ALARM permission, despite app/build.gradle.kts:18 targetSdk = 36. The header comment's excuse ('Alarms are HINTS... every other trigger runs the same reconcile') collapses when the phone sits idle in a pocket: resume/boot/shield-loop/opportunistic triggers all require activity or a running session, so for an idle device this one deferrable alarm IS the mechanism — a routine that should start blocking (or stop blocking) at 20:00 can start at the next Doze maintenance window instead.

• [HIGH] Receiver runs the full reconcile inside goAsync()'s ~10s budget on a cold process; a slow tick can be killed before it rearms the chain
  - Location: app/src/main/java/com/example/service/RoutineAlarmReceiver.kt:21-30
  - Evidence: RoutineAlarmReceiver.kt:22-30: val pendingResult = goAsync() then launches RoutineHeartbeat.tick(...) on Dispatchers.IO. That tick (RoutineHeartbeat.kt:88-183) performs multiple Room snapshot queries, per-session repository.finaliseRoutineSession writes, possible shield service start/stop, and only at step 5 rearms the chain alarm. On a cold-started, Doze-woken process this can exceed the broadcast goAsync timeout (~10s); if the system kills the process mid-tick, pendingResult.finish() never runs and step 5 never executes — the single chain alarm is consumed and not replaced, silently ending all future routine boundaries until an unrelated trigger happens to fire.

• [HIGH] Any exception in the reconcile crashes the app from a background broadcast — receiver scope has no supervisor or exception handler
  - Location: app/src/main/java/com/example/service/RoutineAlarmReceiver.kt:23-29
  - Evidence: RoutineAlarmReceiver.kt:23 creates a bare CoroutineScope(Dispatchers.IO).launch { try { RoutineHeartbeat.tick(...) } finally { pendingResult.finish() } } — try/finally with no catch, no SupervisorJob, no CoroutineExceptionHandler. RoutineHeartbeat.tick (RoutineHeartbeat.kt:57-75) likewise wraps reconcileOnce in try/finally only for the mutex, with no catch. Any Room/repository exception therefore propagates out of the launch to the default uncaught-exception handler and kills the process — triggered by an alarm the user never saw, at a schedule boundary.

• [MEDIUM] Scheduler fails silently: null AlarmManager or PendingIntent just returns, leaving no chain and no trace
  - Location: app/src/main/java/com/example/service/RoutineScheduler.kt:39-44
  - Evidence: RoutineScheduler.kt:40-41: val alarmManager = ... as? AlarmManager ?: return and val intent = chainPendingIntent(...) ?: return — if either lookup fails, rearmChain exits with no log, no fallback (no WorkManager backstop exists in this slice), and no liveness signal; combined with heartbeat step 5 recording lastArmedBoundaryMillis = boundary after the call (RoutineHeartbeat.kt:170-178), the system can believe the chain is armed when nothing was ever scheduled.

• [MEDIUM] Zero test coverage for the scheduler or receiver
  - Location: app/src/test (absent); app/src/main/java/com/example/service/RoutineScheduler.kt:78-82
  - Evidence: Grep for RoutineScheduler, RoutineAlarmReceiver, and setAndAllowWhileIdle across

app/src/test and app/src/androidTest returns no matches at all. Alarm identity, cancel-then-set behaviour, the FLAGNOCREATE armed-probe, legacy-alarm cancellation (request codes id2 / id2+1), and receiver finish-on-failure behaviour are all unverified — in a slice whose own comment (RoutineScheduler.kt:78-82) admits a previous null-contract bug here 'crashed the first-ever tick' and was caught only by a screenshot harness.

heartbeat-retry

Verdict: This slice is a liability dressed as resilience: the retry scheduler can murder a healthy session's shield off a stale, never-cleared fault row while its advertised final retry never fires, and the heartbeat lets a single exception silently kill the only chain alarm with no recovery, logging, or retry of its own.

• [CRITICAL] Healed release faults are never cleared, so a stale fault can kill a later live session's shield
  - Location: app/src/main/java/com/example/service/ReleaseRetryScheduler.kt:73-94
  - Evidence: runDueRetry picks 'dao.getSessionHistorySnapshot().firstOrNull { it.releaseFaultShield && it.releaseFaultedAtWallMillis > 0L }' (history is ORDER BY startWallTimeMillis DESC, MonolithDao.kt:77-78, so the newest faulted row), and stampReleaseFault only ever sets releaseFaultShield=1 (MonolithDao.kt:106-110) — no code anywhere clears it or marks a fault healed. The only heal check is the global 'if (MonolithShieldService.instance == null)' (line 77), which says nothing about which session the live shield protects. So a fault that healed with retriesDone=1 stays 'faulted' forever; when a later, unrelated session starts (instance != null) and any trigger fires — ViewModel init (MonolithViewModel.kt:1113-1115) or Deck-open (1122-1128) — due = fault+300_000 is long past, so line 92 calls MonolithShieldService.stopShield() on the live session's shield and burns its last retry.

• [CRITICAL] The 'Deck-open' third retry is fake: exhausted cadence returns without ever trying the stop
  - Location: app/src/main/java/com/example/service/ReleaseRetryScheduler.kt:83-92
  - Evidence: Header promises retries at '+30s, +5min, and on Deck-open' (lines 15-22, 35-37), but in runDueRetry the exhausted-cadence branch runs before any stop attempt: 'val due = nextRetryWallMillis(...); if (due == null) { cancel(appContext); return@withContext false }' (lines 83-87). stopShield() is only reached at line 92, after the due checks. Once both timed retries are spent, every Deck-open/boot/alarm trigger takes the due==null exit — the advertised third door never once commands a stop, so a shield that survived two retries is abandoned permanently while the docs claim otherwise.

• [HIGH] Heartbeat has zero failure recovery: one exception kills the one-shot chain alarm silently
  - Location: app/src/main/java/com/example/service/RoutineHeartbeat.kt:59-73
  - Evidence: tick() (lines 59-73) has no try/catch — only try/finally unlocking the gate — and reconcileOnce does all fallible work (DB snapshots, finalise loop, materialise) before step 5 rearms the chain (lines 175-186) and step 6 stamps liveness. The chain is a single one-shot alarm (RoutineScheduler.kt:40-44, setAndAllowWhileIdle), consumed when it fires. One Room/repository exception therefore leaves no chain armed and no tick stamp, and tickAsync's SupervisorJob scope (line 50, 76-79) swallows the failure silently. The header's 'nothing is ever lost' (lines 27-29) is also false on this path: an exception aborts the do-while, discarding a pendingAgain flag set mid-pass. Recovery depends entirely on unrelated triggers happening later.

• [HIGH] Late triggers collapse the cadence: next retry is scheduled in the past and fires instantly
  - Location: app/src/main/java/com/example/service/ReleaseRetryScheduler.kt:92-99

  - Evidence: Retries are stamped and rescheduled from the original fault instant, not from now: after stopShield, 'dao.stampReleaseRetries(faulted.id, done)' then 'val next = nextRetryWallMillis(faulted.releaseFaultedAtWallMillis, done); if (next != null) scheduleNext(appContext, next)' (lines 92-99), and scheduleNext blindly calls 'alarmManager.setAndAllowWhileIdle(AlarmManager.RTCWAKEUP, atWallMillis, intent)' (line 42) — an alarm set in the past fires immediately. Any late trigger (lost alarm, Deck-open hours/days later, or the stale-row case above) compresses the '+5min' retry into seconds after the first attempt: an immediate extra RTCWAKEUP wakeup and a second stop attempt with no cadence spacing at all.

• [MEDIUM] Unbounded full-history scan per retry trigger and a full 48h reconcile on every 30s heartbeat
  - Location: app/src/main/java/com/example/service/ReleaseRetryScheduler.kt:70-74
  - Evidence: runDueRetry loads the entire history to find one row: 'dao.getSessionHistorySnapshot()' is 'SELECT * FROM focus_sessions ORDER BY startWallTimeMillis DESC' with no WHERE on releaseFaultShield and no LIMIT (MonolithDao.kt:77-78), executed on every alarm, VM init, and Deck-open — cost grows with every session the user ever records. The heartbeat has the same shape: every pass (driven ~every 30s by the shield loop, MonolithShieldService.kt:88-95, plus opportunistic service callbacks) re-reads all routines, derives over a 48h session lookback (RoutineHeartbeat.kt:47, 95-105), and probes/rearms alarms, with no cheap 'nothing due' early exit.

seal-watchstore

Verdict: This slice is elaborate ceremony over fragile state: WatchStore persists nothing at all and its poke() cannot emit, while the seal transition guards its exactly-once Throw with an unsynchronized set, a single-slot marker committed before the Throw, and annulment tombstones that evaporate on process death — and SessionSealTransitionImpl's prove/recover/executeSeal have no direct tests at all (SealThrowIdentityTest only tests audio rendering, ArmProvingControllerTest uses a fake).

• [HIGH] WatchStore has zero persistence — the 'watch state persistence' slice is an in-memory singleton
  - Location: app/src/main/java/com/example/service/WatchStore.kt:43
  - Evidence: Header admits it: 'Persistence: NONE this wave. The store is in-memory behind this writer interface... nothing in this file touches Room.' WatchSnapshot, lastProvedWallMillis, residency stamps and all nine measurements live only in a MutableStateFlow in a process singleton object (WatchStore.kt:152-156); process death wipes every watch fact and LAST PROVED back to EMPTY/UNMEASURED, and the promised Slice E swap-in does not exist at this commit.

• [HIGH] poke() is a no-op: StateFlow equality conflation swallows the copy()
  - Location: app/src/main/java/com/example/service/WatchStore.kt:192-193
  - Evidence: fun poke() { snapshots.value = snapshots.value.copy() } — WatchSnapshot is a data class, so copy() with no argument changes is equal to the current value, and MutableStateFlow suppresses equal values: collectors are never notified. The KDoc claims it 'forces a snapshot re-emission so derived readers (the Deck warrant) re-read the living facts' — it cannot. It is also written outside the @Synchronized publish() lock, so it can clobber a concurrent publish. Callers in MonolithShieldService.kt:72,157 and MonolithAccessibilityService.kt:175 rely on it.

• [HIGH] executeSeal commits the seal marker before publish()/Throw with no guard — a throwing publish leaves a false 'sealed and heard' marker forever
  - Location: app/src/main/java/com/example/service/SessionSealTransition.kt:140-142

  - Evidence: executeSeal runs writeSealMarker(sessionId) (synchronous commit, lines 117-124), then publish(), then engine.playSealThrow() with no runCatching around the last two. If publish() throws, the on-disk marker already testifies the seal landed and the Throw sounded; the claim stays spent (by design, lines 133-138), and recover() at SessionSealTransition.kt:185-188 will adopt the session silently on the marker — a seal that never sounded is permanently recorded as heard, violating the file's own 'a seal that cannot complete does not sound / marker = proof the Throw sounded' invariant.

• [HIGH] Annulment tombstones are memory-only while the annulled row is deleted — a refused routine window resurrects after process death
  - Location: app/src/main/java/com/example/service/SessionSealTransition.kt:91
  - Evidence: tombstones is a plain in-memory mutableSetOf<Pair<Int,Long>> (line 91), populated only in prove()'s annulment path (lines 251-253); annulment also deletes the row via repository.annulProvingSession, leaving no on-disk trace. RoutineHeartbeat.kt:147 checks isTombstoned before materialising, and its own comment concedes the window 'stays refused for this process's life' — after any process restart the tombstone is gone, the row is gone, and the heartbeat can materialise and re-prove the exact window the machine already refused.

• [MEDIUM] Sealed-set bookkeeping is racy and the persistent marker holds exactly one session
  - Location: app/src/main/java/com/example/service/SessionSealTransition.kt:118
  - Evidence: writeSealMarker does 'sealed += sessionId' (line 118) with no synchronization, while isSealed reads the same HashSet under synchronized(sealed) (lines 145-147) and claim() synchronizes on a different object (claimed, line 113) — concurrent executeSeal/recover calls race on an unsynchronized HashSet. Compounding it, the persistent marker is a single SharedPreferences int (MonolithRepository.kt:188-190, SEALMARKERSESSION_KEY), so sealing a second session overwrites the first session's only on-disk proof.

sound-engine

Verdict: The sound engine leaks audio focus forever, leaks threads and playing AudioTracks for engines nobody ever stops, and its 'persisted fact' is a non-volatile public var pushed even when the disk commit fails — elaborate machinery that does not actually control the resources or the setting it claims to own.

• [CRITICAL] Audio focus is acquire-only theatre: result ignored, no listener, never abandoned, absent on minSdk 24-25
  - Location: app/src/main/java/com/example/service/TactileSoundEngine.kt:521
  - Evidence: TactileSoundEngine.kt:521-536 requestTransientAudioFocus() discards the result (am.requestAudioFocus(req) return unused), sets no OnAudioFocusChangeListener, and is guarded if (Build.VERSION.SDKINT >= Build.VERSIONCODES.O) with no pre-O fallback while app/build.gradle.kts:17 sets minSdk=24, so API 24-25 get no focus at all. grep -rn abandon app/src returns zero hits: focus requested per-command (synth loop line ~458) and in primeForSeal() is never abandoned, and playSealThrow() (767ff) calls track.play() with no focus request on its un-primed synchronous fallback path.

• [HIGH] stopAllAudio() does not stop all audio: legacy track, focus and threads escape it, and the dead engine can still play
  - Location: app/src/main/java/com/example/service/TactileSoundEngine.kt:847
  - Evidence: stopAllAudio() (847-857) only sets running = false, clears the queue and stops/releases sealTrack. It never abandons audio focus, never interrupts/joins synthThread or sealRenderThread

(both poll on 500ms timeouts), and cannot touch the main AudioTrack — that track is a local variable in runFloatPcmSynthLoop() released only in its finally block after the loop notices running==false. The stopped engine stays in liveEngines, triggerWaveform() checks only hapticsEnabled (not running), and playSealThrow() can resurrect a seal track via ensureSealTrack() after stop.

• [HIGH] Every engine eagerly burns two threads and a playing AudioTrack, and the repository/seal engines are never shut down
  - Location: app/src/main/java/com/example/service/TactileSoundEngine.kt:332
  - Evidence: init (332-341) unconditionally starts two daemon threads per engine, the synth thread at Thread.MAX_PRIORITY, and runFloatPcmSynthLoop() builds an AudioTrack and calls track.play() at line 441 before any command exists — construction alone opens a playing track. Repository's lazy engine (MonolithRepository.kt:1831-1833 private val breakSoundEngine by lazy { TactileSoundEngine(context) }) and SessionSealTransition's singleton engine are never passed to stopAllAudio(); grep shows stopAllAudio is called only from MainActivity.onDestroy:150 and MonolithViewModel.onCleared:2458, on the same shared MainActivity engine. The WeakHashMap registry cannot collect them because their owners hold strong references.

• [HIGH] soundEnabled is a non-volatile public var: racy cross-thread reads and a direct-write bypass of the 'persisted fact'
  - Location: app/src/main/java/com/example/service/TactileSoundEngine.kt:291
  - Evidence: Line 291: var soundEnabled: Boolean = SoundEffectsSetting.isEnabled(context) — public, mutable, and not @Volatile, unlike the adjacent @Volatile running/legacyTrack/sealTrack fields (295-322). SoundEffectsSetting.setEnabled pushes writes inside synchronized(liveEngines) (TactileSoundEngine.kt:62-66) but the synth thread reads soundEnabled unsynchronized at line 456, so there is no happens-before guarantee the toggle is ever seen. Being public, it is also written directly, bypassing the persisted fact and the live-engine push, at SessionSealTransition.kt:105: engine.soundEnabled = SoundEffectsSetting.isEnabled(appContext).

• [MEDIUM] Settings consistency is faked twice: commit() failure is ignored, and hapticsEnabled is an unpersisted always-true stub
  - Location: app/src/main/java/com/example/service/SoundEffectsSetting.kt:52
  - Evidence: SoundEffectsSetting.kt:52-60: setEnabled() calls .commit() and throws away its Boolean result, then unconditionally calls pushSoundEnabledToLiveEngines(enabled) — a failed disk commit still flips every live engine (and MonolithViewModel.toggleSound sets its StateFlow from the same unchecked call), so the app reports the new setting until process death silently reverts it. In the same slice, TactileSoundEngine.kt:292 var hapticsEnabled: Boolean = true has zero assignments anywhere in app/src (grep hapticsEnabled = finds none), no SharedPreferences backing and no toggle: a phantom setting that can never be turned off, unlike the sound fact beside it.

deck-screen

Verdict: A 1,148-line, ~60-parameter composable where roughly a third of the API is dead wiring, real state is mutated mid-composition, and the one honesty-critical freshness check runs on a clock nothing observes — ornate, fragile, and far less functional than its signature claims.

• [HIGH] ~20 parameters are dead: no intention field, profile picker, checklist, stats, defer/extend/quick-add, or vault entry is ever rendered
  - Location: app/src/main/java/com/example/ui/screens/TactileDeckScreen.kt:96-140
  - Evidence: grep of the file shows studyPreset, checklist, completedTasksCount, totalQueueMinutes,

selectedProfileName, intentionText, totalFocusedMinutes, totalInterceptions, profiles, apps, shortsReelsSniperEnabled, browserUrlBlockerEnabled (lines 100-113) and onToggleChecklistItem, onAddChecklistItem, onDeferActiveTask, onExtendActiveTaskOvertime, onAddQuickTaskToQueue, onIntentionChanged, onProfileSelected, onOpenCurioVault (lines 120-130) occur only in the parameter list and nowhere in the 1,148-line body; the only hit for onSelectStudyPreset is the FLOW chip at line 722. MainActivity.kt:470-524 wires all of them to real ViewModel methods (viewModel::setIntentionText, viewModel::selectAndApplyProfile, viewModel::deferTask, navigateTo(VAULT), etc.).

• [HIGH] Snapshot state is mutated during composition, not in effects or event handlers
  - Location: app/src/main/java/com/example/ui/screens/TactileDeckScreen.kt:419-501
  - Evidence: In the composable body: 'if (pressLatch != null || commitInFlight) { pressLatch = null; commitInFlight = false }' (line 425), 'fbFrozenKey = activeSession.id; fbFrozenCount = appsCellCount...' (lines 431-436), an else-branch resetting all five fbFrozen* saveables (line 447), and 'seenArmResetKey = armResetKey' with latch clearing at lines 495-500. The comments admit this pattern already caused harm: lines 456-460 record that a body-level sealFreeze check 'threw at composition — the idle Deck crashed in the harness'.

• [MEDIUM] Seal-shot animation recomposes the entire screen every frame for 1050ms
  - Location: app/src/main/java/com/example/ui/screens/TactileDeckScreen.kt:240-262
  - Evidence: Driver: 'while (!shot.heroLanded) { withFrameMillis { }; shot.vertigoT = ...; shot.blurT = ...; if (phase != exitPhase) exitPhase = phase }' (lines 244-262), where vertigoT/blurT are 'var ... by mutableStateOf' on SealShotState (lines 1120-1121). The hero reads them in composition, not in a draw/layer phase: 'val vt = shot.vertigoT; val blurDp = (6f 4f shot.blurT * (1f - shot.blurT)).dp; ... .then(if (blurDp > 0.dp) Modifier.blur(blurDp) else Modifier)' (lines 587-596), so every animation frame re-executes the whole screen including both AndroidView update lambdas and all strip derivations.

• [MEDIUM] Warrant freshness is judged from a non-state clock read, so the 'never print stale LIVE' law fires only on incidental recompositions
  - Location: app/src/main/java/com/example/ui/screens/TactileDeckScreen.kt:353-371
  - Evidence: 'val nowMono = SystemClock.elapsedRealtime()' (line 370) feeds warrantPrintState (line 371), seamMarks (line 531) and refusalSentenceFor(deckWarrant, nowMono) (line 1074), i.e. the code's own 'a warrant older than the store's threshold prints UNPROVEN, never LIVE' law (lines 363-369). Nothing schedules a recomposition at that threshold: the only local clock is LaunchedEffect(Unit) at lines 354-361, which updates nowFrame once per minute, so at idle a stale warrant can keep printing LIVE for up to ~60s past its freshness deadline, purely by luck of unrelated recompositions.

• [MEDIUM] stepsExpanded is a conditional, unkeyed remember inside if (activeTask != null), so expansion state bleeds across tasks
  - Location: app/src/main/java/com/example/ui/screens/TactileDeckScreen.kt:784
  - Evidence: 'var stepsExpanded by remember { mutableStateOf(false) }' sits inside 'if (activeSession-running task block)' — actually inside 'if (activeTask != null)' at line 784, with no key on activeTask.id. When the active task is completed/deferred and the next queued task becomes activeTask without the branch leaving composition, the new task inherits the previous task's expanded/collapsed state; when activeTask briefly goes null the state is discarded entirely.

shield-matrix-screen

Verdict: A polished ledger façade over dead wiring: the sniper, browser-blocker and profile controls are plumbed in and never rendered, the site editor fakes success and loses drafts, and parts of the layout and empty states simply fail on a real phone.

• [HIGH] Seven wired-in controls are dead: sniper, browser blocker, and profiles have no UI at all
  - Location: app/src/main/java/com/example/ui/screens/ShieldMatrixScreen.kt:91
  - Evidence: Params declared at lines 91-109 (shortsReelsSniperEnabled, browserUrlBlockerEnabled, profiles, selectedProfileName, onSelectProfile, onToggleShortsReelsSniper, onToggleBrowserUrlBlocker) appear nowhere else in the file — grep returns only the declarations — yet MainActivity.kt:557-581 wires them to real VM functions (viewModel::toggleShortsReelsSniper, viewModel::toggleBrowserUrlBlocker, viewModel::selectAndApplyProfile). Worse, the 'BLOCKED WEBSITES' header at line 542 prints unconditionally, so when browserUrlBlockerEnabled=false the screen still claims sites are blocked while offering no switch to turn the blocker on.

• [HIGH] Blocked-site ADD fakes success: input is cleared even when the domain is rejected
  - Location: app/src/main/java/com/example/ui/screens/ShieldMatrixScreen.kt:611
  - Evidence: Lines 611-613: 'if (newSiteText.isNotBlank()) { onAddSite(newSiteText); newSiteText = "" }' — the field clears unconditionally with no success/failure path back. The callee can reject input: MonolithRepository.kt:794-796 'normalizeDomain(raw) ?: return false', and MonolithViewModel.kt:1552-1556 swallows that Boolean. Type garbage, tap ADD, text vanishes as if saved, nothing is added, no error is shown.

• [MEDIUM] Website draft state lives in a disposable LazyColumn item and is lost on scroll
  - Location: app/src/main/java/com/example/ui/screens/ShieldMatrixScreen.kt:567
  - Evidence: Line 567, inside a LazyColumn item {} block: 'var newSiteText by remember { mutableStateOf("") }'. It is plain remember, not rememberSaveable, and LazyColumn disposes off-screen items — scroll away from the bottom section (e.g. up the long app catalogue) and the half-typed domain is silently destroyed.

• [MEDIUM] Expanded routine editor row cannot fit a phone width and has no scroll or wrap
  - Location: app/src/main/java/com/example/ui/screens/ShieldMatrixScreen.kt:730
  - Evidence: Lines 730-753: a plain Row (no horizontalScroll, no wrap) packs 'START' + '−30' + '+30' + 'END' + '−30' + '+30' + a loadout chip (BlockitActionChip is 42dp high with 12dp horizontal padding per MonolithCustomComponents.kt:199-206). On a ~360dp-wide phone minus the LazyColumn's 12dp horizontal padding (line ~256) this overflows the right edge, pushing the loadout-cycle chip off-screen/unreachable — unlike the category row in the same file, which explicitly uses horizontalScroll.

• [LOW] Empty-by-category catalogue renders a silent void; the no-match line only fires for search
  - Location: app/src/main/java/com/example/ui/screens/ShieldMatrixScreen.kt:519
  - Evidence: Line 519: 'if (filteredApps.isEmpty() && searchQuery.isNotBlank())' guards the only empty-catalogue message ('NO APP BY THAT NAME ON THIS PHONE'). Pick a category with zero apps and no search text and the screen prints 'SHOWING 0' above a void — contradicting the file's own stated principle at SlipNoneLine: 'a missing section reads as a bug; a printed NONE is a fact.'

drawer-screen

Verdict: The Drawer's navigation is theatre: an ephemeral in-composable stack that system Back and rotation destroy, a search that can trap a sealed user, blank screens for loading/empty/error, and month scoping that doesn't actually scope rows.

• [HIGH] Fake internal navigation stack: ephemeral remember state and system Back ejects to DECK
  - Location: app/src/main/java/com/example/ui/screens/DrawerScreen.kt:164-166
  - Evidence: DrawerScreen.kt:164-166 holds the stack in plain remember: 'var view by remember { mutableStateOf(DrawerView.LANDING) }', 'selectedTask', 'cabinetArtifact' — no rememberSaveable, and AndroidManifest.xml:40-47 declares MainActivity with no configChanges, so rotation/process recreation resets to LANDING and drops the open task/artifact. DrawerScreen.kt contains no BackHandler at all, while MainActivity.kt:348-351 installs 'BackHandler { viewModel.navigateTo(MonolithDestination.DECK) }' for every non-DECK destination — system Back from a task detail, cabinet detail, INDEX or MONTH therefore ejects to DECK instead of popping one internal level.

• [HIGH] Sealed search is a trap: disabled input, no clear button, search hijacks every view
  - Location: app/src/main/java/com/example/ui/screens/DrawerScreen.kt:184,554-556
  - Evidence: DrawerScreen.kt:184 'val searching = drawerQuery.isNotBlank()' overrides the list views whenever the ViewModel-held query is non-blank, but DrawerScreen.kt:554-556 renders the only query editor as 'BasicTextField(value = drawerQuery, onValueChange = onQueryChange, enabled = !isSealed)' and the file contains no clear/reset control. Seal the app with a query still set and the user is locked into search results with a disabled field and no way back to LANDING/INDEX/MONTH from this screen.

• [HIGH] No loading or error state: month navigation renders a blank screen indistinguishable from empty or broken
  - Location: app/src/main/java/com/example/ui/screens/DrawerScreen.kt:134-160,648-705
  - Evidence: The composable signature at DrawerScreen.kt:134-160 accepts rows/results/booleans but no loading or error parameter, and the MONTH branch (DrawerScreen.kt:~648-705) renders only a header plus whatever groups exist. MonolithViewModel.kt:1887-1890 selectDrawerMonth() first sets '_drawerRows.value = emptyList()' then loads asynchronously — so loading, a genuinely empty month, and a failed/slow read are visually identical: a blank screen under a month label, with no spinner, empty-month line, or error text.

• [MEDIUM] buildDayGroups ignores scopeMonth for rows, and the day-boundary ticker desyncs the landing from its data
  - Location: app/src/main/java/com/example/ui/screens/DrawerScreen.kt:91-96,175-186
  - Evidence: buildDayGroups at DrawerScreen.kt:91-96 groups ALL rows unconditionally ('val byDate = rows.groupBy {...}'; 'val dates = byDate.keys.toMutableSet()') and applies scopeMonth only to day-word dates, never to row dates — so scopeMonth does not scope rows. Combined with the 03:30 ticker at DrawerScreen.kt:175-186 re-deriving 'today'/'currentYearMonth' locally while drawerMonth/drawerRows stay in the ViewModel untouched, a Drawer held open across the boundary re-labels/re-scopes yesterday's page as the new current month on LANDING, with drawerHasEarlier still belonging to the old month's pagination.

• [MEDIUM] All navigation is tiny bare-Text clickables with sub-48dp targets and no button semantics
  - Location: app/src/main/java/com/example/ui/screens/DrawerScreen.kt:231-239,866-877
  - Evidence: Every control is a bare Text with Modifier.clickable and no role, semantics or minimum size: detail BACK at DrawerScreen.kt:231-239 is 12sp text with clickable and zero padding, cabinet BACK at :354-371 likewise, MONTH BACK at :668-678 likewise, and the day-word toggles at :866-877 are 11sp text with clickable and no padding. These are a few dp tall — far under the 48dp touch target — and TalkBack users get unlabelled text, not buttons; DrawerNavRow (:820-831) only

adds 6dp vertical padding to 12sp text.

sequencer-screen

Verdict: This screen is a decorative queue with a lying seal: during an active session everything except DONE stays tappable, START and CUE are identical, half its callbacks and its entire deferred/duration surface are dead wiring, and sessions are launched for hardcoded 0-minute 'Study' tasks with no timer logic in the file at all.

• [HIGH] Session 'seal' is fake: only DONE/REOPEN are gated, all destructive/mutating controls stay live during an active session
  - Location: app/src/main/java/com/example/ui/screens/FocusSequencerScreen.kt:610,646,659-711
  - Evidence: Docs claim 'While a session is sealed the perforation is overprinted solid and the tear does not exist'. In code only tear (line 646 if (!isSessionActive)) and REOPEN (line 353) check isSessionActive. SPLIT line 664 Modifier.clickable(enabled = !ghost) { onAutoSplitTask(task.id) }, START/CUE lines 687/699 clickable(enabled = !ghost), DEL line 711 clickable(enabled = !ghost) { onDeleteTask(task.id) }, and sub-step toggle line 610 clickable(enabled = !ghost) { onToggleSubStep(task.id, stepIdx) } ignore isSessionActive entirely — the supposedly sealed head task can be deleted, re-cued, split and edited mid-session.

• [HIGH] Nine declared inputs/callbacks are dead: edit, reorder, promote, defer, duration, add-substep, dismiss-undo and the deferred queue can never happen here
  - Location: app/src/main/java/com/example/ui/screens/FocusSequencerScreen.kt:75-94
  - Evidence: grep shows these names occur only in the parameter list, never in the body: deferredTasks (75), totalQueueMinutes (77), onEditTask (82), onUpdateTaskDuration (83), onPromoteTask (86), onMoveTaskOrder (87), onDeferTask (88), onDismissUndoBanner (91), onAddSubStep (94). Total is recomputed locally as orderedQueue.sumOf { it.estimatedMinutes } (line ~132) instead of using totalQueueMinutes, deferred tasks are rendered nowhere, and the undo banner (lines ~272-303) has an UNDO chip but no dismiss control at all.

• [HIGH] START and CUE are the same action under two different verbs
  - Location: app/src/main/java/com/example/ui/screens/FocusSequencerScreen.kt:667-702
  - Evidence: Head leaf START (lines 687-690): onCueTaskToFocus(task.id); onNavigateToDeck(). Non-head CUE (lines 699-702): identically onCueTaskToFocus(task.id); onNavigateToDeck(). Only font, size and colour differ. CUE does not cue-in-place — it instantly abandons this screen for the Deck exactly like START, so the label lies about the consequence.

• [HIGH] Timer/duration is fiction in this slice: new tasks are created durationless with a fabricated category, and no timer exists in the file
  - Location: app/src/main/java/com/example/ui/screens/FocusSequencerScreen.kt:111-113,138-145
  - Evidence: submitTask (lines 138-145): onAddTask(clean, 0, "Study", "", orderedQueue.isEmpty()) with comment 'the composer asks NOTHING about duration — 0, no estimate given' — every task gets 0 minutes and hardcoded category 'Study' regardless of content, and the only duration-edit callback onUpdateTaskDuration is never called. The file's only timing code is animation cosmetics: delay(MonolithCurves.DURATION_260.toLong()) for the tear ghost (line 113) and 180/260ms animateFloatAsState specs (lines 502,507). START therefore launches a 'focus session' for a 0-minute task with no countdown, deadline or elapsed-time logic anywhere in this screen to be accurate about.

• [MEDIUM] All leaf actions, including instant DEL, are bare 12sp Text clickables with no padding, min

touch target, role or confirmation
  - Location: app/src/main/java/com/example/ui/screens/FocusSequencerScreen.kt:646-711
  - Evidence: SPLIT (line 659-664), CUE (694-701) and DEL (706-711) are Text(text="DEL", fontSize=12.sp, modifier=Modifier.clickable(enabled=!ghost){ onDeleteTask(task.id) }) — no padding/sizeIn, no Role.Button/semantics, tap area is the glyphs themselves, far below a 48dp target. DEL fires deletion in one accidental-sized tap; the same disabled-but-still-rendered texts are also drawn on the 260ms ghost leaf (enabled=!ghost only suppresses the click).

custom-components

Verdict: An 897-line god-file of print-faithful but inaccessible components: sub-48dp unlabelled click targets, tiny fixed-height type that breaks under font scaling, and hard-coded hex/luminance hacks instead of a real theme.

• [HIGH] All interactive components fail accessibility: sub-48dp targets, no semantics/role/labels
  - Location: app/src/main/java/com/example/ui/components/MonolithCustomComponents.kt:193-223,280-292,730-765
  - Evidence: Three clickables use bare Modifier.clickable with zero semantics/role/onClickLabel in the whole file (grep: semantics/contentDescription/Role = 0 hits, clickable at 205,291,765), and targets are undersized: BlockitActionChip '.height(42.dp)' + '.clickable(onClick = onClick)' (205), header guard line is a bare Text '.clickable { onGuardLineTap() }' (291), RefusalSentenceSlot Text clickable inside 'Box(modifier.fillMaxWidth().height(22.dp))' (~730/765) — 42dp and 22dp are both below the 48dp minimum, TalkBack users get an unlabelled text/box, not a button.

• [HIGH] Fixed heights + tiny hard-coded type (10sp/11.5sp) clip/bury text under font scaling
  - Location: app/src/main/java/com/example/ui/components/MonolithCustomComponents.kt:145,273,614-630,752-765,860-875
  - Evidence: Fixed containers vs scalable text: ProofStripBand '.height(53.dp)' with 15.sp/20.sp lineHeight cells (614-625), RefusalSentenceSlot '.height(22.dp)' with 'fontSize = 15.sp, lineHeight = 20.sp' (752-755), ImprintSlot 'Box(Modifier.fillMaxWidth().height(20.dp))', header 'fontSize = 10.sp' (273) and FullBleedSectionHeader 'fontSize = 11.5.sp' (145,158). At 1.3-2.0x fontScale or for low-vision users the 10/11.5sp print is illegible and the 15sp print overflows/clips its fixed box; no maxLines/overflow handling on StripCell/Imprint texts.

• [HIGH] Hard-coded hex colours and duplicated luminance() night-detection bypass the theme system
  - Location: app/src/main/java/com/example/ui/components/MonolithCustomComponents.kt:193,315,321,553-562,846
  - Evidence: grep 0xFF: 193 'isAccentArmed -> Color(0xFFFBF1EC)', 315 'Color(0xFF4A423C)', 321 'Color(0xFFFFE4CF) else Color(0xFFFBF1EC)' (same hex duplicated), 560/562 both 'Color(0xFF9C8E84)'. Night mode is not a theme flag but 'scheme.background.luminance() < 0.5f' repeated at 315,321,553,846 — any theme whose background crosses 0.5 luminance silently flips every control/strip/lip colour, and a token change must be hunted in 4+ places.

• [MEDIUM] 897-line god-file: business logic, theme, animation and UI are inseparably coupled
  - Location: app/src/main/java/com/example/ui/components/MonolithCustomComponents.kt:1-897

  - Evidence: wc -l = 897 lines in one file containing: UI (FullBleedSectionHeader, BlockitActionChip, MonolithPullDownHeader, ProofStripBand, ImprintSlot), pure domain logic (seamMarksFor x2, endsAtText, imprintTenantText/splitTenantText, refusalSentenceFor x2, jamSentenceFor, RefusalAction/RefusalSentence/StripDensity), theme helpers (inkControlFill/onInkControlFill/stripToneColors) and imports spanning 'com.example.service.Warrant/WatchStore/UsageAccess', 'SystemGuardStatus' and Android Settings. Pure warrant/proof logic cannot be reused or tested without pulling the entire Compose/animation/platform file.

• [MEDIUM] Fake remember* API, per-recomposition TextStyle allocation, and platform I/O inside a UI component
  - Location: app/src/main/java/com/example/ui/components/MonolithCustomComponents.kt:75-120,614-625,800-830
  - Evidence: Line 76-77: '@Composable fun rememberMixedTypographyText(...)= buildAnnotatedString {' — named like a memoizing remember* API and marked @Composable, but it calls no remember/cache at all, rebuilding the AnnotatedString on every recomposition and misleading every caller (used in 3 other files). Same anti-pattern in StripCell/RefusalSentenceSlot constructing new 'TextStyle(fontFamily=..., fontSize=15.sp, lineHeight=20.sp)' inline per recomposition (614-625,752-755) instead of MaterialTheme.typography, and ImprintSlot reading 'Settings.Global.getFloat(... ANIMATORDURATIONSCALE)' via LocalContext inside LaunchedEffect (~810-820), hard-wiring an untestable platform check into a presentational component.

airlock-overlays

Verdict: The airlock/disarm overlays are theatre, not a lock: they don't consume touches, don't handle Back, freeze the peek offer on a stale snapshot that strands users at '0 MIN', ship dead reason-selection wiring that fabricates analytics, and don't even restart their own 5-second wall per intercept.

• [CRITICAL] Neither overlay is actually modal: touches pass through the 'wall' to the UI underneath
  - Location: app/src/main/java/com/example/ui/components/AirlockAndDisarmOverlays.kt:588,745 (contrast :326,:500)
  - Evidence: Airlock root Box (588) is only .fillMaxSize().background(NightColorBackground).padding(20.dp).testTag(...) and its Column (596ff) has clip/background/border/padding — no clickable/pointerInput consuming touches. Disarm root Box (745) and Column (755ff) are identical in shape. The same file proves the author knew the guard: the digest sheet (326) and primer modal (500) both add .clickable(enabled = false) {} to their inner Column to block pass-through. A background modifier does not consume pointer events in Compose, so taps in the large empty area around the card fall through to the Deck/tabs rendered underneath in MainActivity — the 'modal' wall can be operated through.

• [HIGH] System Back bypasses both overlays: no BackHandler, so Back exits the activity toward the blocked app
  - Location: app/src/main/java/com/example/ui/components/AirlockAndDisarmOverlays.kt:550,732; MainActivity.kt:347-351
  - Evidence: The slice defines no BackHandler for either overlay (grep: no BackHandler in the file). MainActivity.kt:347-351 only installs BackHandler when parkOpen or currentDestination != DECK — neither condition is tied to airlockState/disarmChallenge visibility, and both overlays commonly stand

over the DECK. MainActivity is launchMode=singleTop (AndroidManifest.xml:43) launched by the shield with FLAGACTIVITYNEW_TASK (MonolithShieldService.kt:182), so an unhandled Back finishes this activity and returns the user to the previous task — the intercepted app — skipping the 5s wall and the peek gate entirely, with only an unproven re-intercept to save it.

• [HIGH] Cooldown trap: locally-ticked snapshot never unlocks the peek and degrades to 'AGAIN IN 0 MIN'
  - Location: app/src/main/java/com/example/ui/components/AirlockAndDisarmOverlays.kt:577-586,663,690-693, 233-235
  - Evidence: Cooldown is seeded once from a snapshot (remember(...){ mutableIntStateOf(state.peekCooldownRemainingSeconds) }, 577) then decremented by its own delay(1000L); cooldownSecs-- loop (580-586) — never re-read from the repository lease end. The peek offer is gated on the frozen state.canUse2MinutePeek (663), which nothing in the overlay ever recomputes. So when the local counter reaches 0 the peek still never appears in a standing airlock, and the withheld branch (690-693) prints peekCooldownPrint(0), which by the helper at 233-235 ((0+59)/60 = 0) renders 'PEEK COOLDOWN — AGAIN IN 0 MIN' — a self-contradictory dead end the user can only escape by leaving and re-triggering the intercept.

• [MEDIUM] Reason-selection and transmute features are dead wiring: parameters accepted, never used, no UI exists
  - Location: app/src/main/java/com/example/ui/components/AirlockAndDisarmOverlays.kt:550-556
  - Evidence: onSelectReason and onTransmuteParticleTap appear exactly twice in the file — the parameter declarations at 552-553 — and are never invoked or bound to any UI element; state.selectedReason and state.transmutedParticles are never even read. MainActivity.kt:795-796 dutifully wires viewModel::selectDistractionReason and viewModel::incrementAirlockParticlePulse into dead parameters. With no reason UI composed, every intercept is recorded with the ViewModel default DistractionTriggerReason.MUSCLE_MEMORY (MonolithViewModel.kt:163), so the trigger-reason analytics this flow exists to collect are silently fabricated defaults.

• [MEDIUM] 5s wall is not per-intercept: same-package re-intercept inherits an expired countdown and gets an instant peek
  - Location: app/src/main/java/com/example/ui/components/AirlockAndDisarmOverlays.kt:562-570,663
  - Evidence: Countdown is remember(state.interceptedPackageName, state.isVisible) (562) and its LaunchedEffect is keyed only on (isVisible, interceptedPackageName) (565); AirlockInterceptState carries no intercept-instance id or timestamp (MonolithViewModel.kt:153-181). A fresh intercept for the same package delivered while the overlay already stands (onNewIntent -> triggerAirlockIntercept replaces the state) changes neither key, so the countdown is not restarted: if it already reached 0, the EMERGENCY PEEK chip (663: countdownSecs == 0 && state.canUse2MinutePeek) is available instantly, and the 5-second breathing wall is skipped. The tick-counting while(countdownSecs>0){ delay(1000L); countdownSecs-- } loop is also not deadline-based, so the '5s' is whatever scheduling drift makes it.

park-drawer

Verdict: ParkDrawer looks meticulously specified but its core safety gate is theatre: interactive guards only the FIX key while every persisted-setting sled stays live mid-animation and on any stray ACTION_UP, and its drag and FIX-return logic are built on racy, unbound coroutine state.

• [HIGH] All three sleds bypass the interactive gate — toggles fire mid-rise, mid-throw, and while closing
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:161-162,433,496,565,734
  - Evidence: Header spec at line 93 states 'Rows are inoperable mid-rise', and lines 161-162 define val settled = fraction.value == 0f / val interactive = visible && settled && !isSealed — but grep shows interactive is used exactly once, at line 433 for the FIX key. The three sled AndroidViews (lines 496, 565, 734) are gated only by an if (isSealed) overlay (lines 517, 586, 755). Since line 159 only returns when !visible && fraction.value >= 1f, the sleds also stay live throughout the 160ms close throw after visible=false.

• [HIGH] Grip drag launches an unordered coroutine per drag event; the 40% dismiss law reads a possibly stale fraction
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:250-275
  - Evidence: onVerticalDrag (lines 269-274) does scope.launch { fraction.snapTo((fraction.value + dragAmount / h).coerceIn(0f, 1f)) } — a fresh, unordered coroutine per drag event competing for the same Animatable. onDragEnd (lines 252-267) then reads fraction.value > 0.4f to decide dismissal while those snapTo launches may still be queued, and its re-seat scope.launch { fraction.animateTo(0f, ...) } races them too. A fast fling can be judged on a stale fraction and re-seat instead of dismissing, or vice versa.

• [MEDIUM] Sleds toggle on any ACTION_UP — a sheet scroll ending on a sled flips the setting
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:301,496,565,734
  - Evidence: The sleds sit inside the sheet's .verticalScroll(rememberScrollState()) column (line 301), but the embedded SledSwitchView (BlockitSuitcaseAndSwitchesViews.kt:168-175) implements onTouchEvent as: if (event.actionMasked == MotionEvent.ACTIONUP) { fireFeel(8L); onToggleRequested?.invoke(); return true } return true — no ACTIONDOWN tracking, no bounds/slop check, no ACTION_CANCEL handling. Any gesture that ends over a sled, including a scroll of the sheet that started on it, toggles a persisted setting (sound, Shorts+Reels sniper, or the browser blocker).

• [MEDIUM] FIX return verdict is not bound to its trip — a second press within 800ms is judged and cleared by the first trip's timer
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:191-210
  - Evidence: Lines 191-210: LaunchedEffect(parkReturnTick) { if (parkReturnTick <= 0 || !fixPressed) return@LaunchedEffect; delay(800L); val item = fixPressedItem ?: return@LaunchedEffect; ...; fixPressed = false; fixPressedItem = null }. The effect captures no item/trip identity; after the delay it judges and clears whatever fixPressedItem happens to hold then. A second FIX press inside the 800ms window gets its dip released and its verdict (including the firm click at lines 203-207) computed by the first trip's timer, against a state it did not wait for.

• [LOW] Scrim stops blocking touches the instant dismissal starts, while it is still visibly covering the screen
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:159,215-235
  - Evidence: The scrim Box applies its dismiss clickable only if (visible) (the Modifier.then branch in the scrim block after line 215), yet the composable does not return until !visible && fraction.value >= 1f (line 159). For the entire close animation the dimmed scrim (alpha 0.32f * (1f - fraction.value)) is still drawn over the underlying screen but no longer consumes touches, so taps fall through the visible scrim onto the screen beneath while the sheet is still on its way down.

receipt-sheet

Verdict: ReceiptSheet is a typographic facade over real ReceiptState data that omits the actual elapsed duration, prints a provably false '0 MIN' for sub-minute banked breaks, inflates every planned duration, and strips all dates from its times — it looks like a record and misreports like a summary.

• [HIGH] Receipt never prints the actual elapsed duration — its central fact is dropped
  - Location: app/src/main/java/com/example/ui/components/ReceiptSheet.kt:121
  - Evidence: ReceiptState carries sealedElapsedSeconds (ReceiptState.kt:180, populated from row.spanElapsedSeconds in MonolithRepository.kt:1642), but ReceiptSheet.kt never references it: lines 121-149 print only ARMED/ENDED wall times, PLANNED, HELD, BREAK BANKED and deflections. A user must subtract two HH:mm clock times — impossible across midnight — to guess the actual studied duration.

• [HIGH] BREAK BANKED prints a false 0 MIN / truncated minutes via integer division
  - Location: app/src/main/java/com/example/ui/components/ReceiptSheet.kt:138
  - Evidence: Line 140: "BREAK BANKED ${receipt.bankedBreakSeconds / 60} MIN" guarded only by bankedBreakSeconds > 0 (line 138). 1-59 banked seconds print 'BREAK BANKED 0 MIN' and 119 seconds print '1 MIN' — integer division fabricates/truncates the figure on a document that claims to be an exact record.

• [MEDIUM] PLANNED duration is silently rounded up to the next minute
  - Location: app/src/main/java/com/example/ui/components/ReceiptSheet.kt:211
  - Evidence: longSpan does val minutes = (seconds + 59) / 60 (line 211) and the ARMED line prints PLANNED ${longSpan(planned)} (line 124). A 61-second plan prints 'PLANNED 2 MIN' and a 3,601-second plan prints 'PLANNED 1H 01M' — the receipt systematically overstates the plan that was actually set.

• [MEDIUM] ARMED/ENDED are bare HH:mm with no date — cross-midnight receipts are ambiguous and misleading
  - Location: app/src/main/java/com/example/ui/components/ReceiptSheet.kt:203
  - Evidence: private fun hhmm(wallMillis: Long) = SimpleDateFormat("HH:mm", Locale.US).format(Date(wallMillis)) — no date, no day offset, and the JVM default (current) timezone applied retroactively to both instants. Lines 124/130 print only 'ARMED 23:50' / 'ENDED 00:10' style times, so a cross-midnight, multi-day, or timezone-changed session reads as ending before it started, with no way to tell which day either time belongs to.

• [MEDIUM] Dismissal/animation spec is fake: AnimatedVisibility is hardcoded visible and dismiss is on click-up, not DOWN
  - Location: app/src/main/java/com/example/ui/components/ReceiptSheet.kt:60
  - Evidence: Header (lines 44-46) promises the slip 'dismisses on the next DOWN anywhere' and defines enter/exit fades, but line 61 hardcodes visible = true (exit fadeOut at line 63 can never run — the caller removes the composition) and line 73 uses .clickable(onClick = onDismiss), which fires on click release/up, not on DOWN. Press-and-hold on the scrim does not dismiss when the spec says it must.

shield-cap-guardline

Verdict: GuardLine's core counting is honest and tested, but ShieldCapSelector is an untested, accessibility-dead control with lying tap feedback, and the guard print inherits a stale-denial blind spot

and a hardcoded denominator.

• [HIGH] A MISSING permission never goes stale, so the guard line can present an ancient denial as current fact forever
  - Location: app/src/main/java/com/example/ui/components/GuardLine.kt:40-45
  - Evidence: GuardLine.kt only distinguishes STALE vs bare name: 'PermPrint.STALE -> "${firstLack.first.boardName} STALE"; else -> firstLack.first.boardName'. But WatchStore.permPrintAt (WatchStore.kt:359-362) only ever returns STALE for GRANTED: 'perm == PermState.GRANTED && nowMono - m.measuredMono > STALEAFTERMS -> PermPrint.STALE ... else -> PermPrint.MISSING'. A MISSING measured hours ago prints bare and current indefinitely, while a GRANTED expires after 45s — asymmetric honesty baked into the render path.

• [HIGH] Selector stops are accessibility-dead: bare clickable Boxes with no label, role, or selected state
  - Location: app/src/main/java/com/example/ui/components/ShieldCapSelector.kt:163-176
  - Evidence: Tap zones are plain Boxes: '.clickable { soundEngine?.playFirmClick(); onSelect(i) }.testTag("shieldcapstop_$i")' — no semantics{}, no Role.RadioButton/Tab, no selected state, no contentDescription. The visible words live in entirely separate layers (track Row at lines 113-133, cap Text at line 141), so TalkBack gets three unlabeled click targets and no way to announce which loadout is selected.

• [MEDIUM] Tap targets are only 36dp tall, below the 48dp minimum touch target
  - Location: app/src/main/java/com/example/ui/components/ShieldCapSelector.kt:98-102
  - Evidence: The track Box is '.height(36.dp)' and the tap-zone Row is Modifier.fillMaxSize() inside it, so each stop's hit area is 36dp high despite the root slot being 84dp (line 81) — the extra 48dp of slot height is wasted instead of extending the hit area.

• [MEDIUM] Re-tapping the already-selected stop plays the firm click and re-fires onSelect, faking a change and re-applying the loadout
  - Location: app/src/main/java/com/example/ui/components/ShieldCapSelector.kt:166-169
  - Evidence: The clickable is unconditional: 'soundEngine?.playFirmClick(); onSelect(i)' with no i != selectedIndex guard. The caller (ShieldMatrixScreen.kt:287-295) treats every onSelect as a real pick — it sets selectedLoadoutIndex and calls onApplySmartLoadout(STUDY/WORK_SPRINT/SLEEP) — so a no-op tap on the resting cap produces change-feedback sound plus a full loadout re-application.

• [MEDIUM] GuardLine hardcodes the denominator '4' in both print strings while comparing against COUNTED.size
  - Location: app/src/main/java/com/example/ui/components/GuardLine.kt:39-45
  - Evidence: 'if (warranted == WatchItemId.COUNTED.size) return GuardLinePrint("GUARD 4/4")' and 'return GuardLinePrint("GUARD $warranted/4 — $name")'. The whole-check is size-driven but the printed fraction is a literal: add a fifth counted item in WatchStore.COUNTED and the line would print 'GUARD 4/4' for 5/5 warranted and 'GUARD 5/4' for a lack — a false fraction by construction.

• [LOW] ShieldCapSelector accepts an unbounded selectedIndex and has zero test coverage for its duplicated index mapping
  - Location: app/src/main/java/com/example/ui/components/ShieldCapSelector.kt:62,141
  - Evidence: No require/coerce anywhere: 'val words = listOf("STUDY", "WORK", "SLEEP")' and 'text = words[selectedIndex]' means any index outside 0..2 crashes with IndexOutOfBoundsException, and capOffset (lines 91-95) would animate the cap off the track for out-of-range values. The

index↔loadout mapping is duplicated in three places (this words list, LoadoutStore.capIndexOf, ShieldMatrixScreen's 'when (level) { 0 -> STUDY; 1 -> WORK_SPRINT; else -> SLEEP }' where else silently means SLEEP for any garbage index), and grep of app/src/test and app/src/androidTest finds zero references to ShieldCapSelector or its testTags — unlike GuardLine, which has four dedicated tests in ChromeWaveUnitTest.kt.

blockit-core-views

Verdict: A 2,416-line monument to spec-comment theatre: it claims zero-allocation and disciplined lifecycles while allocating in onDraw, redrawing ~500 canvas ops per frame, leaking animators/trackers on detach, and shipping zero accessibility, measurement or saved-state support.

• [CRITICAL] RotaryWheelView lifecycle: detach starts an animator and leaks the other animators + VelocityTracker
  - Location: app/src/main/java/com/example/ui/views/BlockitCoreViews.kt:1294-1301
  - Evidence: onDetachedFromWindow() only calls stopPhysics() then 'if (hubPressDepth > 0f) startHubReturn()' — it STARTS a new ValueAnimator on a detached view, and never cancels closureAnimator (created L396) or hubPressAnimator (L462/L480), nor recycles velocityTracker (obtained L1050, recycled only on UP/CANCEL L1120). Detached view keeps animating/invalidate()-ing and leaks the tracker.

• [CRITICAL] RotaryWheelView.onDraw is a per-frame performance catastrophe: ~500 segmented canvas calls
  - Location: app/src/main/java/com/example/ui/views/BlockitCoreViews.kt:602-657
  - Evidence: Socket loop 'while (socketDeg < 360f) { socketDeg += 3f }' = 120 iterations x 2 drawArc = 240 arcs, plus chamfer loop 65.2..245.2 step 2 (~90 drawArc), knurl loop 0..360 step 3 (120 drawLine), 35 tick drawLine, and hub-core wedge fan — each with per-segment cos() trig and Paint color churn. This entire ~500-call scene is redrawn on every invalidate() from physics (L1291 postFrameCallback) and every animator tick.

• [HIGH] 'Zero allocation in onDraw' is a lie — onDraw allocates Path, List, arrays and Strings every frame
  - Location: app/src/main/java/com/example/ui/views/BlockitCoreViews.kt:55,657,884,930
  - Evidence: Header promises L55 'Pre-allocated final Paint, final RectF, final Path fields — zero allocation in onDraw()', but onDraw allocates: L657 'listOf(117f to 0x33FFFFFF, 126f to 0x2E000000)' (list + Pairs), L884 'val corePath = android.graphics.Path()', L930 'val wellAzimuths = floatArrayOf(-67.5f, ...)', L804 'm.toString()' for every numeral, plus a local 'fun inkWith' lambda. The central architectural claim of the file is false.

• [HIGH] No onMeasure, no saved state, no accessibility: custom views are touch-only black boxes
  - Location: app/src/main/java/com/example/ui/views/BlockitCoreViews.kt:60,986,1826
  - Evidence: grep of the whole 2416-line file finds no onMeasure, no onSaveInstanceState/onRestoreInstanceState, no performClick, no contentDescription/accessibility delegate, and only a single-arg 'View(context)' constructor (BlockitCanvasView L60). Both interactive views suppress lint instead: '@SuppressLint("ClickableViewAccessibility") override fun onTouchEvent' (L986, L1826). TalkBack/keyboard users cannot operate the wheel or the commit slider at all, wrap_content has no intrinsic size, and rotation/process-death loses wheelAngleDeg/displayed/sealed state.

• [HIGH] SlideToStartView spring-back double-fires onEnd on every cancel, including on detach

- Location: app/src/main/java/com/example/ui/views/BlockitCoreViews.kt:2361-2382
- Evidence: animateSpringBack registers the same callback twice: 'onAnimationEnd { onEnd?.invoke() }' and 'onAnimationCancel { onEnd?.invoke() }' (L2369-2376). Android cancel() fires BOTH callbacks, so the host's onPressEnd fires twice per cancel, and cancelling in onDetachedFromWindow (L2382 'springBackAnimator?.cancel()'), resetToRest(), beginAttempt() and animateSpringBack() itself spuriously signals press-end to the host mid-teardown/re-entry.

blockit-suitcase-views

Verdict: Three pretty canvas toys with no saved state, no accessibility, a switch that fires on any stray ACTION_UP, and an emergency valve whose headline 'banked' progress evaporates on recreation — decoration, not dependable controls.

• [CRITICAL] Emergency valve's 'banked' 15/30/45s progress is volatile and unrecoverable, with no lifecycle cleanup
- Location: app/src/main/java/com/example/ui/views/BlockitSuitcaseAndSwitchesViews.kt:553
- Evidence: held/banked progress is only in-memory fields: 'private var heldMs: Long = 0L' / 'private var bankedMs: Long = 0L' (553-554), resetHold() zeroes them, and grep finds no onSaveInstanceState or onDetachedFromWindow anywhere in this file. Any recreation, dismissal/recomposition of the overlay, rotation or process death silently destroys up to 45s of supposedly 'banked' progress, and the 32ms hold/drain runnables (postDelayed at 614/635) and the other views' ValueAnimators are never cancelled on detach.

• [HIGH] SledSwitchView toggles on ANY ACTION_UP, regardless of where the gesture started
- Location: app/src/main/java/com/example/ui/views/BlockitSuitcaseAndSwitchesViews.kt:168
- Evidence: onTouchEvent is only: 'if (event.actionMasked == MotionEvent.ACTIONUP) { fireFeel(8L); onToggleRequested?.invoke(); return true } return true' (168-176). There is no ACTIONDOWN tracking, no check the press started on the sled/track, no x/y bounds check, and no ACTIONCANCEL handling, so any UP delivered to the view toggles a safety blocker; the project's own test even documents this: 'SledSwitchView fires onToggleRequested on any ACTIONUP' (ShortsToggleRenderTest.kt:37).

• [HIGH] All three controls are accessibility-dead: not clickable, unlabelled, no performClick
- Location: app/src/main/java/com/example/ui/views/BlockitSuitcaseAndSwitchesViews.kt:167
- Evidence: All three controls override onTouchEvent under '@SuppressLint("ClickableViewAccessibility")' (SledSwitch 167, AppRow 431, Emergency 715) and grep finds no performClick override, no isClickable=true, no contentDescription/stateDescription in this file or its BlockitCanvasView base. Canvas-drawn text is invisible to TalkBack, so the blocker switch, the SHUT/PEEK/OPEN rocker and the 60s hold valve expose no name, state or action to assistive tech.

• [MEDIUM] Dead/misleading setters: chromeMode and requiredSeconds mutate state without invalidating or rescaling
- Location: app/src/main/java/com/example/ui/views/BlockitSuitcaseAndSwitchesViews.kt:69
- Evidence: 'var chromeMode: Boolean = false' (69) is a plain field while titleText/subtitleText/isChecked all have invalidating setters; assigning chromeMode after creation changes the flag but never redraws, so the PARK vs row furniture stays stale until an unrelated invalidate. Likewise Emergency 'var requiredSeconds: Int = 60' (534) has no setter: the Airlock caller assigns it in AndroidView update, but changing it neither invalidates nor rescales heldMs/bankedMs even though totalMs/bankIntervalMs derive from it, corrupting bank thresholds mid-hold. Related: sledProgress is hardcoded 1f (71) while isChecked's setter animates on set, so a view created with

isChecked=false visibly throws from ON on first appearance.

• [MEDIUM] AppRow rocker draw geometry and touch geometry disagree
  - Location: app/src/main/java/com/example/ui/views/BlockitSuitcaseAndSwitchesViews.kt:378
  - Evidence: onDraw computes the thumb slot as 'val slotW = (rockerW - pad * 2f) / 3f' with pad=3dp, i.e. 54dp (378), but drag handling uses 'val slotW = rockerW / 3f' = 56dp (460) and tap zoning uses '((event.x - rLeft) / (rockerW / 3f)).toInt()' ignoring pad entirely (492). Thumb travel, drag distance and tap-stop boundaries are computed from two different geometries, so a one-slot visual drag under-advances the index and tap zones are shifted from the drawn stops.

blockit-tower-views

Verdict: This slice is misleading by construction: a dead tower that caps its own math at 56 while claiming 70 and invents taps, paired with a 'proportional' bar that pads every estimate to 10 minutes and runs off-screen on long queues.

• [CRITICAL] Tower lies above 56 bricks: count/header claim up to 70, only 56 slots are ever drawn and the crown brick disappears
  - Location: app/src/main/java/com/example/ui/views/BlockitTowerFadersAndCatalogViews.kt:28,53,98
  - Evidence: Setter allows 70: 'val clamped = value.coerceIn(1, 70)' (line 28), but onDraw does 'val totalFloors = ((bricks + 6) / 7).coerceIn(1, 8)' (line 53) and loops only 'for (fl in 0 until displayFloors)' with max brickNumber 8*7=56. For 57-70 bricks the header still prints the raw count (line 79-86, bricks.toString()) and 'FL 08', no slot exists for bricks 57-70, and the crown test 'brickNumber == bricks' (line ~118) can never fire — the newest brick silently vanishes.

• [HIGH] onBrickTapped is fake: it never identifies a brick, fires on any tap anywhere, and consumes all touch events
  - Location: app/src/main/java/com/example/ui/views/BlockitTowerFadersAndCatalogViews.kt:143-151
  - Evidence: 'override fun onTouchEvent' fires only on ACTION_UP with 'onBrickTapped?.invoke(totalBricksCount)' (line 147) regardless of event.x/y — tapping any brick, the header, or empty card all report the total, never the tapped brick. It also 'return true' for every action (lines 148,150) including DOWN/MOVE/CANCEL, swallowing gestures, with @SuppressLint("ClickableViewAccessibility") and no performClick/contentDescription anywhere in the file.

• [HIGH] TowerStreakView is dead code with a fabricated default streak
  - Location: app/src/main/java/com/example/ui/views/BlockitTowerFadersAndCatalogViews.kt:26-28,52
  - Evidence: grep across app/src/main finds TowerStreakView only as an unused import at AirlockAndDisarmOverlays.kt:62 — no instantiation, no AndroidView, no XML reference. If it were instantiated with defaults it would also lie: 'var totalBricksCount: Int = 19' (line 26) clamped to minimum 1 (line 28) and redrawn as 'totalBricksCount.coerceAtLeast(1)' (line 52), so a zero-day streak can never be represented.

• [HIGH] TaskHorizonBarView is not proportional: every task is silently padded to at least 10 minutes
  - Location: app/src/main/java/com/example/ui/views/BlockitTowerFadersAndCatalogViews.kt:194,198,243
  - Evidence: Proportions use inflated weights: 'totalMins = tasks.sumOf { it.estimatedMinutes.coerceAtLeast(10) }' (line 194) and 'weight =

task.estimatedMinutes.coerceAtLeast(10) / totalMins' (line 198), while the printed label uses the raw value 'task.estimatedMinutes.toString()' (line 243). A 0m name-only task — which the comment at lines ~225-228 says must stand unfigured rather than wear a false figure — still gets a full 10-minute share, and a real 5m task is doubled; the strip is not proportional to the estimates it displays.

• [MEDIUM] Horizon bar overflows off-canvas with many tasks instead of degrading
  - Location: app/src/main/java/com/example/ui/views/BlockitTowerFadersAndCatalogViews.kt:193,254
  - Evidence: 'val availW = (w - pad 2f - gap (tasks.size - 1)).coerceAtLeast(dp(40f))' (line 193) forces a positive width even when gaps alone exceed the view, then 'curX += segW + gap' (line 254) advances past w with no task-count cap, truncation, or clipping logic. Total drawn width becomes pad2+availW+gap(n-1) > w, so with a long queue the later segments are drawn off the right edge and are invisible while still counted in totalMins.

rail-view

Verdict: The rail looks spec-precise at 390dp but is functionally rotten underneath: cancelled animations report completed navigations, the primary nav is invisible to accessibility, animators outlive the view, and hit-testing invents stops from the margins.

• [CRITICAL] Cancelling an animation fires its seat callback and teleports the cap
  - Location: app/src/main/java/com/example/ui/views/MonolithRailView.kt:166-192,313
  - Evidence: animateCapTo() starts with 'animator?.cancel()' (line 167) and ACTION_DOWN does the same (line 313), yet the only listener override is onAnimationEnd at lines 185-190: 'capPos = target; fireFeel(4L); ...; onSeat?.invoke()'. Android delivers onAnimationEnd after onAnimationCancel on cancel(), so interrupting a travel snaps the cap to the old target, buzzes, and fires the old onSeat — i.e. onTabSelected/onParkInvoked for a navigation the user just interrupted.

• [HIGH] Primary navigation is completely inaccessible: no semantics, no performClick, no focus
  - Location: app/src/main/java/com/example/ui/views/MonolithRailView.kt:282,307-360
  - Evidence: The whole control is canvas text — 'canvas.drawText(RailGeometry.WORDS[i], cx, wordBaseline, paintMono)' (line 282) — handled only in onTouchEvent under '@SuppressLint("ClickableViewAccessibility")' (line 307). The file contains no performClick, contentDescription, accessibility delegate/node info, focusable/clickable setup, or keyboard handling (grep finds none), and the AndroidView host in MainActivity.kt:701-737 sets none either. TalkBack/keyboard users cannot discover or activate DECK/SHIELD/TASKS/LEDGER/PARK at all.

• [HIGH] Animator is never cancelled on detach — post-detach navigation callbacks and a retained view
  - Location: app/src/main/java/com/example/ui/views/MonolithRailView.kt:145,166-193
  - Evidence: The class creates a running ValueAnimator holding 'target' and host lambdas in onSeat (lines 176-193) but defines no onDetachedFromWindow override anywhere in the file (grep: none). Nothing cancels/nulls 'animator' on detach, so a view removed mid-travel (recomposition, sheet/overlay churn) keeps animating, invalidating a detached view and can invoke onTabSelected/onParkInvoked and fireFeel after it has left the window.

• [HIGH] Hit-testing fabricates stops outside the recess and taps commit the DOWN stop, not the UP position
  - Location: app/src/main/java/com/example/ui/views/MonolithRailView.kt:64-68,317,341-355
  - Evidence: stopForX() (lines 64-68) never bounds-checks: 'if (xDp >= parkStartDp(...)) return PARKINDEX' and 'floor((xDp - INSETDP) / seg).toInt().coerceIn(0, 3)' — so taps in the 20dp left margin

(relX negative) select DECK and taps in/right of the right margin select PARK. Worse, ACTION_UP (lines 341-355) never recomputes x: it commits 'val stop = pressedStop' captured on DOWN, so press in one stop, slide 7dp (below the 8dp drag threshold, line ~330) into the next/PARK, release — the DOWN stop, not the released stop, is committed.

• [MEDIUM] onMeasure ignores MeasureSpec modes and geometry collapses to negative/zero segments on narrow widths
  - Location: app/src/main/java/com/example/ui/views/MonolithRailView.kt:38-45,161-164
  - Evidence: onMeasure (lines 161-164) is 'val w = MeasureSpec.getSize(widthMeasureSpec); setMeasuredDimension(w, dp(HEIGHTDP).toInt())' — mode of either spec ignored, no ATMOST/UNSPECIFIED or minimum-width handling. Geometry has no floor either: segmentDp (line 38) is '(recessWidthDp - 2*INSETDP - PARKZONE_DP)/4f' and capWidthDp (line 45) is 'segmentDp(...) - 8f', so below ~96dp view width the segment is <=0 and cap width negative, and stopForX divides by that zero/negative segment. Split-screen/multi-window/small-width hosts get a broken or NaN-positioned rail instead of a degraded/clamped one.

theme

Verdict: The 'strict 19-token Monolith system' is a facade: two half-wired schemes with library-default holes and a light token smuggled into the dark scheme, a dead XML palette that contradicts it, 87 lying aliases, type slots left to stock Material, and raw hex scattered across screens — nothing here is a single source of truth.

• [HIGH] Both ColorSchemes are half-wired: consumed slots (surfaceBright etc.) are never assigned, and the dark scheme's tertiary is a light-mode token
  - Location: app/src/main/java/com/example/ui/theme/Theme.kt:17-80
  - Evidence: Neither lightColorScheme (Theme.kt:17-40) nor darkColorScheme (Theme.kt:42-80) assigns surfaceBright, surfaceDim, surfaceContainer*, inverseSurface/inversePrimary, scrim or surfaceTint, yet scheme.surfaceBright is rendered at AirlockAndDisarmOverlays.kt:324, AirlockAndDisarmOverlays.kt:498 and ReceiptSheet.kt:78 — those raised surfaces take Material3 library defaults, not TokenColorSurfaceBright (#FAF6F0, Color.kt:15) / NightColorSurfaceBright (#362C27, Color.kt:32), which exist precisely for that role and are never wired into either scheme. Worse, the dark scheme sets tertiary = ElectricIris (Theme.kt:51), and ElectricIris is a LIGHT token: 'val ElectricIris = TokenColorSecondaryContainer' = #615B5A (Color.kt:119).

• [HIGH] Three divergent copies of the palette: the XML colors.xml palette is dead code and its night values contradict the Kotlin tokens
  - Location: app/src/main/res/values-night/colors.xml:7-23
  - Evidence: values-night/colors.xml still ships the palette Color.kt says was deliberately killed: colorSecondaryContainer=#FFFFB38F (line 16) vs Kotlin NightColorSecondaryContainer=#FF4A423C, with Color.kt:38 commenting the peach 'was peach #FFB38F, a second floodlight at night'; colorOnSurfaceVariant=#FFFAE4D3 (line 19) vs Kotlin #FF776A60, which Color.kt:41 says replaced #FAE4D3 because 'no mute existed'; colorBackground=#FF201D1A (line 7) vs Kotlin #FF171412; textColorPrimary=#FFF8F5F5 (line 23) vs Kotlin #FFD2C6BB. Even light values/colors.xml:19 has colorOnSurfaceVariant=#FF787170 vs Kotlin #FF5F5856. Nothing consumes the XML (zero '@color/' references in res and zero R.color usages in Kotlin), so it is a dead third copy free to rot — and it has.

• [HIGH] Hard-coded colours outside the theme bypass the token system — some untokenised, some

literal copies of tokens
  - Location: app/src/main/java/com/example/ui/screens/FocusSequencerScreen.kt:734-742
  - Evidence: 19 Color(0x...) literals exist outside ui/theme. Some are colours that exist nowhere in the 19-token palette: FocusSequencerScreen.kt:734-742 hard-codes six gradient/edge tones (#FF4E4847, #FF3B3735, #FF2D2927, #FFA8A39F, #FFBDB7B4, #FFCCC7C3). Others are hand-copied duplicates of tokens that will silently drift: MonolithCustomComponents.kt:315 returns Color(0xFF4A423C) (= NightColorSurfaceSheet) and :321 Color(0xFFFFE4CF)/Color(0xFFFBF1EC) instead of referencing the tokens; :560 and :562 copy NightTextColorSecondary as Color(0xFF9C8E84); ShieldCapSelector.kt:64 sets marker = Color(0xFFFBF1EC) (= TokenColorOnPrimary); TactileDeckScreen.kt:654 copies NightColorOnSurface as Color(0xFFEDE3D8). Plus raw Color.Black scrims at AirlockAndDisarmOverlays.kt:140/315/489 and ParkDrawer.kt:219, a colour in neither palette.

• [MEDIUM] Color.kt is half alias landfill: 87 aliases with lying names, a wrong RGB comment, and an admittedly dead token counted in the 19
  - Location: app/src/main/java/com/example/ui/theme/Color.kt:110-209
  - Evidence: After the 38 real tokens, Color.kt appends 87 alias vals (lines 110-209 of a 209-line file), most collapsing to the same 4-5 tokens, with names that lie about the colour: StarkWhite = TokenColorPrimary = #252323 near-black (Color.kt:170), BauhausYellow = TokenColorPrimaryContainer = orange #F55314 (:162), SolarYellow and BauhausCanary likewise orange (:185, :200), ElectricViolet and BauhausBlue = TokenColorSecondaryContainer = grey #615B5A (:175, :161), DeepIndigo = TokenBlockBarSurface = brown-black #2E2A26 (:176), GraphiteSubInk = NightColorSurfaceBright, a night token aliased for general use (:158). Header comments lie too: Color.kt:26 documents TokenColorOnSurfaceVariant as '(R:120, G:113, B:112)' but the value 0xFF5F5856 is (95, 88, 86). NightAlert is a dead token — Theme.kt:63-71 admits the dark error slot was re-voiced to the accent and 'nothing renders it', yet it remains in NightZeroGreenTokens (Color.kt:57, 106) inflating the '19 tokens' count.

• [MEDIUM] Typography has stock-Material holes and a false ladder claim; Theme.kt's API lies about dark/dynamic colour
  - Location: app/src/main/java/com/example/ui/theme/Type.kt:152-165
  - Evidence: Typography (Type.kt:152-165) assigns only 12 of the 15 Material3 slots — displaySmall, headlineSmall and titleSmall are absent (grep finds no use or definition anywhere), so any component using them falls back to stock Material type at off-ladder sizes in the default font, breaking the claimed system. The header comment (Type.kt:62-63) claims 'six sizes only - 104 / 72 / 50 / 22 / 15 / 10', but no 50.sp style exists in Type.kt; 50.sp appears only as an inline SpanStyle hack in TactileDeckScreen.kt:644-650. Slots are also semantically collapsed: bodyLarge = AppTitleTextStyle, a BOLD title style (Type.kt:159 with AppTitleTextStyle at :128-134), bodyMedium and bodySmall are the identical BodyTextStyle (:160-161), and labelLarge/Medium/Small are all the same PillBadgeTextStyle (:162-164). Additionally MyApplicationTheme ignores the OS entirely: darkTheme defaults to false (Theme.kt:103), isSystemInDarkTheme appears nowhere in the app, and the dynamicColor parameter (Theme.kt:104) is accepted and never read — MainActivity.kt:105 hard-codes it false.

resources

Verdict: The res/ tree is a Potemkin design system — strings, colours, dimens and themes are mostly dead, stale or bypassed while the real UI is hard-coded English and hex in Kotlin, with no localisation

whatsoever.

• [CRITICAL] strings.xml is decorative: the entire UI bypasses it with hard-coded literals
  - Location: app/src/main/res/values/strings.xml:1-25
  - Evidence: strings.xml holds only 22 strings; grep found 0 stringResource() calls and only 2 R.string uses in all Kotlin (MonolithShieldService.kt:203,206 for the notification channel). The 6 tab (strings.xml:7-12) and 10 ritual (strings.xml:15-24) strings have zero code references, while grep found 197 Text(calls and 70 text="..." literals, e.g. AirlockAndDisarmOverlays.kt:154 text = "NIGHTSTAND DESK CLOCK • TAP ANYWHERE TO EXIT" and :505 text = "SYSTEM GUARD PRE-FLIGHT".

• [HIGH] Zero localisation: no locale resources at all
  - Location: app/src/main/res/values/strings.xml:1-25
  - Evidence: res/ contains only values/ and values-night/ — ls app/src/main/res/values-* returns only values-night/colors.xml, no values-hi, values-es or any other locale, and no locales/locale-config or translatable attributes were found anywhere. Combined with the hard-coded literals above, not one UI sentence can be translated without editing Kotlin.

• [HIGH] colors.xml, dimens.xml and themes.xml are dead and diverged from the Compose truth
  - Location: app/src/main/res/values/colors.xml:7-25
  - Evidence: grep found 0 R.color and 0 R.dimen references in Kotlin. themes.xml is one line: <style name="Theme.MyApplication" parent="android:Theme.DeviceDefault.NoActionBar"/> with no colour/font attributes and no values-night/themes.xml. The real palette lives in ui/theme/Color.kt as Color(0x...) literals (57 Color(0x occurrences app-wide) and it has already diverged from the XML: values-night/colors.xml:7 colorBackground=#FF201D1A vs Color.kt:32 NightColorBackground=Color(0xFF171412); values-night/colors.xml:16 colorSecondaryContainer=#FFFFB38F vs Color.kt:41 NightColorSecondaryContainer=Color(0xFF4A423C) (comment admits the XML peach 'was' replaced); values/colors.xml:19 colorOnSurfaceVariant=#FF787170 (3.61:1 on #E4DFDA, below WCAG AA 4.5:1) vs Color.kt light token #FF5F5856. dimens.xml:11 countdowncountertext_size=148dp is likewise unused. Editing res/ changes nothing the user sees.

• [MEDIUM] Launcher icon hard-codes an off-palette colour set that contradicts the design tokens
  - Location: app/src/main/res/drawable/iclauncherforeground.xml:9-24
  - Evidence: Every fill is a raw hex literal, none referencing @color/*: #1E40FF (line 9), #FCF9F2 (13), #F5B800 (17), #E6332A (21), #141318 (24). None of these values exists in values/colors.xml or values-night/colors.xml, whose declared palette is warm stone #E4DFDA / accent #F55314 with a 'Strictly Zero-Green' law — the icon the user actually sees is blue/yellow/red Bauhaus that the design tokens disown. iclauncherbackground.xml:9,13 does the same with #F7F3EA/#EDE6D8.

• [MEDIUM] Fonts are 80% of res/ bloat: 8 bundled TTFs, Sora variable font shipped as 4 statics
  - Location: app/src/main/res/font/sora_bold.ttf:1
  - Evidence: du -cb app/src/main/res/font/ = 493,384 bytes, ~80% of the entire 612K res/ tree, for 8 static TTFs with no res/font/.xml family files. Sora alone is shipped as 4 separate files (sora.ttf 32,012B, soramedium 58,508B, sorasemibold 58,604B, sorabold 58,568B) wired manually in ui/theme/Type.kt:20-24 despite Type.kt:19 calling them 'true instances of the Sora variable font' — a variable font shipped as four statics. anonymouspro_bold.ttf is 153,616B, 4x its regular weight (38,884B). No downloadable-fonts or subsetting is used.

tests-1

Verdict: All three files compile cleanly against current main APIs (verified signature-by-signature: reconcile, groupedByRule, MonolithRepository sync/tier methods, ArmProvingController constructor, ArmSessionPort, SessionSealTransition) and many assertions are genuinely specific, but the suite's proudest guarantees are hollow — the seal ordering is proven against the test's own fake, two of three category-default branches are never asserted, and the empty-query test passes even if the whole sync silently dies.

• [HIGH] Seal-ordering 'proof' is circular: the test asserts against its own hand-written fake, never the production transition
  - Location: app/src/test/java/com/example/ArmProvingControllerTest.kt:93-127,266
  - Evidence: FakeTransition (test lines 93-127) hardcodes the order itself: 'events += "marker"; publish(); events += "sound"', and FakePort.sealSession emits the 'seal' event. The headline assertion at line 266, 'assertTrue(events.indexOf("seal") in 0 until events.indexOf("sound"))', therefore only proves the fake does what the fake was written to do. SessionSealTransitionImpl (service/SessionSealTransition.kt) is never instantiated in this file, so a production reorder of publish-before-sealRow would leave this suite fully green.

• [MEDIUM] Reconcile category-default branches LOCKED and OPEN are never asserted — the only SHORT_VIDEO app is preset-masked
  - Location: app/src/test/java/com/example/AppCatalogReconcileTest.kt:47-73,76-81
  - Evidence: Production reconcile (data/AppCatalogReconcile.kt:88-92) has three category branches: SHORTVIDEO/GAMES -> LOCKED, MESSAGING -> PEEK2M, else -> OPEN. The test's only video-labelled app is com.google.android.youtube, which is masked by its FACTORYPRESET (PEEK2M,30), and the 25 apps in reconcilediscoveryHasNoCap all fall through the fake categorizer to SHOPPING yet their tiers are never asserted — only the insert count. Swapping or deleting the LOCKED and OPEN category defaults would fail no assertion here; only the MESSAGING branch is actually pinned (Fresh Chat -> PEEK2M).

• [MEDIUM] Empty-query test cannot distinguish the guard working from the entire sync silently failing
  - Location: app/src/test/java/com/example/AppListTruthTest.kt:137-150
  - Evidence: The test registers zero ResolveInfos and asserts the stored row survives and no PruneNote exists. But production ensureSeededAndSyncPackages wraps the whole sync in 'catch (_: Exception)' and skips reconcile when 'installed.isNotEmpty()' is false (data/MonolithRepository.kt, sync block ~lines 230-272), so the identical green result occurs if queryIntentActivities throws, if loadLabel filters every row out, or if reconcile silently dies for any other reason. No variant makes the query actually throw. The test proves 'nothing happened', not 'the empty-result guard held'.

• [MEDIUM] ArmProvingControllerTest's two clocks and witnesses are entirely fake — production wiring is exercised nowhere
  - Location: app/src/test/java/com/example/ArmProvingControllerTest.kt:129-186,290-314
  - Evidence: Every time source is a test double: clock = { now } with manual 'now', framePoster queues into an ArrayDeque, and FakeWitnesses returns stamps decoupled from that clock — heldMarksFlushWithTheirWitnessStamps sets shieldMono = l+300 while now is still l+20..l+30 and asserts t0 = l+300. RepositoryArmSessionPort, LiveArmWitnesses, the Choreographer default framePoster and the SystemClock default clock (ui/ArmProvingController.kt:175-234,236-256) are never instantiated, so the tests prove arithmetic inside a puppet theatre, not that the real witnesses/clock/frame wiring produces those stamps.

• [LOW] groupedByRule count assertions are tautological padding in both test files

  - Location: app/src/test/java/com/example/AppCatalogReconcileTest.kt:115-130
  - Evidence: AppRules defines 'val peekCount: Int get() = peek.size' and 'val shutCount: Int get() = shut.size' (data/AppRules.kt:16-17). Both test files assert the group lists and then assert the counts equal the list sizes just asserted (AppCatalogReconcileTest.kt:128-129; AppListTruthTest.kt:196-197). These count assertions are definitionally incapable of failing independently — they pad the tests with assertions that carry zero information.

tests-2

Verdict: The repository/migration/pure-logic tests in this half are genuinely substantive, but the suite is polluted by a 917-line assertion-free screenshot factory and a zero-assertion render test that write to hardcoded paths on the author's machine, one order-dependent test that swaps its own assertions based on leftover singleton-DB state, a self-pinned 'golden' hash, and an instrumented suite that is still the IDE template - so a green run proves far less than it claims. (Compile status checked statically only, per read-only rules: all 60 com.example imports in this slice resolve to current main-source declarations, including DB v12 migrations; no Gradle run was performed.)

• [CRITICAL] ScreenshotHarnessTest is a screenshot factory masquerading as a test suite: most tests assert nothing and the one real check can pass vacuously
  - Location: app/src/test/java/com/example/ScreenshotHarnessTest.kt:98
  - Evidence: 917 lines, 15 @Test methods but only 11 lines containing 'assert' in the whole file (2 are imports); captureAllDestinations (line 182), parkSheetDay (358), wheelMidDrag (378) etc. contain no assertions at all - they pass if nothing crashes while writing a PNG. Default output is a hardcoded path on the author's machine: line 98 'File(target ?: "/home/hatch/workspace/clone-inspect/shots")'. settle() line 130 swallows failure: 'catch (_: Throwable) { // ... capture anyway }'. Worst, zzAccentCensus silently skips missing inputs - line 848 'if (f.isFile) cases += Case(...)' and lines 867/905 'if (!f.isFile) continue' - so it can pass vacuously or against stale PNGs left by a previous run, and it depends on the 'zz' name hack for JUnit ordering.

• [HIGH] TetradRenderTest has literally zero assertions and writes PNGs to a hardcoded absolute path on the developer's machine
  - Location: app/src/test/java/com/example/TetradRenderTest.kt:44
  - Evidence: Zero assertions in the entire file (grep: 1 @Test, 0 'assert'). The single test tetradClosures() just renders RotaryWheelView 5 times; the header admits the output is 'judged in a picture' by a human. Line 44 hardcodes the author's machine: 'val dir = File("/home/hatch/workspace/clone-inspect/shots/tetrad")' - on any other machine/CI this writes outside the repo or fails for purely environmental reasons, and inside the suite it can never go red on a visual regression.

• [HIGH] The only instrumented test in the project is the untouched IDE template asserting the package name
  - Location: app/src/androidTest/java/com/example/ExampleInstrumentedTest.kt:20
  - Evidence: The entire androidTest source set is this one 22-line file, the untouched Android Studio template: useAppContext() only does 'assertEquals(BuildConfig.APPLICATION_ID, appContext.packageName)' (line 20). This is an app whose whole reason to exist is an accessibility service, a notification listener, device-admin and usage-access flows - none of it has a single on-device test; everything is Robolectric shadows pretending to be the platform.

• [HIGH] TokenBreakRoutineSeamTest is order-dependent against a shared singleton database and

degrades its own assertions based on leftover state
  - Location: app/src/test/java/com/example/TokenBreakRoutineSeamTest.kt:163
  - Evidence: Both tests run against the production singleton: line 45/142 'MonolithDatabase.getInstance(context)', with raw-SQL cleanup as the isolation strategy. The header admits 'the token suite leaves its own spend behind' and line 59 'class order inside a run is not a law (the amended run's order exposed exactly this...)'. The second test is openly non-deterministic: line 139 'NOTE: this test is Refused-safe' and line 163 'if (spendsBefore == 0) { assertEquals(BreakResult.Completed, result); ... } else { assertEquals(BreakResult.RefusedNoToken, result) }' - leftover state from another class silently swaps which contract is actually verified.

• [MEDIUM] SealThrowIdentityTest's 'golden' hash is self-pinned from the code's own first output - a tautology dressed as proof
  - Location: app/src/test/java/com/example/SealThrowIdentityTest.kt:54
  - Evidence: The header (lines 16-19) states the method plainly: 'The GOLDENSHA256 below is pinned from the first green run's own output... the golden is defined by the render the spec produces, then pinned'. goldenHashPinnedFromFirstGreenRun (line 54) asserts the render hash equals TactileSoundEngine.SEALBUFFERGOLDENSHA256 (line 65) - a constant generated by the very code under test. It detects drift but is incapable of detecting that the original render was wrong; presented as 'the Bit-Identical Seal, proven', it proves only self-consistency.

architecture

Verdict: Architecture is a circular, DI-free tangle of god classes under the template com.example package — data, service and UI layers depend on each other in every direction, so nothing can be changed, tested or reused in isolation.

• [CRITICAL] God classes dominate: Repository (2932 lines), ViewModel (2548), BlockitCoreViews (2416) are unmaintainable monoliths
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:90
  - Evidence: 2932 lines, ~106 functions, single class at line 90 'class MonolithRepository(private val context: Context, private val dao: MonolithDao)'; MonolithViewModel.kt is 2548 lines with class at line 203 and ~113 StateFlow references; BlockitCoreViews.kt is 2416 lines. wc -l total main = 24187 across only 49 files.

• [CRITICAL] Circular data<->service coupling: data layer imports and drives services, services construct the repository
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:15
  - Evidence: MonolithRepository.kt:15-23 imports 'com.example.service.MonolithAccessibilityService, MonolithDeviceAdminReceiver, MonolithShieldService, RoutineScheduler, TactileSoundEngine, UsageAccess, Warrant, WatchStore' and calls MonolithShieldService.protecting:633, startShield:1099, RoutineScheduler.rearmChain:913; reverse: MonolithShieldService.kt:20 imports MonolithRepository and :81 'val repository = MonolithRepository(applicationContext, dao)', same in RoutineHeartbeat.kt:92 and SessionSealTransition.kt:317.

• [HIGH] No DI or single source of truth: manual getInstance scattered, services bypass repository for direct DAO access
  - Location: app/src/main/java/com/example/MainActivity.kt:83
  - Evidence: No Hilt/Koin/Application class found; MainActivity.kt:83-84 'val database = MonolithDatabase.getInstance(applicationContext); val repository =

MonolithRepository(applicationContext, database.monolithDao())' repeated in ShieldService:81, RoutineHeartbeat:92, SessionSealTransition:317. Services bypass repository entirely via direct DAO: MonolithAccessibilityService.kt:134,193, MonolithBootReceiver.kt:36, MonolithNotificationListener.kt:71, ReleaseRetryScheduler.kt:72 'MonolithDatabase.getInstance(...).monolithDao()'. Repository itself re-fetches DB at 1382,1423,1482,2226 despite injected dao.

• [HIGH] Template package com.example and no domain layer: flat 3-bucket structure, service package misused as junk drawer
  - Location: app/build.gradle.kts:12
  - Evidence: build.gradle.kts:12 'namespace = "com.example"' vs :16 'applicationId = "com.aistudio.monolith.kxtpzr"'; all 49 main files are only com.example, com.example.data (15), com.example.service (13), com.example.ui.* — no domain/usecase package. service/ is a junk drawer: WatchStore.kt:52-147 holds WarrantState, UsageAccess, Warrant, WatchSnapshot domain models, TactileSoundEngine.kt:41 is an audio engine, SoundEffectsSetting.kt:34 is a SharedPreferences object — none are Android Services.

• [HIGH] Layering inverted everywhere: repository does Android-framework/UI work, ViewModel holds Context+sound engine, services depend on MainActivity
  - Location: app/src/main/java/com/example/data/MonolithRepository.kt:1831
  - Evidence: MonolithRepository holds Context, ~80 references to AppOpsManager/DevicePolicyManager/UsageStatsManager/AudioManager/PowerManager/Settings and getSharedPreferences at 189,455,464,500, plus lazy TactileSoundEngine at 1831 — permission checks, prefs, scheduling, sound and DB in one data class. ViewModel at MonolithViewModel.kt:203-206 holds 'appContext: Context, repository, val soundEngine: TactileSoundEngine'. Services import UI root: MonolithAccessibilityService.kt:10 and MonolithShieldService.kt:16 'import com.example.MainActivity'. MainActivity.kt:155-821 contains the entire MonolithRootApp composable/navigation on top of onCreate DI and intercept-intent handling at 126.

security-privacy

Verdict: Security/privacy is a high-privilege surveillance stack (accessibility content, notification bodies, usage stats, overlay, device admin) guarding data with forgeable crypto and plaintext pre-unlock storage, wrapped in disclosures that understate what it does — the one honest part is no INTERNET permission/backup-off, and exact alarms are not actually used (only setAndAllowWhileIdle).

• [CRITICAL] Other apps' private notification title+body persisted indefinitely in unencrypted, device-protected (pre-unlock) Room DB
  - Location: app/src/main/java/com/example/service/MonolithNotificationListener.kt:66-90
  - Evidence: Listener extracts Notification.EXTRATITLE/EXTRATEXT (lines 66-68) and inserts both into Room as CapturedNotificationEntity with senderTitle/messageBody (MonolithModels.kt:462-472). The DB is built with plain Room.databaseBuilder (MonolithDatabase.kt:237) on createDeviceProtectedStorageContext() (line 233) — no SQLCipher/encryption, available before first unlock — and the DAO has no TTL/auto-delete, only manual delete/clear (MonolithDao.kt:164-168). Private message content from blocked apps therefore sits indefinitely, in plaintext, in pre-unlock storage.

• [CRITICAL] HMAC 'hardware integrity' is forgeable theatre: public hardcoded fallback key and

blank-signature bypass
  - Location: app/src/main/java/com/example/data/CryptoIntegrityManager.kt:27-30,84
  - Evidence: Hardcoded fallback key shipped in the APK: SecretKeySpec("MonolithHardwareHMACFallback32B!"...) at lines 27-30, used whenever KeyStore init fails (lines 60-75, catch-all fallback), and verifySessionState() short-circuits: 'if (session.hmacSignature.isBlank()) return true' (line 84). Anyone who extracts the APK can forge signatures with the public key, and anyone who can edit the DB can simply blank the signature to pass — the claimed hardware tamper/penalty lock detects neither.

• [HIGH] Accessibility service silently reads all window content and browser URLs while its Play-facing description claims it only detects app launches
  - Location: app/src/main/res/xml/accessibilityserviceconfig.xml:6
  - Evidence: Config requests canRetrieveWindowContent=true with flagIncludeNotImportantViews|flagReportViewIds (accessibilityserviceconfig.xml:2-7); the service recursively dumps all window text/contentDescription (collectWindowTextLowercase, MonolithAccessibilityService.kt) and reads URL-bar nodes in Chrome/Brave/Firefox/Edge/Samsung Internet and Settings screens. Yet the disclosed description (strings.xml:4) says only it 'Detects when a blocked app is launched... and presents the Intention Airlock screen' — it never mentions reading page content, URLs or Settings text. Using the Accessibility API for non-disability app-blocking plus an understated description is a textbook Play AccessibilityAPI declaration/rejection risk, and no in-app prominent disclosure/privacy policy was found in app/src/main.

• [HIGH] Device Admin abused as an uninstall guard, with an irrelevant password policy and a mislabelled description
  - Location: app/src/main/res/xml/deviceadminreceiver.xml:4
  - Evidence: Manifest receiver MonolithDeviceAdminReceiver is labelled 'Monolith Uninstall Guard' / 'Kernel-Level Uninstall Guard' whose sole job is blocking deactivation mid-session (MonolithDeviceAdminReceiver.kt:8-15, onDisableRequested returns a warning nag), yet deviceadminreceiver.xml:3-5 declares the unrelated <limit-password/> policy, and the manifest (AndroidManifest.xml:123) even sets its description to @string/accessibilityservicedescription. Device-admin is deprecated/restricted by Play to narrow enterprise/device-owner use cases; a consumer focus app using it as an uninstall friction-gimmick, with a bogus policy and a copy-pasted wrong description, is a policy and review liability — and 'kernel-level' is simply false.

• [MEDIUM] Exported MainActivity blindly trusts intercept extras from any other app
  - Location: app/src/main/java/com/example/MainActivity.kt:126-146
  - Evidence: AndroidManifest.xml declares MainActivity android:exported="true" with only a LAUNCHER filter; MainActivity.kt:126-146 handleIncomingInterceptIntent() reads EXTRAINTERCEPTEDPACKAGE/APPNAME/BADGE/PATH (plain string constants, MonolithShieldService.kt:284-291) from any incoming intent with no caller/signature check and calls viewModel.triggerAirlockIntercept(pkg, name, badge, path). Any installed app can therefore launch Monolith with fabricated extras, spoof an intercept/airlock for an arbitrary package/badge, and supply path=APPTIER/SHIELD_APP — the very paths InterceptPath.canHonourPeekLease treats as peek-lease-eligible.

performance-battery

Verdict: Performance/battery is reckless: a 2s foreground-service loop doing full DB writes and UsageStats scans, a duplicate 1s ViewModel ticker writing the same row, and an unthrottled

accessibility pipeline spawning DB-heavy coroutines per event will drain battery and contend on Room throughout every session - and much of it runs even with no session at all.

• [CRITICAL] Foreground shield service polls every 2s with full DB writes, two full app-table scans, UsageStats query and notification rebuild
  - Location: app/src/main/java/com/example/service/MonolithShieldService.kt:88-193
  - Evidence: MonolithShieldService.kt:88-193: while(isActive){ dao.getActiveSessionSnapshot(); repository.updateSessionProgress(activeSession, stampShieldBeat=true) at :108; dao.getAllAppsSnapshot() at :165 AND again at :180; repository.queryForegroundBlockedApp() at :178; updateNotification() at :142; delay(2000L) at :193 }. updateSessionProgress does dao.updateSession() and accrueActiveTaskSeconds() which loads getAllTasksSnapshot() (MonolithRepository.kt:1968,2424-2435); queryForegroundBlockedApp does UsageStatsManager.queryUsageStats (MonolithRepository.kt:564-580). No backoff when idle/foreground app unchanged - same work every 2s for a multi-hour session, plus a NotificationManager.notify() rebuild every iteration.

• [CRITICAL] Accessibility service spawns an unbounded coroutine with 4+ DB reads and recursive window-tree walks for every accessibility event
  - Location: app/src/main/java/com/example/service/MonolithAccessibilityService.kt:164-273
  - Evidence: accessibilityserviceconfig.xml:3-8 requests typeWindowStateChanged|typeWindowContentChanged, flagIncludeNotImportantViews, notificationTimeout="80". MonolithAccessibilityService.kt:164-192: on every event it synchronously touches rootInActiveWindow, then scope.launch{...} with no throttle, Job tracking or cancellation; each launch does getActiveSessionSnapshot() :194, getGuardSettingsSnapshot() :203, getAllAppsSnapshot() twice :225 and :273, getAllSitesSnapshot() :255, plus recursive tree walks collectWindowTextLowercase() :405-411 (depth 14) and isSelectedBottomTabShortsOrReels() recursion. The launch happens before the no-session check inside the coroutine, so this fires 24/7 even with no active session; content-changed events fire on scrolling/typing/animation.

• [HIGH] Two concurrent tickers (ViewModel 1s + shield 2s) both persist session progress - duplicate DB writes every second
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:945-1020
  - Evidence: MonolithViewModel.kt:945-1020 startTickerLoop(): while(isActive){ repository.latchShieldGapIfStale() :972; repository.updateSessionProgress(current) :973; delay(1000L) :1020 } runs concurrently with the shield's identical 2s updateSessionProgress call. Comment in MonolithRepository.kt:1986-1990 admits 'two tickers drive this function'. Each call re-queries boot count (Repository:1939), verifies HMAC elapsed, runs accrueActiveTaskSeconds() (full tasks snapshot + possible updateTask) and dao.updateSession() - i.e. ~1.5 Room session writes/sec sustained, with lock contention between the two writers, for zero extra accuracy.

• [HIGH] Startup does the full installed-app PackageManager sync twice, and PARK-open repeats it
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:767-781
  - Evidence: MonolithViewModel.kt:767-781 init launches ensureSeededAndSyncPackages() then refreshSystemGuard(); refreshSystemGuard() at :2449-2453 itself calls ensureSeededAndSyncPackages() again. That function (MonolithRepository.kt:193,220-260) does pm.queryIntentActivities(ACTION_MAIN/CATEGORY_LAUNCHER), loadLabel(pm) for every launcher app, queryRealTodayUsageMinutes() (UsageStats query), getAllAppsSnapshot() reconcile and a dao.updateApp() per usage update. MainActivity.kt LaunchedEffect(parkOpen) re-calls

refreshSystemGuard() on every PARK drawer open, so opening a UI drawer re-scans/reconciles the entire installed-app catalog.

• [MEDIUM] Perpetual 1s/700ms UI polling loops run with no session and no screen need
  - Location: app/src/main/java/com/example/ui/MonolithViewModel.kt:945-1020
  - Evidence: MonolithViewModel.kt:945-1020: ticker unconditionally sets _themeNow.value=System.currentTimeMillis() every 1s even when activeSession is null, forcing collectors/recomposition for the VM's whole life. ShieldMatrixScreen.kt:167-169: LaunchedEffect(Unit){ while(true){ delay(700); ... } } runs even when restingFlips is empty. TactileDeckScreen.kt:354-358 adds another perpetual while(true) 60s-aligned loop. No loop pauses on lifecycle STOPPED/background - viewModelScope keeps ticking while the activity exists.

usability-accessibility

Verdict: Usability is a hostile obstacle course — six unexplained special-access permissions doled out one silent Settings trip at a time, wrapped in ritual jargon, and the Canvas-drawn UI offers literally zero accessibility semantics, so the users who most need assistive tech cannot operate the app at all.

• [CRITICAL] Screen-reader users are completely locked out: zero accessibility semantics in the entire UI
  - Location: app/src/main/java/com/example/ui/views/MonolithRailView.kt:98,204,308; app/src/main/java/com/example/ui/views/BlockitCoreViews.kt:371,987
  - Evidence: grep for contentDescription/semantics across app/src/main/java returns ZERO hits in any UI file — the only matches are MonolithAccessibilityService.kt:352,409 reading OTHER apps' nodes. The primary navigation rail (MonolithRailView.kt:98,204,308), the rotary time dial (BlockitCoreViews.kt:371 RotaryWheelView, onTouchEvent at :987) and the switches/faders are raw Canvas Views driven by onDraw + onTouchEvent, annotated @SuppressLint("ClickableViewAccessibility"), with no performClick override, no AccessibilityNodeInfo, no contentDescription anywhere. A TalkBack user gets an unlabelled blob for navigation, time-setting and arming — in an app that itself demands an accessibility service.

• [CRITICAL] Six special-access permissions, each a separate Settings scavenger trip, stand between install and any value
  - Location: app/src/main/java/com/example/service/WatchStore.kt:80-102; app/src/main/AndroidManifest.xml:5-13; app/src/main/java/com/example/MainActivity.kt:362-410
  - Evidence: WatchStore.kt:93-96 defines PERMISSIONS as six separate special-access grants — ACCESSIBILITY, USAGEACCESS, DEVICEADMIN, NOTIFICATIONLISTENER, OVERLAY, BATTERY — on top of manifest permissions (POSTNOTIFICATIONS, SYSTEMALERTWINDOW, PACKAGEUSAGESTATS, FOREGROUNDSERVICESPECIAL_USE in AndroidManifest.xml). Each grant is a separate manual trip into a different Android Settings pane (MainActivity.kt openFixPane, ~362-410, including a hardcoded com.android.settings.DeviceAdminSettings component guess with fallback). Device Admin ('Uninstall Guard') and battery-optimization exemption are demanded as part of the same stack a first-time user must clear before blocking is 'LIVE' (WatchStore.deriveWarrant: OFF unless accessibility bound AND usage ALLOWED).

• [HIGH] No onboarding or explanation exists — the permission primer was deliberately deleted, first run dumps a jargon board
  - Location: app/src/main/java/com/example/MainActivity.kt:283-287,787-791; app/src/main/java/com/example/ui/components/ParkDrawer.kt:305-355;

app/src/main/res/values/strings.xml:16-25
  - Evidence: MainActivity.kt:787-791 comment states 'The ShieldPermissionPrimerModal binding stays REMOVED'. Instead, first run merely auto-opens the PARK drawer (MainActivity.kt:283-287, parkIntroPending) — a board of nine bare codenames ('ACCESSIBILITY / USAGE ACCESS / DEVICE ADMIN... LAST PROVED · SELF-REPORT', ParkDrawer.kt:305-355) with literally zero explanatory copy: grep for why/because/'in order to' in ParkDrawer.kt returns nothing. Strings double down on ritual jargon (strings.xml: 'SLIDE TO SEAL VAULT', 'ONE-WAY RATCHET ENGAGED', 'INTENTION AIRLOCK CHAMBER'). A new user is shown verdicts, never reasons.

• [HIGH] Denied/failed permission recovery is a serial one-at-a-time loop with silent failure
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:175-179,195-215,416-462
  - Evidence: ParkDrawer.kt:175-179 computes firstMissing = PERMISSIONS.firstOrNull { MISSING } and the single FIX key (ParkDrawer.kt:416-462) names only that one item, so a fully ungranted user must make up to six sequential Settings round-trips, re-opening PARK each time. On return, success is deliberately silent: ParkDrawer.kt:202-205 — 'The repaired line alone gets the firm click; the hand goes back in silence — no word prints, ever.' If the user toggled the wrong thing in Settings, they get no error, no instruction, just the same key again.

• [MEDIUM] Touch targets under the 48dp minimum and 10sp labels at 3.5:1 contrast in night mode
  - Location: app/src/main/java/com/example/ui/components/ParkDrawer.kt:513-514; app/src/main/java/com/example/ui/theme/Type.kt:85-92; app/src/main/java/com/example/ui/theme/Color.kt:44
  - Evidence: Every sled switch in PARK is .width(76.dp).height(34.dp) (ParkDrawer.kt:513-514, 582-583, 751-752) — 34dp tall, below Android's 48dp minimum touch target — and the Deck's FLOW key is 11sp text with 5dp vertical padding (TactileDeckScreen.kt:~715-735, ~31dp tall). Meanwhile PillBadgeTextStyle (Type.kt:85-92) sets virtually all micro-labels at 10sp, and in night mode those labels render in onSurfaceVariant #776A60 on #171412 — Color.kt:44 itself documents this as '3.5:1', below WCAG AA's 4.5:1 for text, at a size already too small to read comfortably.

docs-vs-code

Verdict: The documentation is a parallel fantasy app: 149k lines of plans, audits and 'VERIFIED' matrices describing files, fonts, screens and features that do not exist in the 24k lines of code, with its flagship audits run on an external tree and its 'settled shipped' facts already stale against this commit.

• [CRITICAL] .aistudio satisfaction matrix stamps VERIFIED/PASSED on files, fonts and ViewModel APIs that do not exist
  - Location: .aistudio/artifacts/brain/126e1213-db14-4552-8d59-57595258f3a4/completerequirementsandsatisfactionmatrix.md:46-90
  - Evidence: REQ-01 (line 46) claims '15 dedicated android.view.View subclasses' in files HeroTimerCardView.kt, HapticPresetStripView.kt, BagTilesSuitcaseView.kt, RombiKnobView.kt, TileNumberCounterView.kt, DayFadersView.kt, FallingSquaresView.kt, CurioCollectionGridView.kt, MonolithBottomNavView.kt, MonolithTheme.kt and MonolithHaptics.kt (REQ-06, line 51) — none of these files/classes exist; the whole app has only 9 View classes consolidated in 4 Blockit*.kt files + MonolithRailView.kt. REQ-03 (line 48) and checklist #3 (line 67) mark 'Plus Jakarta Sans renders all words; JetBrains Mono renders all numerals' VERIFIED/PASSED, while the real Type.kt:57-59 states 'The old lying font aliases (SpaceGrotesk / PlusJakartaSans / JetBrainsMono ...) is dead' — the shipped fonts are Sora/Atkinson/Anonymous Pro (res/font/sora.ttf etc.). Part IV (lines 85-90)

inventories ViewModel state/methods setFrictionTier, selectCurio, applySessionStamp, exportEncryptedStateJson, selectSoundscape, setMixerChannelLevels, playHarmonicEasterEggChord, grantTwoMinutePeekPass, showDeskClockOverlay — grep count in MonolithViewModel.kt is 0 for every one.

• [CRITICAL] Feature catalog and REQ-12 present an entire deleted feature universe (Curio Vault, Stamp Studio, 432Hz synth, mixer) as shipped
  - Location: .aistudio/artifacts/brain/126e1213-db14-4552-8d59-57595258f3a4/monolithcompletefeature_catalog.md:133,176-200
  - Evidence: REQ-12 (line 57) marks '100% Preservation' VERIFIED for '6 Curios + FallingSquaresView, 432Hz Chord Synth, Stamp Press Studio, Always-On Desk Clock, RombiKnob smart loadouts'; grep for CurioVaultScreen/StampPressStudioModal/FallingSquares across app/src returns 0 hits. The companion catalog (monolithcompletefeature_catalog.md:176-200) documents a whole 'Tab 4: Curio Vault & Telemetry (CurioVaultScreen.kt)', a StampPressStudioModal with 5 ink pads and 6 motifs, and (line 133) a '3-channel balance mixer (BROWN NOISE, WARM RAIN, 432Hz DRONE)' — a mixer Sarthak explicitly ordered removed in the matrix's own Part I (Prompt 4, Q3: 'Remove the sound mixer all together'). The app has exactly 4 screens: TactileDeckScreen, ShieldMatrixScreen, FocusSequencerScreen, DrawerScreen.

• [HIGH] MASTER-PLAN's 'SETTLED — SHIPPED' facts are already false against this trunk (DB v8 vs v12, frozen radii, phantom APK)
  - Location: studio/plan/MASTER-PLAN.md:60-61,146,173; studio/plan/CHROME-BUILD-READY.md:12-15
  - Evidence: MASTER-PLAN.md:173 declares '[SETTLED — SHIPPED Oct 29: DB v8, MIGRATION78 registered, the destructive fallback deleted, the token ledger-derived in trunk]' but MonolithDatabase.kt:25 is version = 12 with migrations through MIGRATION1112. MASTER-PLAN.md:146 freezes 'Radius 8dp keys / 16dp panels' while BlockitCoreViews.kt:122-123 hardcodes radiusSmall = dp(12f), radiusLarge = dp(24f) — the radii the .aistudio matrix (not the master plan) describes. MASTER-PLAN.md:60-61 claims the 60s hold is 'Built and shipped — monolith-banked.apk'; find for *.apk in the repo returns nothing. CHROME-BUILD-READY.md states it is written against 'DB v9, sha a414972de18bc213, suite 112/112 green' — a tree that is not this commit (8438f5a, DB v12), so all its file/line citations are to a ghost trunk.

• [HIGH] Documentation outweighs code ~6:1 and the flagship audits were performed on a different, external tree with evidence outside the repo
  - Location: studio/plan/LIE-REGISTER-3.md:5-8; studio/plan/OPEN-LOOPS.md:9-17; shots-redirect.txt:1
  - Evidence: studio/ + .aistudio contain 385 Markdown files totalling 149,078 lines (studio alone 12MB) versus 24,187 lines of production Kotlin in app/src/main — six lines of planning prose per line of code. Worse, the audits were not run on this repo: LIE-REGISTER-3.md:5-8 states its ground was '~/workspace/clone-inspect/Focus-app/ as Ren's hand merge left it', 31 files under studio/plan reference clone-inspect, and OPEN-LOOPS.md's method depends on mail/ren/, BUILD-CHANGES.md and COMMONS.md — none of which exist in this repo. FINISH-LIST-1.md cites as its evidence 'shots/01deck.png … 05guard.png, shots/night/n01–n05', but there is no shots/ directory in the repo; shots-redirect.txt points outside it to ~/workspace/clone-inspect/builds/truth-3-sound/shots-own. The repo's own audit evidence is unverifiable from the repo.

• [MEDIUM] Finish/audit docs grade LEDGER and GUARD screens that the master plan abolished and the code never shipped
  - Location: studio/plan/FINISH-LIST-1.md:18-45; studio/plan/MASTER-PLAN.md:131-142
  - Evidence: FINISH-LIST-1.md devotes its worst finding (F-01) and F-03/F-08/F-09 to the 'LEDGER' screen (tower, faders, '150 numeral', '16 BRICKS') and F-10 to the 'GUARD' screen's eight REMOVE billboards, graded from renders. But MASTER-PLAN.md itself records 'Guard → DISSOLVED' (no fifth stop; repair board moves to PARK) and 'the kept tab opens a storage drawer, not a report card... No scores, streaks, grades' — and the shipped code has DrawerScreen.kt and no Ledger or Guard screen at all. One planning corpus grades, in builder-number detail, screens another planning corpus abolished; neither is reconciled with the 4 screens that exist.

release-issue-map

Verdict: Not shippable: v1 has no green CI build, no signed release build, no version bump, tests that do not compile on main, and 22/22 open issues where process documentation (58k lines in studio/plan) dwarfs the code it claims to be releasing.

• [CRITICAL] CI has never built anything: 5/5 main runs failed, HEAD included, blocked by a gitignored keystore the build demands
  - Location: app/build.gradle.kts:36; .gitignore:18; .github/workflows/rescue-build.yml:22
  - Evidence: GitHub Actions API: all 5 push runs on main failed — #1 b792727, #2 b981ca1, #3 6d2eebe, #4 817e8c7, #5 8438f5a (run 40958472117) 'completed/failure'; job steps for #5: 'Install SDK platform 36.1 success, Assemble debug failure, upload-artifact skipped, Unit tests skipped'. Root cause in issue #3 comments: ':app:validateSigningDebug: Keystore file .../debug.keystore not found'. Code: app/build.gradle.kts:36-38 debugConfig storeFile=file("${rootDir}/debug.keystore"); .gitignore:18 lists 'debug.keystore' and the file is absent in the clone. No fix on main at 8438f5a.

• [CRITICAL] The sole 'release' is a manually-built debug APK mislabelled as a full release; no releasable build exists
  - Location: app/build.gradle.kts:19-20,27-38; .github/workflows/rescue-build.yml:22
  - Evidence: Releases API: exactly 1 release, tag 'batch-9' (fff9edd), name 'Batch 9 — Truth Car 3 (test build, Oct 30)', prerelease=false despite its own body saying 'Debug/test build' and 'The unit tests do not compile yet'. app/build.gradle.kts:19-20 versionCode=1, versionName="1.0" never bumped; :27-32 release signing needs ${rootDir}/my-upload-key.jks (absent) plus STOREPASSWORD/KEYPASSWORD env; workflow rescue-build.yml:22 runs only './gradlew assembleDebug', never assembleRelease/bundle. Download count for monolith-batch9.apk: 0.

• [HIGH] Issue #2 is real and still broken on main — the fix sits unmerged on side branches while main's tests do not compile
  - Location: app/src/test/java/com/example/TruthCar3SweepTest.kt:6; app/build.gradle.kts (main 8438f5a vs tyler/safe-fixes e5eec1a)
  - Evidence: Issue #2 body: ScreenshotHarnessTest.kt:350 missing onToggleSocialSites, ShortsToggleRenderTest.kt:66, SoundAndSocialTruthTest.kt:382, TruthCar3SweepTest.kt:314 and :6-7 unresolved assertExists/assertDoesNotExist, TruthCar3CaptureBodyTest.kt:70 no candidate — all still present on main. The fix exists only off-main: local branch tyler/safe-fixes e5eec1a 'test: fix batch-9 ParkDrawer call-sites and CaptureBody SBN constructor (issue #2)' (5 test files, +23/-7) and remote agent/issue-2 8a256a4; origin has only main at 8438f5a. CI never reaches testDebugUnitTest anyway because assembleDebug fails first, so the failure is doubly masked.

• [HIGH] Issues #1-#22 are process theatre layered over real work: 0 closed, several non-work by design, comment bloat on #3
  - Location: GitHub issues #1,#3,#18,#19,#20,#21,#22 (all state=open)
  - Evidence: Issues API: 22 open, 0 closed; #1-#22 all created in a ~5-minute burst 2026-10-29T21:17-21:22Z. #1 is a charter, #22 is an index of the others, #18 is 'frozen...record only', #19/#20 are single unanswered questions, #21 is 'do not work until Sarthak rules' (labels decision/blocked). Meanwhile #3 alone has 19 comments — verdict templates, pre-written closing templates, interruption ladders, 15-person cells — for a one-line keystore fix that still has not landed on main; #12 has 15 comments. Volume of process exceeds shipped fixes.

• [HIGH] Batch-10 (#15) and passes #6-#16 are unshippable epics stacked on unmerged local lanes and a DB migration that does not exist
  - Location: app/src/main/java/com/example/data/MonolithDatabase.kt:25; studio/plan/; GitHub issue #15
  - Evidence: git log main: only 8 commits; HEAD 8438f5a adds no app code — it migrates studio/ plan records. studio/ is 12M, 431 files, studio/plan/.md alone 58,374 lines vs 49 Kotlin files in app/src/main (2.2M). Issue #15 (Batch 10 epic) requires 'One composed MIGRATION1213' plus 8+ lane merges (Frist, Voss on DB v10 base, Wecker, Wache, Stempel, Pastille, Empfang), but MonolithDatabase.kt:25 is still version=12 with no MIGRATION1213, and the lanes live only as remote agent/issue- branches (issue-2,3,6,8,9,13,15,16,17) or 'DECLARED locally' per #14/#16 — declared is not merged, verified, or built.